

Studienprojekt I

**Konzeption und Entwicklung eines Zweikanal-Funktionsgenerators
auf Basis eines Waveshare ESP32-S3 2,8-Zoll-Touch-LCD-Displays**

Name, Vorname:	Klose, Alexander Linus
Matrikelnummer:	77201148558
Name, Vorname:	Dimitrijević, Dejan
Matrikelnummer:	77203661440
Jahrgang:	2024
Hochschule – Fachbereich:	HWR Berlin – Duales Studium - Technik
Fachrichtung:	Elektrotechnik
Modul:	IE-M26-6045 Studienprojekt I
Betreuer Hochschule:	Prof. Dr. Luís Fernando Ferreira Furtado
Unternehmen:	Berliner Wasserbetriebe AöR
Betreuer Unternehmen:	Dipl.-Ing. (FH) Thomas Freitag

Zur Kenntnis genommen:

20.07.2026 i.A.

Datum/Unterschrift Unternehmensbetreuer

13.07.2026

Datum/Unterschrift Studierender

13.07.2026

Datum/Unterschrift Studierender

Ehrenwörtliche Erklärung

Wir erklären ehrenwörtlich:

1. dass wir unser Studienprojekt selbständig verfasst haben,
2. dass wir die Übernahme wörtlicher Zitate aus der Literatur sowie die Verwendung der Gedanken anderer an den entsprechenden Stellen innerhalb der Arbeit gekennzeichnet haben und
3. dass wir unser Studienprojekt bei keiner anderen Prüfung vorgelegt haben.

Wir sind uns bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

13.07.2026



Datum/Unterschrift Studierender

13.07.2026



Datum/Unterschrift Studierender

Kurzfassung

Die Hochschule für Wirtschaft und Recht Berlin (HWR Berlin) ist eine der großen Hochschulen Berlins und bietet mit einem Portfolio von 57 Studiengängen eine große fachliche Bandbreite. Mit dem Studiengang der industriellen Elektrotechnik bietet die HWR Studierenden die Möglichkeit, eine der ältesten Disziplinen im Bereich der Ingenieurwissenschaften zu erlernen.

Ein essenzielles Themengebiet der Elektrotechnik stellt die Messtechnik dar. Im Studium werden die Inhalte sowie der Umgang mit Messgeräten frühzeitig vermittelt. Das vermittelte Wissen wird gegen Ende des Semesters durch eine Laborprüfung überprüft. Im Rahmen der Überarbeitung des Moduls Messtechnik I (ET2013) soll der aktuell verwendete Funktionsgenerator (HAMEG HM 8030-4) zukünftig durch einen Zweikanal-Funktionsgenerator auf Basis eines Waveshare ESP32-S3 2,8-Zoll-Touch-LCD-Displays ersetzt werden. Dieser soll in der Lage sein, vordefinierte Parameter für Amplitude, Frequenz, Phasenverschiebung und Signalform über eine CSV-Datei zu importieren und die entsprechenden Signale an den Ausgängen bereitzustellen.

Im Entwicklungsprozess wurden zwei Implementierungsmethoden, die unterschiedliche Architekturen für die Signalgenerierung verwenden, betrachtet und anschließend verglichen. Die Implementierungsmethode A nutzt für die Erzeugung der Spannungssignale die internen 8-Bit-Digital-Analog-Wandler (DAC) eines zweiten Mikrocontrollers (ESP32 DevKit C V4) und berechnet die DAC-Stützwerte rein softwaretechnisch. Die Implementierungsmethode B basiert auf der Direct Digital Synthesis (DDS) und verwendet zwei DDS-ICs (AD9833) in Kombination mit zwei digitalen Potentiometermodulen (MCP42010) und einem Operationsverstärker (OPA1612), um zwei einstellbare Spannungssignale zu erzeugen.

Beide Methoden sind in ihrer Bedienung, der Benutzeroberfläche sowie in der Übertragung der Parameter mittels UART vom HMI (Master) an einen Slave-ESP32, welcher bei beiden Methoden die Spannungsausgabe steuert, identisch. Die Varianten unterscheiden sich jedoch in ihrer Schaltungs- und Softwarekomplexität sowie in den Kosten, Abmessungen und ihrer Genauigkeit. Als bewertende Kriterien wurden die Erfüllung der funktionalen und nicht-funktionalen Anforderungen mit besonderem Fokus auf die Vorgaben für die Spannungsgenauigkeit ($\leq \pm 5\%$), Phasengenauigkeit ($\leq \pm 2^\circ$) und Frequenzgenauigkeit ($\leq \pm 5\%$) herangezogen. Bei der Gegenüberstellung der Messergebnisse beider Implementierungsvarianten hat sich gezeigt, dass Implementierungsmethode B eine deutlich bessere Genauigkeit und einen deutlich größeren Frequenz- sowie Spannungsbereich aufweist (Sinus: 1 kHz - 80 kHz bei einem Amplitudenbereich von 0,5 V - 10 V, Rechteck: 10 kHz - 80 kHz bei einem Amplitudenbereich von 2 V - 10 V). Variante A besitzt hingegen einen deutlich kleineren Frequenz- sowie Spannungsbereich (Sinus: 100 Hz - 500 Hz bei einem Amplitudenbereich von 0,5 V - 1 V, Rechteck: 10 Hz - 100 Hz bei einem Amplitudenbereich von 0,3 V - 1,1 V).

Anhand aller betrachteten Gesichtspunkte ist abschließend festzuhalten, dass die Methode B trotz der höheren Kosten, der größeren Schaltungs- und Softwarekomplexität sowie der größeren Abmessungen besser für den Einsatz im Modul Messtechnik I ET2013 geeignet ist. Methode A stellt hingegen eine geeignete Lösung für Anwendungen mit geringeren Anforderungen an Genauigkeit und Funktionsumfang dar, bei denen ein besonders kompaktes Gerät erforderlich ist.

Inhaltsverzeichnis

1	Einleitung	1
2	Stand der Technik	3
2.1	Human-Machine-Interfaces (HMIs) in der Industrie	3
2.2	Mikrocontroller und Embedded-Plattformen anhand der ESP32-Familie	3
2.3	Architekturen für die Signalgenerierung	4
3	Anforderungen und Rahmenbedingungen	6
3.1	Anwendungsfall: Laborprüfung	6
3.2	Funktionale Anforderungen	7
3.3	Nicht-funktionale Anforderungen	9
4	Konzept und Systemarchitektur	11
4.1	Komponentenübersicht (Waveshare ESP32-S3)	11
4.2	Architekturübersicht	12
4.3	Komponentenauswahl (ESP32 DevKit C V4)	13
4.4	Datenübertragung zwischen Master und Slave	14
5	Implementierungsmethode A: Signalgenerierung mittels interner Digital-Analog-Wandler eines ESP32	16
5.1	Kommunikationsprotokoll und Schnittstellendesign	16
5.2	Schaltungsentwicklung	17
5.3	Ansteuerungslogik	19
5.4	Messaufbau, Testversuche und Genauigkeitsbewertung	21
6	Implementierungsmethode B: Signalgenerierung mittels Direct Digital Synthesis	27
6.1	Konzept und Komponentenauswahl	27
6.2	Schaltungsentwicklung	32
6.3	Ansteuerungslogik	37
6.4	Messaufbau, Testversuche und Genauigkeitsbewertung	40
7	Entwicklung der Benutzeroberfläche	43
7.1	Gestaltungsprinzipien	43

7.2	Umsetzung mittels EEZ Studio und der LVGL-Library	43
8	Mechanisches Design	45
8.1	Konstruktion	45
8.2	Materialauswahl und Fertigung	46
9	Fazit	47
9.1	Vergleich	47
9.2	Bewertung und Empfehlung	49
	Quellenverzeichnis	51
	Anhang	55

1. Einleitung

Die Metropolregion Berlin zählt zu den bedeutendsten Wissenschaftsstandorten der Welt. Ihre Geschichte ist geprägt von Namen wie Planck, Humboldt und Einstein. Ihre heutige Stellung in der Forschung und Lehre verdankt sie den insgesamt vier Universitäten, sieben Hochschulen und über 70 außeruniversitären Forschungseinrichtungen. Über 200.000 Studierende aus In- und Ausland leben und lernen in Berlin. Mit rund 12.000 Studierenden ist die Hochschule für Wirtschaft und Recht Berlin (HWR Berlin) eine in der Hochschullehre fest verankerte Größe. Sie ist eine der bedeutendsten Hochschulen Berlins und bietet mit einem Portfolio von 57 Studiengängen eine große fachliche Bandbreite. Von Wirtschaftswissenschaften über Rechts- und Sicherheitsmanagement bis hin zu ingenieurwissenschaftlichen Studiengängen bietet die HWR einen engen Praxisbezug, vielfältige Forschung, hohe Qualitätsstandards und eine starke internationale Ausrichtung. [1]

Mit dem Studiengang der industriellen Elektrotechnik bietet die HWR die Möglichkeit, eine der traditionsreichsten Fachrichtungen der Ingenieurwissenschaften zu erlernen. Im Vordergrund steht dabei die praktische Anwendbarkeit des vermittelten Wissens. Die Studierenden sollen befähigt werden, an der Entwicklung elektronischer bzw. elektrotechnischer Produkte sowie an der Planung und Leitung von Projekten der Automatisierungstechnik und Energietechnik mitzuwirken. Neben den technischen spielen dabei ebenfalls die wirtschaftlichen Aspekte stets eine zentrale Rolle. [2, S. 3]

Bereits im ersten Semester begegnet den Studierenden im Studiengang der industriellen Elektrotechnik das Modul Messtechnik I (ET2013). Im Rahmen dieses Moduls erwerben die Studierenden tiefgreifende Kenntnisse im Umgang mit Messinstrumenten, in grundlegenden Aspekten der Planung und Realisierung von Messaufbauten sowie in der Durchführung von Fehleranalysen zur Bestimmung der resultierenden Messabweichungen. Um dieses Wissen vermitteln zu können, werden die Grundbegriffe der Messtechnik wie SI-Einheiten, Kalibrieren, Abgleichen, relative und absolute Messabweichung definiert, in der Theorie aufgearbeitet und mit Übungen gefestigt. Des Weiteren wird auf Fehlerarten, Fehlerfortpflanzung sowie Messwertanalysen eingegangen. Zum Ende des Semesters werden die in der Theorie erlernten Inhalte in einem Laborversuch praktisch überprüft. Durch die Erstellung einer Laborarbeit können die Studierenden die Ergebnisse unter wissenschaftlichen Gesichtspunkten auswerten und dokumentieren. Der Umgang mit Messgeräten wie Oszilloskopen und analogen bzw. digitalen Multimetern ist dabei essenziell. [2, S. 11]

Im Zuge der Neustrukturierung des Moduls, die ebenfalls eine Überarbeitung der aktuellen Prüfungsleistung beinhaltet, werden die Studierenden zukünftig als Teil der Laborversuche verschiedene Spannungssignale unter zahlreichen Gesichtspunkten messen und analysieren müssen. Um diesen Prozess für die große Anzahl an Arbeitsgruppen zu vereinfachen und zu standardisieren, soll auf die Verwendung eines klassischen Funktionsgenerators (Abbildung 1), der von den Studierenden manuell eingestellt werden muss, verzichtet werden. Stattdessen soll zukünftig ein Funktionsgenerator mit bereits vordefinierten Parametern für die einzelnen Gruppen und einer einfachen sowie benutzerfreundlichen Gruppenauswahl über ein Human-Machine-Interface (HMI) verwendet werden. [3]

Im Rahmen der nachfolgenden Arbeit werden der Entwurf, die Konzeption und der konstruktive

Aufbau eines solchen Funktionsgenerators auf der Grundlage eines Waveshare ESP32-S3 2.8-Zoll-Touch-LCD-Displays dokumentiert. Der Fokus der Ausarbeitung liegt dabei auf der Darstellung des Designprozesses sowie der Probleme und Lösungsansätze bei der Entwicklung des Geräts. Dies umfasst ebenfalls die ausführliche Durchführung von Tests zur Validierung der Funktionsweise und den Vergleich zweier Konzepte zur Signalgenerierung unter Berücksichtigung konkreter Parameter wie der Signalqualität, Genauigkeit und Wirtschaftlichkeit. Bei diesen zwei Konzepten handelt es sich konkret um die Signalgenerierung mittels interner Digital-Analog-Wandler (DAC) eines ESP32 und der Signalgenerierung mittels externer Direct Digital Synthesis (DDS) ICs.



Abbildung 1: Aktuell verwendeter Funktionsgenerator im Messtechniklabor der HWR Berlin am Standort Lichtenberg (E-Labor 6B.158) (HAMEG HM 8030-4)[4]

2. Stand der Technik

2.1. Human-Machine-Interfaces (HMIs) in der Industrie

Im Zuge der Industrie 4.0 sind intelligente Maschinen und digitale Technologien aus modernen Produktionsumgebungen nicht mehr wegzudenken. Eine zentrale Rolle nehmen dabei HMIs ein. Durch das Zusammenspiel von Hardware und Software ermöglichen sie dem Nutzer die Überwachung von Prozessen, die Visualisierung von Anlagenzuständen sowie die Anpassung relevanter Parameter. Typische Anwendungsbeispiele reichen von der Steuerung eines Förderbandes bis hin zur Regelung der Temperatur eines Wassertanks. [5, S. 2-3]

In industriellen Anlagen werden HMIs in der Regel nicht isoliert, sondern als Bestandteil eines übergeordneten Supervisory Control and Data Acquisition (SCADA) Systems eingesetzt. Ein solches System dient der Steuerung und Überwachung umfangreicher Anlagen oder ganzer Standorte und erfasst zugleich sämtliche betriebsrelevante Daten. Das System kombiniert verschiedene Teilsysteme wie speicherprogrammierbare Steuerungen (SPS), Sensoren und Remote Terminal Units (RTUs), während das HMI die Schnittstelle zwischen Nutzer und Gesamtsystem bildet. [5, S. 3] Ein praxisnahes Beispiel ist der Einsatz von HMIs bei den Berliner Wasserbetrieben (BWB). Sie visualisieren die Fördermenge sowie die Anzeige von Pumpenzuständen und Fehlermeldungen. [6, S. 73]

2.2. Mikrocontroller und Embedded-Plattformen anhand der ESP32-Familie

Mikrocontroller stellen den vorläufigen Abschluss einer Entwicklung digitaler Logiksysteme dar, welche das grundlegende Ziel der Loslösung von der starren Verdrahtung der logischen Funktionen verfolgte. Diese Entwicklung begann bereits im Jahr 1965 mit der fest verdrahteten Logik oder auch hardwired logic. Hierbei wurden logische Funktionen und komplizierte sequenzielle Abläufe ausschließlich mittels der festen Verdrahtung von Bauelementen wie Gattern, Flipflops und Schieberegistern realisiert. Das Programm ergab sich also aus der Verdrahtung der einzelnen Komponenten. Der Hauptnachteil dieses Logiktyps liegt darin, dass die logischen Funktionen bzw. das Programm eines fertiggestellten Aufbaus kaum oder nur sehr umständlich veränderbar sind. Jede Anpassung war mit enormem Aufwand und Arbeitszeit verbunden. [7, S. 1]

Mit der Entwicklung der ersten Halbleiterspeicher ROM (read only memory) und RAM (random access memory) um das Jahr 1972 wurde die Entwicklung eines neuen Logiktyps möglich. Bei der sogenannten mikroprogrammierten Logik wird ein Teil des Programms in einem Festwertspeicher und nicht durch die reine Verdrahtung von Bauelementen realisiert. Gleichzeitig wurden immer mehr flexible digitale Bausteine entwickelt, die die mikroprogrammierte Logik weiter unterstützten. [7, S. 1-2]

Durch die Entwicklung der Mikroprozessoren und Mikrocontroller im Jahr 1973 erreichte die Hardware einen universellen Charakter, wurde platzsparender und konnte in zahlreichen Geräten eingesetzt werden. Dabei liegen sowohl die Logik als auch die Sequenzen normalerweise im Programm, also der Software des Mikrocontrollers. Der Mikrocontroller ist dabei als ein hoch-

integrierter Baustein zu verstehen, der neben dem Mikroprozessor einen oder mehrere serielle und parallele Portbausteine, Datenspeicher und andere Sonderfunktionen an Hardware auf einem einzigen Chip vereint. Ein Mikrocontroller besitzt also im Gegensatz zum Mikroprozessor bereits zahlreiche Peripheriebausteine. Aufgrund des geringen Aufwands bezüglich des Hardwaredesigns eignen sich Mikrocontroller besonders für einfache Steuerungsaufgaben wie z. B. Aufzugs-, Waschmaschinen- und Steuerungen aller Art. Auch bei umfangreicheren Aufgaben, wie beispielsweise in Kraftfahrzeugen, finden sie Anwendung. [7, S. 2-3]

Im Bereich des Internet of Things (IoT) und in Embedded-Projekten haben sich in den letzten Jahren besonders Mikrocontroller der Firma Espressif Systems Company, welche sich als eine leistungsstarke und kostengünstige Plattform für die Entwicklung zahlreicher Anwendungen erwiesen haben, etabliert. [8] Seit der Markteinführung im Jahr 2016 vertreibt Espressif Systems seine Mikrocontroller überwiegend unter der Bezeichnung ESP32. [9]

Dabei bietet das Unternehmen zahlreiche Produktvarianten für unterschiedliche Anwendungsbereiche an. Neben den im Privatbereich weit verbreiteten Modulen wie dem ESP32-WROOM oder ESP32-MINI, die sich durch Xtensa-Prozessoren mit einem oder zwei Kernen, Taktfrequenzen von bis zu 240 MHz sowie zahlreiche Peripherieschnittstellen wie kapazitive Touchsensoren, Hall-Sensoren, Ethernet, SPI, UART, I²C und I²S auszeichnen, bietet Espressif Systems mit der ESP32-S3-Serie ebenfalls Mikrocontroller für komplexere Anwendungen wie die Verarbeitung neuronaler Netze und die digitale Signalverarbeitung an. [10]

Durch Funktionen wie integrierte Wi-Fi- und Bluetooth-Konnektivität, eine große Anzahl an General-Purpose-Input/Output-(GPIO)-Pins sowie die vergleichsweise hohe Rechenleistung eignen sich ESP32-Module besonders für anspruchsvollere Embedded-Projekte. [8]

2.3. Architekturen für die Signalgenerierung

Für die Generierung von Spannungssignalen haben sich über die Jahre einige Architekturen etabliert. Sie unterscheiden sich primär in der realisierbaren Signalqualität, Frequenzauflösung, Komplexität und den Kosten. Im einfachsten Fall wird ein periodisches Signal durch die softwaregesteuerte Ausgabe von vorberechneten Stützwerten über einen DAC erzeugt. [11, S. 1] In einigen Mikrocontrollern sind solche DAC-Ausgänge bereits integriert und ohne zusätzliche Hardware nutzbar. [12, S. 45]

Eine leistungsfähigere Methode zur Signalerzeugung, die in den Bereichen der Nachrichtentechnik und allgemein in industriellen Anwendungen immer häufiger zum Einsatz kommt, ist die Direct Digital Synthesis. Bei der DDS erfolgen die Signalgenerierung und die Digital-Analog-Wandlung meist in einem einzigen dedizierten Baustein. Das Funktionsprinzip basiert auf einem Phasenakkumulator, der mit jedem Taktzyklus eines Referenztaktes um einen festen, digital programmierbaren Wert, das sogenannte tuning-word, inkrementiert wird. [11, S. 1] Gegenüber einer rein softwarebasierten Lösung bietet die DDS-Architektur dadurch eine deutlich feinere Frequenzauflösung, ein schnelles und phasenkontinuierliches Umschalten zwischen Frequenzen sowie eine hohe Unempfindlichkeit gegenüber Bauteilalterung und Temperaturveränderungen. [11, S. 3]

Im Rahmen dieser Ausarbeitung werden genau diese zwei Varianten der Signalgenerierung unter

verschiedenen Gesichtspunkten analysiert, miteinander verglichen und hinsichtlich ihrer Eignung für den Einsatz in einem Funktionsgenerator bewertet.

Eine detaillierte Beschreibung der Funktionsweise, des internen Aufbaus sowie der charakteristischen Kennwerte der einzelnen Verfahren zur Signalgenerierung erfolgt im Rahmen der jeweiligen Kapitel, welche die Umsetzung der jeweiligen Methoden behandeln (Kapitel 5 und Kapitel 6).

3. Anforderungen und Rahmenbedingungen

In den Nachfolgenden Abschnitten werden die Anforderungen bezüglich des zu entwickelnden Funktionsgenerators auf Grundlage des vorhandenen Prüfungsmaterials des Moduls Messtechnik I (ET2013) und die daraus entwickelten funktionalen und nicht-funktionalen Anforderungen beschrieben.

3.1. Anwendungsfall: Laborprüfung

Wie bereits in der Einleitung dieser Ausarbeitung erwähnt, durchläuft das Modul Messtechnik I (ET2013) aktuell einer Neustrukturierung, die ebenfalls eine Überarbeitung der Laborprüfung beinhaltet. Die neu entwickelten Aufgaben der Laborarbeit umfassen das Analysieren und Messen verschiedener sinus- sowie rechteckförmiger Spannungssignale. Im Zuge dieser Aufgaben sollen die Studierenden zahlreiche Parameter bezüglich der verschiedenen Spannungssignale messen und berechnen. Diese Parameter sind in Tabelle 1 aufgelistet.

Konkret lauten die Aufgabenstellungen wie folgt:

- „Analysieren Sie die sinusförmigen Spannungssignale im Oszilloskop in Kanal 1 und Kanal 2 und geben Sie die Messwerte gemäß der folgenden Tabelle ein:“ [13, S. 1]
- „Analysieren Sie das rechteckförmige Spannungssignal im Oszilloskop in Kanal 1 und geben Sie die Messwerte gemäß der folgenden Tabelle ein:“ [13, S. 2]

Die Messwerte beziehen sich dabei auf die in Tabelle 1 angegebenen Parameter. Anhand dieser Parameter und Aufgabenstellungen könnten bereits zahlreiche funktionale Anforderungen aufgestellt werden. Neben den eigentlichen Aufgaben, welche die Studierenden im Rahmen der Laborarbeit erhalten und bearbeiten müssen, gibt es zudem eine vorgefertigte Tabelle, in welcher die unterschiedlichen Parameter der einzelnen Arbeitsgruppen bereits vordefiniert sind. Die einzelnen Arbeitsgruppen erhalten also nicht alle einheitliche Parameter, sondern jede Gruppe erhält für jede Aufgabe individuelle Spannungssignale. Ein kleiner Ausschnitt aus den Tabellen für die sinus- sowie rechteckförmigen Spannungssignale ist in den Tabellen 15 und 16 auf den Seiten 55 und 56 im Anhang zu finden. Da die Studierenden unter Verwendung des aktuellen Funktionsgenerators die Spannungssignale selbstständig einstellen müssten, wären ihnen die tatsächlichen Werte bereits bekannt und ein Lerneffekt durch das gezielte Bestimmen mittels des Oszilloskops würde weitestgehend ausbleiben. Aufgrund dieser Tatsache soll der neu entwickelte Funktionsgenerator eine Möglichkeit bieten, die Spannungssignale mit den entsprechenden Werten der einzelnen Arbeitsgruppen ohne das eingreifen eines Studierenden auszugeben. Dies könnte beispielsweise über eine Comma-Separated-Values (CSV) -Datei oder über das Eingeben der einzelnen Parameter durch den Professor über eine serielle Verbindung mit einem beliebigen digitalen Endgerät erfolgen. Dabei würden die Werte für die Frequenz, Ausgangsspannung und Phasenverschiebung ohne das eingreifen der Studierenden festgelegt werden. Diese müssten nur über eine Bedienoberfläche auf dem HMI ihre Arbeitsgruppe auswählen und die Ausgabe des Spannungssignals starten und stoppen. Durch diese Struktur wären den Studierenden die vorgegebenen Parameter zu keinem Zeitpunkt

Tabelle 1: Zu erhebende Messwerte der Studierenden bezüglich der sinus- sowie rechteckförmigen Spannungssignale als Teil der Laborarbeit des Moduls Messtechnik I (ET2013) [13, S. 1-2]

sinusförmige Spannungssignale	
T	Periodendauer
f	Frequenz
ω	Kreisfrequenz
$\bar{u}$	Offset (Mittelwert)
$\hat{u}$	Scheitelwert (max)
$\check{u}$	Scheitelwert (min)
U_e	Spitze-Tal-Wert
U_{rms}	Effektivwert
Δt	Phasenverschiebung in Zeit von Kanal 2 bezüglich Kanal 1
$\varphi[^\circ]$	Phasenverschiebung in Grad von Kanal 2 bezüglich Kanal 1
$\varphi[rad]$	Phasenverschiebung in Radiant von Kanal 2 bezüglich Kanal 1
$\varphi[\%]$	Phasenverschiebung in Prozent von Kanal 2 bezüglich Kanal 1
rechteckförmige Spannungssignale	
T	Periodendauer
f	Frequenz
ω	Kreisfrequenz
$\Delta u = \check{u} - 0 \text{ V}$	Offset von Null
$\hat{u}$	Scheitelwert (max)
$\check{u}$	Scheitelwert (min)
U_e	Spitze-Tal-Wert
U_{rms}	Effektivwert
τ	Impulsdauer
$D = \tau/T$	Tastgrad in Prozent (%)

bekannt, wodurch eine Anpassung oder Manipulation der Messergebnisse durch die Studierenden verhindert werden könnte. Dadurch wäre sichergestellt, dass jede Arbeitsgruppe ausschließlich mit den ihr zugewiesenen Signalparametern arbeitet, die Versuchsdurchführung unter vergleichbaren Bedingungen erfolgt und die Messergebnisse tatsächlich eigenständig durch die Studierenden mithilfe des Oszilloskops bestimmt werden.

3.2. Funktionale Anforderungen

Der in Abschnitt 3.1 beschriebene Anwendungsfall sowie die vorgegebenen Aufgabenstellungen ermöglichen die Aufstellung zahlreicher funktionaler und nicht-funktionaler Anforderungen. Da sich aus den Aufgabenstellungen und den Messwerttabellen jedoch nicht alle Zielerfordernisse ableiten lassen, wurde beschlossen, diese durch weitere essenzielle Punkte zu ergänzen. Darunter fallen beispielsweise die Einbindung von Hilfestellungen zur Anwendung der Laborgeräte (FA-12) und das Speichern der Gruppenparameter in einem Zwischenspeicher für den Fall, dass der Funkti-

ongenerator abstürzt oder ausgeschaltet werden sollte (FA-11). Für die Validierung der Genauigkeit und der Funktion der Funktionsgeneratoren werden dabei folgende Messwerte herangezogen: die Amplitude der Spannungssignale (entspricht dem Scheitelwert $\hat{u}$), die Phasenverschiebung in Grad zwischen den Signalen und die Frequenz der Spannungssignale. Die Implementierung eines variablen Offsets sowie eines variablen Tastgrads wird nicht durchgeführt. Die final aufgestellten funktionalen Anforderungen sind in Tabelle 2 aufgelistet.

Tabelle 2: Funktionale Anforderungen des Projekts

Requirement number	Beschreibung
FA-1:	Das System muss zwei unabhängige analoge Ausgänge (Kanal A, Kanal B), welche gleichzeitig genutzt werden können, bereitstellen.
FA-2:	Bei den analogen Ausgangssignalen der Kanäle A und B soll es sich um null symmetrische sinusförmige sowie rechteckförmige Signale handeln.
FA-3:	Der Funktionsgenerator soll in der Lage sein, auf beiden Kanälen die Frequenz des Signals zu verändern.
FA-4:	Der Funktionsgenerator soll in der Lage sein, auf beiden Kanälen die Amplitude des Signals zu verändern.
FA-5:	Zwischen den beiden Ausgangssignalen muss eine definierte Phasenverschiebung einstellbar sein.
FA-6:	Das System muss die Verwendung vordefinierter Parametersätze unterstützen. Ein Parametersatz umfasst mindestens: • Gruppenidentifikation • Frequenz Kanal 1 • Frequenz Kanal 2 • Amplitude Kanal 1 • Amplitude Kanal 2 • Phasenverschiebung • Signalform
FA-7:	Der Funktionsgenerator soll über ein HMI mit Touch-Funktion verfügen.
FA-8:	Über die Benutzeroberfläche (HMI) muss ein schnelles Umschalten zwischen den einzelnen Gruppen, ohne, dass das System neu gestartet werden muss, möglich sein.

Requirement number	Beschreibung
FA-9:	Die Festlegung der Parameter für die Ausgangssignale der einzelnen Gruppen erfolgt über eine Eingabe per UART, das Einlesen einer CSV-Datei oder vergleichbare Lösungen für den Import von Parametern.
FA-10:	Über die Benutzeroberfläche muss die Ausgabe der Signale auf beiden Kanälen gestartet und gestoppt werden können. Dies kann beispielsweise über dedizierte Buttons umgesetzt werden.
FA-11:	Sollte das System ausgeschaltet werden oder abstürzen und es werden keine neuen Daten über eine microSD-Karte oder UART übergeben, werden die Parameter der letzten Sitzung bzw. Anwendung verwendet.
FA-12:	Auf dem HMI können die Studierenden Hilfestellungen für die verwendeten Laborgeräte erhalten. Zu diesen Hilfestellungen gehören beispielsweise das Messen der Phasenverschiebung, der Amplitude und der Frequenz mittels des Oszilloskops.
FA-13:	Für die Navigation auf dem HMI sind dedizierte Buttons oder anderweitige Bedienelemente implementiert.

3.3. Nicht-funktionale Anforderungen

Neben den funktionellen Anforderungen wurden auf der Grundlage des in Kapitel 3.1 beschriebenen Laborversuchs zudem zahlreiche nicht-funktionale Anforderungen definiert. Diese wurden, genau wie die funktionalen Anforderungen, durch ergänzende Parameter erweitert. Die final aufgestellten nicht-funktionalen Anforderungen sind in Tabelle 3 aufgelistet.

Tabelle 3: Nicht-funktionale Anforderungen des Projekts

Requirement number	Beschreibung
NFA-1:	Amplitudenfehler $\leq \pm 5 \%$
NFA-2:	Phasenfehler $\leq \pm 2^\circ$
NFA-3:	Frequenzfehler $\leq \pm 5 \%$
NFA-4:	Einstellbarer Frequenzbereich mind. 10 Hz bis 10 kHz. Akzeptiert: Falls dieser Frequenzbereich nicht erreicht werden kann, sollte der Frequenzbereich so groß sein, dass für alle Gruppen unterschiedliche Frequenzen festgelegt werden können.

Requirement number	Beschreibung
NFA-5:	Der einstellbare Spannungsbereich muss es ermöglichen, dass für alle Gruppen unterschiedliche Amplitudenwerte festgelegt werden können.
NFA-6:	Touch-Eingaben müssen in weniger als 100 ms für den Benutzer sichtbar umgesetzt und an den Ausgängen wirksam sein.
NFA-7:	Der Funktionsgenerator muss einen stabilen Betrieb ohne Absturz oder Neustart ermöglichen.
NFA-8:	Bei fehlerhaften Eingabedateien oder Hardwarefehlern fällt das Gerät in einen definierten sicheren Zustand und es wird eine eindeutige Fehlermeldung angezeigt.
NFA-9:	Die Stromversorgung wird über ein Netzteil oder optional über einen internen Akku geregelt. Falls ein Akkubetrieb vorgesehen ist: Die Laufzeit sollte ≥ 2 h bei typischer Nutzung betragen. Zudem sollte ein Batteriemanagementsystem (BMS) implementiert sein.
NFA-10:	Der Programmcode ist strukturiert und modular aufgebaut. Einzelne Funktionen sind, um die Verständlichkeit zu verbessern, durch Kommentare beschrieben. Die Modularität des Programmcodes ermöglicht sowohl Wiederverwendbarkeit als auch Skalierbarkeit.
NFA-11:	Der Funktionsgenerator besitzt ein labortaugliches Gehäuse mit eindeutiger Beschriftung der einzelnen Anschlüsse und guter Bedienbarkeit.
NFA-12:	Die Hinweise zur Anwendung der Laborgeräte sind verständlich und gut lesbar.

4. Konzept und Systemarchitektur

4.1. Komponentenübersicht (Waveshare ESP32-S3)

Das Waveshare ESP32-S3-Touch-LCD-2.8 bildet die zentrale Steuer- und Bedieneinheit des entwickelten Funktionsgenerators. Das Board basiert auf dem ESP32-S3R8-Mikrocontroller von Espressif Systems, der einen Xtensa 32-Bit LX7 Dual-Core-Prozessor mit einer Taktfrequenz von bis zu 240 MHz besitzt. Der verfügbare Arbeitsspeicher umfasst 512 KB SRAM sowie 8 MB PSRAM, ergänzt durch 16 MB Flash-Speicher für die dauerhafte Ablage von Programm- und Konfigurationsdaten. [15, S. 1-2.]

Das Board verfügt über ein integriertes 2,8-Zoll-IPS-Display mit einer Auflösung von 240×320 Pixeln und einem kapazitiven Touchscreen, der 5-Punkt-Touch über die I²C-Schnittstelle unterstützt. In der Abbildung 2 ist das Board in der Front- und Rückansicht abgebildet. Die Kombination aus Display und Touchscreen ermöglicht die Realisierung einer intuitiven und benutzerfreundlichen Mensch-Maschine-Schnittstelle. Darüber hinaus unterstützt das Board die Light and Versatile Graphics Library (LVGL), wodurch die Entwicklung einer strukturierten, leistungsfähigen und reaktionsschnellen grafischen Benutzeroberfläche ohne zusätzliche Hardwarekomponenten ermöglicht wird. [15, S. 1]

Für die Anbindung externer Peripheriebausteine stehen GPIO-, UART- sowie I²C-Schnittstellen über dedizierte Header-Anschlüsse auf der Rückseite des HMIs zur Verfügung. Zudem verfügt das Board über einen integrierten microSD-Karten-Slot, der über den SPI-Bus angesprochen wird. Dieser würde beispielsweise das Einlesen der gruppenspezifischen Parameterdatei im CSV-Format ermöglichen. [15, S. 2-3] Die Pinbelegung sowie alle Anschlussmöglichkeiten des Boards sind in Abbildung 3 übersichtlich dargestellt.



(a) HMI Frontansicht



(b) HMI Rückansicht

Abbildung 2: Waveshare ESP32-S3-Touch-LCD-2.8 [15, S. 5]

4.2. Architekturübersicht

Für die Umsetzung des Funktionsgenerators wurde eine Master-Slave-Architektur, deren Aufbau in Abbildung 4 modellhaft dargestellt ist, entwickelt. Das Waveshare-HMI-Modul verfügt zwar über umfangreiche Peripherie zur Anbindung von Display, Touchcontroller und microSD-Karte, jedoch über keine für die Signalerzeugung nutzbaren DAC-Ausgänge. Zudem sind durch die internen Anbindungen die meisten GPIO-Pins des ESP32-S3 bereits belegt, sodass über die externen Header-Anschlüsse lediglich zwei freie GPIO-Pins (GPIO-15 und GPIO-18) verbleiben. [15, S. 5] Eine Realisierung der Signalerzeugung auf dem Master selbst, unter Verwendung externer Signalbausteine, ist daher nicht sinnvoll umsetzbar. Aus diesem Grund wird ein zweiter Mikrocontroller in das System integriert. Bei diesem handelt es sich um ein ESP32 DevKit mit zwei internen DAC-Pins an GPIO 25 und GPIO 26. Die Kommunikation zwischen beiden Controllern erfolgt über eine UART-Schnittstelle, sodass der Master den Slave durch die Übergabe der gruppenspezifischen Parameter ansteuern kann.

Aus dieser Aufteilung ergeben sich drei wesentliche Vorteile. Erstens entsteht eine klare funktionale Trennung zwischen Bedienlogik und Signalerzeugung, wodurch beide unabhängig voneinander entwickelt und getestet werden können. Zweitens wird die Echtzeitfähigkeit des Systems verbessert, da die zeitkritische Signalausgabe nicht durch Touch-Ereignisse oder das Rendern der grafischen Benutzeroberfläche unterbrochen wird. Drittens erhöht die Aufteilung die Modularität des Gesamtsystems. Der Master bleibt für unterschiedliche Implementierungsansätze identisch, während die Signalerzeugung auf der Slave-Seite ausgetauscht werden kann.

Die Rollenverteilung ergibt sich somit unmittelbar aus diesem Konzept. Der Master übernimmt

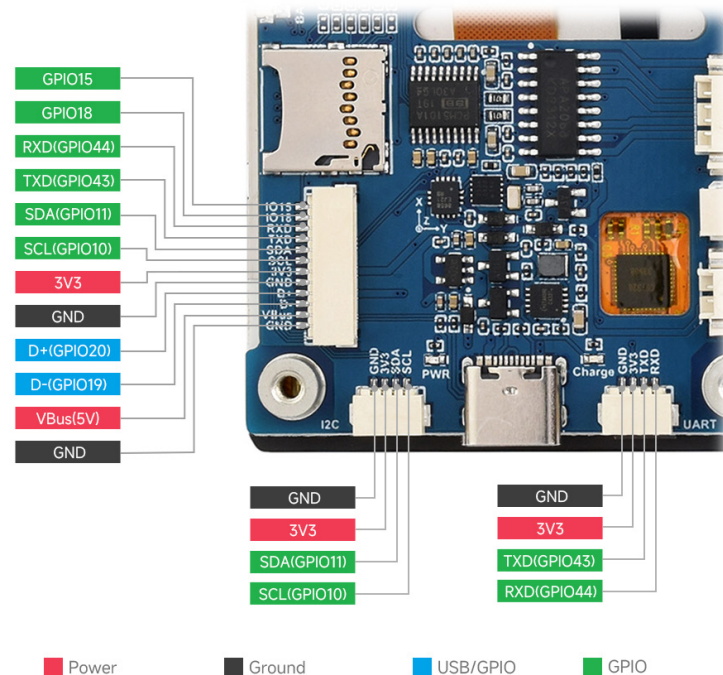


Abbildung 3: Pinbelegung und Anschlussmöglichkeiten des Waveshare ESP32-S3-Touch-LCD-2.8 [16]

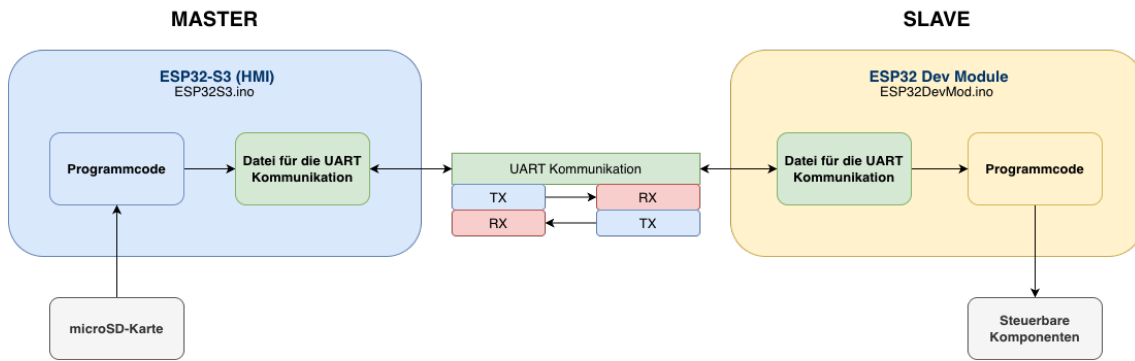


Abbildung 4: Modellhafte Darstellung des Master-Slave-Prinzips

die Darstellung der Benutzeroberfläche, das Einlesen und Verarbeiten der Parameterdatei von der microSD-Karte sowie die Übergabe der ausgewählten Parameter an den Slave. Der Slave ist hauptsächlich für die Erzeugung der Spannungssignale gemäß den empfangenen Vorgaben zuständig.

4.3. Komponentenauswahl (ESP32 DevKit C V4)

Für die Realisierung der Signalerzeugung wurde das ESP32 DevKit C V4 der Firma AZ-Delivery (Abbildung 5) als zweiter Mikrocontroller ausgewählt. Das DevKit basiert auf dem ESP32-WROOM-32-Modul, welches eine Taktfrequenz von bis zu 240 MHz besitzt. Der verfügbare Arbeitsspeicher umfasst 520 KB SRAM, ergänzt durch 4 MB SPI-Flash für die Ablage von Programmcode und Konfigurationsdateien. [17, S. 2]

Entscheidend für die Auswahl dieses Moduls sind die beiden 8-Bit-Digital-Analog-Wandler an GPIO 25 und GPIO 26, die für die Umsetzung der in Kapitel 5 beschriebenen Implementierungsmethode A unverzichtbar sind. Da weder das Waveshare-HMI noch leistungstärkere Varianten der ESP32-S3-Serie über derartige DAC-Ausgänge verfügen, stellt das ESP32-WROOM-32-Modul innerhalb der ESP32-Familie eine technisch konsistente und zugleich kostengünstige Lösung dar.

Neben den DAC-Pins bietet das DevKit eine große Anzahl an frei verfügbaren GPIO-Pins sowie die

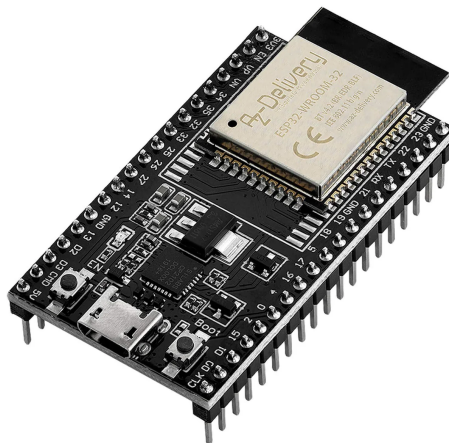


Abbildung 5: ESP32 DevKit C V4 der Firma AZ-Delivery [18]

für beide Implementierungsmethoden erforderlichen Schnittstellen. Über die UART-Schnittstelle erfolgt der Empfang der gruppenspezifischen Parameter vom Master, während die SPI-Schnittstelle in der in Kapitel 6 beschriebenen Implementierungsmethode B zur Ansteuerung der externen Komponenten genutzt wird. [17, S. 16-17] Mit einem Betriebsspannungsbereich von 3,0 V bis 3,6 V lässt sich das Modul direkt über die 3,3-V-Versorgung des Waveshare-Boards versorgen, sodass für den Slave keine eigenständige Spannungsversorgung erforderlich ist. [17, S. 26]

4.4. Datenübertragung zwischen Master und Slave

Wie in Abschnitt 4.2 beschrieben, übernimmt der Master die Verwaltung der gruppenspezifischen Parameter, während der Slave für die Erzeugung der Spannungssignale verantwortlich ist. Damit der Slave die Parameter ausgeben kann, müssen diese zunächst auf dem Master eingelesen und anschließend über die UART-Schnittstelle übergeben werden. Der gesamte Datenfluss gliedert sich in die drei Phasen: Einlesen, Verarbeitung und Übertragung.

Die Parameter der einzelnen Arbeitsgruppen werden auf einer microSD-Karte in einer CSV-Datei abgelegt. Jede Zeile beschreibt eine Arbeitsgruppe und enthält neun durch Kommata getrennte Werte: die Gruppen-ID sowie jeweils Amplitude (in mV), Frequenz (in Hz), Phase (in °) und Signalform (0 = SIN, 1 = SQU) für die beiden Kanäle. Die erste Zeile der Datei dient als Header und enthält die Spaltenbezeichnungen. Tabelle 4 zeigt den Inhalt der verwendeten Datei mit drei Beispielgruppen. Die Wahl des CSV-Formats begründet sich durch dessen einfache Handhabung. Parametersätze lassen sich ohne großen Aufwand in einem beliebigen Tabellenkalkulationsprogramm (z.B. Excel) erstellen und anpassen. Zudem ist das zeilenorientierte Format ressourcenschonend, da während des Einlesens stets nur eine Zeile im Arbeitsspeicher gehalten werden muss.

Sobald der Nutzer auf dem HMI die Funktion zum Laden der Parameter auslöst, wird über den Master der SPI-Bus initialisiert, die Datei geöffnet und zeilenweise eingelesen. Dabei wird die Header-Zeile übersprungen und jede weitere Zeile wird in eine Datenstruktur des Typs `SignalGroup` überführt und in einem Array im Arbeitsspeicher des Masters abgelegt. Die eigentliche Zerlegung einer Zeile übernimmt die in Listing 1 auf Seite 57 im Anhang gezeigte Funktion `csv_parse_line()`. Sie wertet die Zeichenkette mittels `csv_read_value()` feldweise von links nach rechts aus und konvertiert die Werte je nach Datentyp über `atof()` bzw. `atoi()`. Vor dem Schreiben der Variablen wird die Struktur mittels `memset()` mit Nullwerten initialisiert. Die maximale Anzahl an Gruppen ist, um den Speicherverbrauch zu minimieren, durch die Konstante `MAX_GROUPS` begrenzt.

Tabelle 4: Signalparameter – drei Beispielgruppen aus der CSV-Datei

Gruppe	CH1				CH2			
	Ampl.	Freq.	Phase	Form	Ampl.	Freq.	Phase	Form
1	410	50	0	0	830	50	90	0
2	1500	100	0	1	1000	100	45	1
3	620	500	0	1	750	500	20	1

Nachdem die Parameter aller Gruppen eingelesen wurden, werden diese an den Slave übertragen. Das Übertragungsprotokoll ist text- und zeilenbasiert aufgebaut. Sowohl der serielle Monitor des Masters als auch der des Slaves geben sämtliche Nachrichten im Klartext aus, was die Fehlersuche während der Entwicklung erheblich vereinfacht.

Eine vollständige Übertragung gliedert sich in drei Phasen:

1. **Ankündigung:** Der Master sendet die Nachricht COUNT:n, wobei n für die Anzahl der folgenden Gruppen steht. Der Slave kann dadurch den erforderlichen Speicher reservieren und überprüfen, ob die anschließende Übertragung vollständig empfangen wurde.
2. **Parameter:** Für jede Gruppe wird eine Zeile mit allen neun Parametern im Format G:<id>,A1:<Wert>,F1:<Wert>,P1:<Wert>,S1:<Wert>,A2:... übertragen.
3. **Abschluss:** Mit der Nachricht DONE signalisiert der Master das Ende der Übertragung.

Alle Nachrichten werden vom Slave mit einem ACK (Acknowledge) oder NACK (Negative Acknowledge) quittiert. Der Master wartet für jeden gesendeten Datensatz maximal 500 ms auf eine Bestätigung, bevor die Übertragung als fehlgeschlagen erkannt und abgebrochen wird. Durch diesen Mechanismus kann ein nicht reagierender Slave identifiziert werden und eine Fehlermeldung kann ausgegeben werden.

Neben der Übertragung der Gruppenparameter, werden zusätzlich zwei weitere Befehle vom Master übermittelt. Mit dem Befehl START:<id> fordert der Master den Slave auf, die Signalausgabe für die Gruppe mit der angegebenen ID zu starten. Mit der Nachricht STOP soll die aktive Ausgabe beendet werden. Beide Befehle erwarten ein ACK vom Slave als Bestätigung. Diese Aufteilung der Kommunikation in einen einmaligen Initialisierungs- und einen wiederkehrenden Steuerungsteil vermeidet überflüssigen Datenverkehr während der Nutzung, da die einmal übertragenen Parameter im Speicher des Slaves verbleiben und über die Gruppen-ID jederzeit angesprochen werden können. Abbildung 12 auf der Seite 59 im Anhang zeigt die Aufnahmen der drei Phasen einer vollständigen Übertragungssequenz.

5. Implementierungsmethode A: Signalgenerierung mittels interner Digital-Analog-Wandler eines ESP32

In diesem Kapitel wird die erste Methode zur Signalerzeugung vorgestellt. Ausgehend von der in Kapitel 4.2 beschriebenen Master-Slave-Aufteilung werden die Umsetzung des Kommunikationsprotokolls sowie die Generierung der Spannungssignale beschrieben. Der Schwerpunkt liegt auf der Überprüfung der in Kapitel 3 erarbeiteten Anforderungen an das System. Die durchgeführten Tests bilden zugleich die Grundlage für den späteren Vergleich der beiden Implementierungsansätze in Kapitel 9.

Die Verarbeitung der Parameter sowie die Ausgabe der Signale wurde softwareseitig in der Programmiersprache C++ umgesetzt. Als Entwicklungsumgebung wurde die Arduino IDE (Version 2.3.X) gewählt. Diese bietet eine Vielzahl bereits vorhandener Bibliotheken und verfügt über eine große globale Nutzerbasis, wodurch sowohl die Entwicklung als auch die Fehlersuche erleichtert werden. [19]

5.1. Kommunikationsprotokoll und Schnittstellendesign

Während in Abschnitt 4.4 der Datenfluss aus Sicht des Masters dargestellt wurde, beschreibt dieses Unterkapitel die slaveseitige Umsetzung des Protokolls auf dem ESP32 DevKit. Der Fokus liegt auf der Darstellung der Empfangslogik, dem Parsing der eingehenden Nachrichten sowie dem Zusammenspiel zwischen den UART-Schnittstellen.

Die Empfangslogik teilt sich beim Slave in zwei Zustände auf. Diese sind im Code durch die boolsche Variable `receiving` realisiert worden. Im Grundzustand ist die Variable auf `false` gesetzt und wartet auf eine `Count:<id>`-Nachricht, die den Beginn einer neuen Parameterübertragung signalisiert. Mit dem Empfang der Nachricht setzt der Slave die Variable `receiving` auf `true` und beginnt, die einzelnen Gruppen einzulesen. Diese werden zunächst in einem internen Puffer zwischengespeichert. Nach Erhalt der Nachricht `DONE` ist die Übertragung beendet und der Slave speichert alle Gruppenparameter im Non-Volatile Storage (NVS). Zuletzt wird der Slave wieder in den Grundzustand (`receiving = false`) zurückgesetzt.

Die Steuerbefehle `START:<id>` und `STOP` sind bewusst zustandsunabhängig implementiert. Sie greifen ausschließlich auf den NVS zu, in welchem die Datensätze gespeichert sind. Somit können diese Befehle immer verarbeitet werden und sind getrennt von der Datenübertragung. Diese Trennung ist die unmittelbare technische Umsetzung der Anforderung FA-11. Da die im NVS gespeicherten Daten auch nach einem Neustart oder einem Stromausfall verfügbar sind, kann der Master eine zuvor übermittelte Gruppe jederzeit erneut starten, ohne die Parameter ein zweites Mal übertragen zu müssen. Tabelle 5 fasst die vom Slave erkannten Nachrichten, die ausgelöste Reaktion sowie die jeweilige Quittierung zusammen.

Die eigentliche Überführung einer empfangenen Parameterzeile in die in Abschnitt 4.4 eingeführte Datenstruktur `SignalGroup` übernimmt die Funktion `parse_group()`. Sie verwendet einen festen `sscanf`-Format-String, der exakt die neun Felder einer Gruppenzeile beschreibt. Der Rückgabewert von `sscanf` entspricht der Anzahl der erfolgreich eingelesenen Felder und dient unmittelbar als

Tabelle 5: Verarbeitete Nachrichten des Slaves, ausgelöste Aktion und Quittierung.

Nachricht	Aktion	Antwort
COUNT:<n>	Anzahl prüfen ($0 < n \leq \text{MAX_GROUPS}$), Zähler zurücksetzen, Zustand <code>receiving = true</code>	ACK / NACK
G:<id>,A1:...,S2:<Wert>	Zeile parsen, Werte in den RAM-Puffer übernehmen	ACK / NACK
DONE	Vollständigkeit prüfen, RAM-Puffer in den NVS schreiben, Zustand <code>receiving = false</code>	ACK / NACK
START:<id>	Gruppe aus NVS laden, DAC-Ausgabe starten	ACK / NACK
STOP	DAC-Ausgabe beenden, Kanäle auf den DAC-Mittelwert (128) zurücksetzen	ACK

Plausibilitätsprüfung. Werden nicht alle neun Felder erkannt, gilt die Zeile als ungültig, es wird eine Fehlermeldung über den seriellen Monitor ausgegeben und die Zeile an den Master mit einem NACK quittiert. Erst bei einer vollständig erkannten Zeile werden die eingelesenen Werte in die Variablen der `SignalGroup`-Struktur übernommen. Listing 2 auf Seite 58 im Anhang zeigt die Funktion in ihrer im Quellcode vorliegenden Form.

Die `SignalGroup`-Strukturen werden jedoch nicht unmittelbar dauerhaft gespeichert. Eine zentrale Designentscheidung des Slaves besteht vielmehr darin, eingehende Gruppenzeilen zunächst in einem RAM-Puffer der festen Größe `MAX_GROUPS` zwischenspeichern und erst nach Empfang der abschließenden `DONE`-Nachricht in den NVS zu übernehmen. Dadurch bleibt der bisher gespeicherte Parametersatz auch dann unversehrt, wenn eine Übertragung mitten in der Sequenz abbricht, beispielsweise weil das Verbindungskabel gelöst wird oder der Master einen unerwarteten Reset durchläuft. Erst eine vollständige und fehlerfreie Übertragung führt zu einer Veränderung des persistenten Speichers. Diese Pufferung und Persistierung ist in der Datei `save_data.cpp` umgesetzt.

Mit dem Empfang einer gültigen `START:<id>`-Nachricht endet der Aufgabenbereich des Kommunikationsteils. Die Funktion `dac.start_output()`, deren interne Funktionsweise in Abschnitt 5.3 ausführlich beschrieben wird, übernimmt die in der `SignalGroup`-Struktur enthaltenen Parameter und beginnt mit der Ausgabe der entsprechenden Spannungssignale an den beiden DAC-Pins.

5.2. Schaltungsentwicklung

In diesem Abschnitt wird der schaltungstechnische Aufbau der Implementierungsmethode A zur Signalerzeugung beschrieben. Der vollständige Schaltplan ist in Abbildung 13 auf Seite 60 im Anhang zu finden.

Die Spannungsversorgung des HMIs beträgt 5 V DC und erfolgt über die integrierte USB-C-Schnittstelle. Das HMI (Master) versorgt anschließend den Slave mit 3,3 V. Eine gemeinsame Masseverbindung zwischen den beiden Boards stellt das für die UART-Kommunikation erforderliche Bezugspotential sicher. Die UART-Pins des Masters, U0TXD (GPIO 43) und U0RXD (GPIO

44), sind mit den entsprechenden Pins des Slaves, U2TXD (GPIO 17) und U2RXD (GPIO 16), gekreuzt verbunden, sodass TX jeweils auf RX trifft.

Die internen DACs des ESP32 können keine negativen Spannungswerte ausgeben. Somit sind die Spannungssignale nicht symmetrisch um den Nullpunkt und weisen einen Gleichspannungsoffset auf. Deswegen ist an den beiden DAC-Ausgängen des Slaves je ein Keramik-Koppelkondensator mit einer Kapazität von 100 nF in Reihe geschaltet. Diese entfernen den Gleichspannungsanteil des Ausgangssignals und zentrieren das Signal um den Nullpunkt, wie es in FA-2 gefordert ist.

Der Koppelkondensator bildet gemeinsam mit dem Eingangswiderstand des Oszilloskops von $R = 1 \text{ M}\Omega$ einen Hochpassfilter erster Ordnung. [23, S. 22] Dessen Grenzfrequenz f_g berechnet sich nach der Formel 1.

$$f_g = \frac{1}{2\pi \cdot R \cdot C} = \frac{1}{2\pi \cdot 1 \text{ M}\Omega \cdot 100 \text{ nF}} \approx 1,59 \text{ Hz} \quad (1)$$

Die Grenzfrequenz liegt damit unterhalb des in der NFA-4 festgelegten Nutzfrequenzbereichs. Die Dämpfung des Nutzsignals lässt sich über den Amplitudengang des Hochpasses nach Formel 2 bei einer Frequenz von 10 Hz bestimmen. [25, S. 241-243]

$$A = 20 \cdot \log_{10} \left(\frac{1}{\sqrt{1 + \left(\frac{f_g}{f} \right)^2}} \right) = 20 \cdot \log_{10} \left(\frac{1}{\sqrt{1 + \left(\frac{1,59 \text{ Hz}}{10 \text{ Hz}} \right)^2}} \right) \approx -0,11 \text{ dB}. \quad (2)$$

Die Dämpfung ist somit ab einer Frequenz von etwa 10 Hz vernachlässigbar. Lediglich im Bereich der niedrigsten getesteten Frequenz (1 Hz) ist mit einer merklichen Dämpfung sowie einer durch den Hochpass verursachten Phasenverschiebung zu rechnen, was bei der Bewertung der Messergebnisse in Abschnitt 5.4 entsprechend zu berücksichtigen ist.

Die Wahl eines Keramik-kondensators ist durch dessen geringe Eigenverluste (niedriger ESR), die hohe Verfügbarkeit sowie das stabile Frequenzverhalten begründet. Keramik-kondensatoren weisen über einen weiten Frequenzbereich eine niedrige Impedanz auf, sodass das Wechselspannungssignal ohne nennenswerte Verfälschung übertragen wird. [24]

Zusätzlich wurde zur Rückmeldung an den Anwender ein adressierbarer LED-Streifen des Typs WS2812B, der über GPIO 15 des Slaves angesteuert und für visuelles Feedback, sowie ein passiver Buzzer für akustische Rückmeldung, welcher über GPIO 15 des Masters angesteuert wird, integriert. Diese Komponenten dienen ausschließlich der Benutzerinteraktion und haben keinen Einfluss auf die Signalqualität. Der adressierbare LED-Streifen signalisiert dem Anwender den aktuellen Ausgabestatus. Im Stopp-Zustand leuchtet er grün, sobald eine Gruppe gestartet und ein Spannungssignal ausgegeben wird, wechselt die Farbe auf Rot. Der Buzzer erzeugt einen Ton, wenn eine Schaltfläche auf dem Display betätigt wird, sowie wenn die microSD-Karte eingeführt oder entfernt wird.

5.3. Ansteuerungslogik

Die Implementierung der Signalerzeugung und Gerätesteuerung auf dem Slave-ESP32 wurde mittels vier dedizierten C++-Dateien, einer zentralen Ino-Datei und fünf Header-Dateien umgesetzt. Die Auflistung der einzelnen Dateien sowie ihre Funktion ist in Tabelle 6 zu finden.

Die Erzeugung der periodischen Spannungssignale wird durch den Mikrocontroller übernommen, indem dieser in regelmäßigen Abständen vorberechnete Stützwerte über die zwei internen 8-Bit-DAC-Kanäle ausgibt. Die Generierung der Signalwerte erfolgt vollständig durch den Programmcode des Mikrocontrollers, während der DAC ausschließlich die Funktion eines spannungsausgebenden Endglieds übernimmt.

Die internen DACs des ESP32 sind nach dem Prinzip des sogenannten R-String-DAC aufgebaut. Eine Reihenschaltung von 256 gleich großen Widerständen liegt zwischen der Referenzspannung V_{ref} und Masse und bildet einen Spannungsteiler mit 256 Abgriffen. Ein digital gesteuertes Multiplexer-Netzwerk wählt anhand des 8-Bit-Eingangswortes den entsprechenden Abgriff aus, dessen Spannung anschließend durch einen integrierten Verstärker niederohmig am Ausgangspin bereitgestellt wird. Beide DAC-Kanäle arbeiten unabhängig voneinander und können simultan mit unterschiedlichen Werten beschrieben werden. Als Referenzspannung dient die Versorgungsspannung VDD3P3_RTC, die im Datenblatt mit 3,3 V angegeben ist. [20, S. 745-746] [21, S. 70]

Die am Ausgang anliegende Spannung berechnet sich für jeden Kanal nach Formel 3, wobei V_{ref} die Referenzspannung und $digiValue$ den geschriebenen digitalen Wert im Bereich von 0 bis 255 bezeichnet. [22]

Tabelle 6: Übersicht der Dateien für die Ansteuerung der Komponenten und Ausgabe der Spannungen des Slave-ESP32

Dateiname	Funktion
ESP32DevMod.ino	Hauptdatei. Verantwortlich für die Verknüpfung der einzelnen Unterdateien und das Ausführen der Funktionen.
dac_output.cpp und dac_output.h	Dateien für die Steuerung der Spannungsausgabe über die internen DAC-Pins, falls diese über das HMI gestartet oder gestoppt wird.
ws2812b.cpp und ws2812b.h	Dateien für die Ansteuerung des adressierbaren LED-Streifens.
save_data.cpp und save_data.h	Dateien für das Speichern der Gruppenparameter im NVS.
signal_config.h	Datei für die Definition der Signalkonfiguration der UART-Übertragung.
uart.cpp und uart.h	Dateien für die Umsetzung der UART-Kommunikation zwischen dem HMI (Master) und dem ESP32 DevKit (Slave).

$$V_{Out} = \frac{V_{ref} \cdot digiValue}{255} \quad (3)$$

Aus dieser Beziehung ergibt sich die kleinste auflösbare Spannungsstufe nach Formel 4.

$$V_{step} = \frac{V_{ref}}{2^n - 1} = \frac{3,3 \text{ V}}{255} \approx 12,9 \text{ mV} \quad (4)$$

Eine messtechnische Überprüfung der tatsächlichen Ausgangsspannung am DAC-Pin zeigte jedoch, dass die realen Spannungswerte von dieser theoretischen Berechnung abweichen. Bei einem Eingangswort von 0 wurde am Oszilloskop eine Restspannung von etwa 0,1 V gemessen. Bei einem Eingangswort von 255 erreichte die Ausgangsspannung lediglich 3,127 V anstelle der erwarteten 3,3 V. Um die Signalgenerierung ausschließlich im linearen Bereich des DAC durchzuführen, wurde der nutzbare Spannungsbereich softwareseitig auf 0 V bis 3 V begrenzt.

Da der DAC für einen gegebenen Eingangswert lediglich eine statische Spannung ausgibt, muss der Mikrocontroller die zur jeweiligen Signalform gehörenden Werte fortlaufend berechnen und in den DAC schreiben. Hierfür wurde auf dem Slave ein softwarebasierter numerischer Oszillator nach dem Phaseninkrement-Verfahren implementiert. Dieser greift auf dasselbe Grundprinzip, das auch der in Kapitel 6 beschriebenen DDS zugrunde liegt, zurück, führt die Berechnung jedoch vollständig im Programmcode auf dem ESP32 durch.

Für jeden Kanal wird hierzu eine Phasenvariable φ_{acc} eingeführt, die in jedem Durchlauf der Hauptschleife um ein Phaseninkrement entsprechend der vergangenen Zeit Δt und der gewünschten Frequenz f erhöht wird. Formel 5 zeigt die Berechnung der Phasenvariable.

$$\varphi_{acc}[n] = \varphi_{acc}[n - 1] + 2\pi \cdot f \cdot \Delta t \quad (5)$$

Die Zeitdifferenz Δt wird über die Funktion `micros()` zwischen zwei Schleifendurchläufen bestimmt, sodass keine fest definierte Sample-Rate erforderlich ist. Sobald φ_{acc} den Wert 2π überschreitet, wird der Wert der Phasenvariable um eine volle Periode reduziert, damit diese nicht unbegrenzt wächst. Anschließend wird der momentane Signalwert in Abhängigkeit der ausgewählten Signalform berechnet. Sinusförmige Signale entstehen durch die Auswertung von $\sin(\varphi_{acc})$ über die C-Standardfunktion `sinf()`, während rechteckförmige Signale durch die Auswertung des Vorzeichens des Sinus erzeugt werden. Diese Lösung kommt ohne eine separate Tabellenstruktur aus, wie sie etwa bei DDS-ICs verwendet wird. Dies geht jedoch zu Lasten der erreichbaren Ausgangsfrequenz, da pro Abtastwert eine vollständige Gleitkommarechnung durchgeführt werden muss.

Die in Anforderung FA-5 geforderte Phasenverschiebung zwischen den beiden Kanälen wird durch eine unterschiedliche Initialisierung der jeweiligen Phasenvariable realisiert. Beim Start einer Gruppe werden $\varphi_{acc,1}$, sowie $\varphi_{acc,2}$ mit den in Grad vorgegebenen und in Radiant umgerechneten Phasenwerten initialisiert. Das Vorzeichen des Phasenwertes wird dabei invertiert, sodass ein positiver Vorgabewert einer Nacheilung des betreffenden Kanals entspricht. Anschließend wird die Zeitbasis für beide Kanäle neu gesetzt, sodass beide Kanäle synchron starten und der vorgegebene

Phasenversatz exakt eingehalten wird.

Der interne DAC kann ausschließlich positive Spannung im Bereich von 0 V bis 3 V (nach durchgeführter Kalibrierung) ausgeben. Der Laborversuch benötigt jedoch um den Nullpunkt symmetrische Signale. Um dies zu erreichen, wird der Mittelwert des Aussteuerungsbereichs als virtueller Nullpunkt definiert. Da der DAC den ganzzahligen Wertebereich von 0 bis 255 abbildet, liegt dieser rechnerische Mittelwert bei 127,5. Alle Signalwerte schwingen symmetrisch um diesen Mittelwert, sodass am DAC-Pin ein Wechselnungssignal anliegt, das mit einem Gleichspannungsoffset von etwa 1,5 V überlagert ist. Ein in Reihe liegender Kondensator blockiert den Gleichspannungsanteil und gibt nur den Wechselspannungsanteil an den Ausgang weiter. Damit steht eine nutzbare Amplitude von $\pm 1,5$ V um den Nullpunkt am Ausgang des Funktionsgenerators zur Verfügung. Die Umrechnung der gewünschten Amplitude in DAC-Schritte erfolgt nach Formel 6 mit $V_{max} = 1,5$ V.

$$counts = \frac{V_{soll}}{V_{max}} \cdot 127,5 \quad (6)$$

Amplituden, die diesen Wert überschreiten, werden auf den maximalen Hub begrenzt. Der schließlich an den DAC übergebene digitale Wert berechnet sich nach Formel 7:

$$digiValue = \text{round}(127,5 + counts \cdot \text{wave}(\varphi_{acc})) \quad (7)$$

Der Term $\text{wave}(\varphi_{acc})$ liefert den momentanen Wert der Wellenform im Bereich von -1 bis $+1$ und entspricht, je nach gewählter Signalform, entweder dem Sinuswert selbst oder dessen Vorzeichen. Da der DAC ausschließlich ganzzahlige Eingangswerte verarbeiten kann, wird der intern berechnete Wert vor der Übergabe an die Funktion `dacWrite()` durch die Funktion `roundf()` auf die nächste ganze Zahl gerundet. Eine zusätzliche Begrenzung mittels `constrain()` auf den Bereich von 0 bis 255 stellt sicher, dass auch bei Rundungsfehlern an den Rändern des Aussteuerungsbereichs keine ungültigen Werte an den DAC übergeben werden. Die vollständige Funktion zur DAC-Ausgabe ist in Listing 3 auf Seite 61 im Anhang dargestellt.

Im Stopp-Zustand werden beide DAC-Kanäle auf den nächstliegenden ganzzahligen Mittelwert 128 zurückgesetzt, sodass nach dem in Reihe liegenden Kondensator eine Ausgangsspannung von 0 V anliegt. Abbildungen 14 und 15 auf den Seiten 62 und 63 im Anhang zeigen exemplarisch die beiden Ausgangssignale bei unterschiedlichen Ausgangsfrequenzen.

Die hier beschriebene Kalibrierung beschränkt sich somit auf die softwareseitige Anpassung des nutzbaren Aussteuerungsbereichs auf den linearen Arbeitsbereich des DACs. Eine darüber hinausgehende messtechnische Bewertung der erzielten Signalqualität, insbesondere der verbleibenden Amplituden- und Frequenzabweichungen, erfolgt im Rahmen des Abschnitts 5.4.

5.4. Messaufbau, Testversuche und Genauigkeitsbewertung

Zur Bewertung der in Abschnitt 5.3 beschriebenen Signalerzeugung wurde eine systematische Messreihe durchgeführt. Ziel dieser Messung ist die Überprüfung der in Abschnitt 3.3 aufgelisteten

nicht-funktionalen Anforderungen. Insbesondere liegt der Fokus auf den ersten drei NFA sowie die Identifizierung möglicher Abweichungen und deren Ursachen. Die Ergebnisse bilden zugleich die Datenbasis für den späteren Vergleich der beiden Implementierungsmethoden in Kapitel 9.

Für die Erfassung der Ausgangssignale wurde ein Keysight InfiniiVision 2000 X-Series Oszilloskop verwendet, wie es auch in den Laborräumen der HWR Berlin für die in Abschnitt 3.1 beschriebenen Versuche eingesetzt wird. Damit erfolgt die Validierung des Funktionsgenerators unter den späteren Einsatzbedingungen. Die relevanten Spezifikationen des Messgeräts sind in Tabelle 7 zusammengefasst.

Tabelle 7: Spezifikationen des verwendeten Oszilloskops (Keysight InfiniiVision 2000 X-Series) [23, S. 4-5]

Parameter	Wert
Bandbreite	70 MHz
Abtastrate (interleaved)	2 GSa/s
Abtastrate (pro Kanal)	1 GSa/s
Anzahl analoger Kanäle	2
Vertikale Auflösung	8 bit

Die Bandbreite und Abtastrate des Oszilloskops liegen mehrere Größenordnungen über den vom Funktionsgenerator erzeugten Signalen. Eine Beeinträchtigung der Messgenauigkeit durch das Oszilloskop ist daher sehr gering [23, S. 5].

Die Messreihe wurde so konzipiert, dass alle drei zu prüfenden nicht-funktionalen Anforderungen (Amplituden-, Frequenz- und Phasengenauigkeit) systematisch erfasst werden. Hierzu wurden die getesteten Signalformen (Sinus und Rechteck) für mehrere Ausgangsfrequenzen von 1 Hz bis 10 kHz untersucht. Innerhalb jeder Frequenzstufe wurde die Soll-Amplitude in Schritten von 100 mV bis 1500 mV variiert. Die Soll-Phase zwischen Kanal 1 und Kanal 2 betrug bei allen Messungen 90°. Die einzelnen Parametersätze wurden nacheinander ausgegeben. Zur Erfassung der Ist-Werte wurden die Messwerte mithilfe der automatischen Messfunktion des Oszilloskops unter Verwendung der DC-Kopplung ausgelesen.

Insgesamt wurden auf diese Weise 150 Messpunkte erfasst. Für jede der getesteten Frequenzstufen wurden 15 Amplitudenstufen, sowohl für ein sinus- als auch für ein rechteckförmiges Signal, aufgenommen. Die ausgelesenen Ist-Werte wurden gemeinsam mit den zugehörigen Soll-Werten tabellarisch erfasst und bilden die Grundlage der nachfolgenden Auswertung. Zur Bewertung der einzelnen Anforderungen wird im Folgenden jeweils der relative Fehler zwischen Soll- und Ist-Wert betrachtet.

Amplitudengenauigkeit (NFA-1):

Die Amplitudenmessung wurde mittels der Top-Messung durchgeführt. Im Anhang auf den Seiten 64 und 65 zeigen die Abbildungen 16 und 17 den relativen Amplitudenfehler in Abhängigkeit der eingestellten Soll-Amplitude, getrennt nach sinus- und rechteckförmigem Signal. Für beide Signalformen wird anhand der Messdaten ein klarer Trend erkennbar.

Mit zunehmender Soll-Amplitude sinkt der relative Fehler deutlich ab. Beim Sinussignal ist zu erkennen, dass die relative Abweichung bei niedrigen Amplituden bis zu rund 80 % betragen kann. Sowohl im Mittel als auch im Median liegen die Messwerte bei einer Soll-Amplitude von 100 mV etwa 79 % oberhalb des vorgegebenen Wertes. Dies stellt eine erhebliche Verletzung der Anforderung NFA-1 dar, weshalb dieser Bereich für den Anwendungsfall ausscheidet.

Ab einer Soll-Amplitude von 200 mV weichen Mittel- und Medianwert deutlich voneinander ab. Während die mittlere Abweichung bei 200 mV und 300 mV mit rund 7 % bis 8 % oberhalb des Toleranzbereichs liegt, beträgt die mediane Abweichung lediglich etwa 1,5 % bzw. 2,7 % und liegt damit innerhalb der Toleranz. Diese Differenz entsteht dadurch, dass der Mittelwert durch einzelne Ausreißer nach oben verzerrt wird, während der Median als robustes Lagemaß von solchen Einzelwerten nahezu unbeeinflusst bleibt. Verantwortlich für diese Ausreißer sind in erster Linie die Messwerte bei der Ausgangsfrequenz von 1 Hz, deren Signalfrequenz im Bereich der Grenzfrequenz des Koppelkondensators liegt und die daher nur eingeschränkt zuverlässig erfasst werden konnten.

Über den gesamten Bereich ab 200 mV liegt die mediane Abweichung jeder Amplitudenstufe innerhalb der geforderten ± 5 %-Toleranz und erreicht einen maximalen Wert von rund 4,4 % bei einer Soll-Amplitude von 500 mV. Das typische Messverhalten erfüllt die Anforderung NFA-1 in diesem Bereich somit zuverlässig, und die nach Mittelwert auftretenden Überschreitungen bei 200 mV, 300 mV und 500 mV beschränken sich auf einzelne Ausreißer.

Beim Rechtecksignal fällt der relative Fehler über den gesamten Amplitudenbereich deutlich geringer aus. Selbst bei der kleinsten Soll-Amplitude von 100 mV beträgt die mittlere Abweichung lediglich rund 5 % (maximal etwa 9 %). Der Median liegt hier mit rund 4 % bereits innerhalb der Toleranz, ohne den beim Sinus beobachteten starken Ausreißer aufzuweisen. Über den übrigen Amplitudenbereich bewegen sich die medianen Abweichungen durchgehend zwischen etwa 2,7 % und 4,6 % und damit vollständig innerhalb der Toleranz, während die Mittelwerte vereinzelt knapp darüber liegen.

Dieses unterschiedliche Verhalten der beiden Signalformen lässt sich wahrscheinlich auf die Art der Signalerzeugung zurückführen. Während das sinusförmige Signal treppenförmig aus einer Vielzahl von DAC-Stufen zusammengesetzt wird, nutzt das Rechtecksignal lediglich zwei feste Pegel, den oberen und den unteren DAC-Wert. Die Amplitude ergibt sich somit unmittelbar aus der Differenz dieser beiden Pegel und wird auch bei kleinen Amplituden zuverlässig erfasst.

Insgesamt liegen 71 der 90 sinusförmigen und 50 der 60 rechteckförmigen Messpunkte innerhalb der geforderten Toleranz von ± 5 %. Die Anforderung NFA-1 wird damit über weite Teile des nutzbaren Aussteuerungsbereichs erfüllt. Im Bereich sehr kleiner Amplituden, insbesondere bei $V_{out} \leq 100$ mV im Sinusbetrieb, jedoch deutlich verletzt. Da die den einzelnen Arbeitsgruppen zugewiesenen Amplitudenwerte gemäß NFA-5 frei festgelegt werden können, lässt sich dieser kritische Bereich im praktischen Einsatz umgehen. Für den vorgesehenen Anwendungsfall kann die Anforderung NFA-1 daher als erfüllbar betrachtet werden, wobei die eingeschränkte Genauigkeit bei kleinen Amplituden als bauteilbedingte Limitierung der DAC-basierten Methode festzuhalten ist.

Phasengenauigkeit (NFA-2):

Abbildung 18 auf Seite 66 im Anhang zeigt die absolute Phasenabweichung von der vorgegebenen Phasenverschiebung von 90° in Abhängigkeit der Ausgangsspannung und Frequenz. Der Phasenfehler zeigt keinen ausgeprägten Zusammenhang mit der eingestellten Amplitude, sondern variiert in erster Linie mit der Frequenz. Die geforderte Toleranz von $\pm 2^\circ$ wird nur in einem mittleren Frequenzbereich zuverlässig eingehalten, während die Abweichungen zu niedrigen und hohen Frequenzen hin zunehmen.

Beim Rechtecksignal sind die Ergebnisse im unteren Frequenzbereich am besten. Bei 10 Hz und 100 Hz liegen alle Messpunkte innerhalb der Toleranz (max. $0,12^\circ$ bzw. $0,63^\circ$). Mit steigender Frequenz nimmt der Fehler deutlich zu. Bei 500 Hz steigt der Fehler auf rund 4° (8 von 15 Punkten konform), bei 1 kHz auf etwa $9,5^\circ$ (nur 2 von 15 Punkten konform).

Beim Sinussignal wird die geringste Abweichung im mittleren Frequenzbereich erreicht. Bei 100 Hz liegen 13 von 15 Messpunkten innerhalb der Toleranz (max. $2,43^\circ$). Zu niedrigen Frequenzen hin steigt der Fehler auf bis zu $6,47^\circ$ (10 Hz) und $8,75^\circ$ (1 Hz). Bei höheren Frequenzen von 500 Hz, 1 kHz und 2 kHz liegt der Fehler bei bis zu $4,87^\circ$, $3,74^\circ$ bzw. $4,61^\circ$.

Zusätzlich wächst die Phasenabweichung im Dauerbetrieb langsam an, wodurch nach mehr als 10 Minuten eine signifikante Abweichung entsteht. Ursache dafür ist die fortlaufende numerische Akkumulation der Phasenwerte. Eine Korrektur durch periodisches Zurücksetzen beider Phasenwerte auf den Sollwert wäre möglich, würde jedoch einen kurzen Aussetzer im Signalverlauf des Oszilloskops verursachen. Somit gilt die FA-5 nur als teilweise erfüllt.

Insgesamt erfüllen 50 der 90 sinus- und 40 der 60 rechteckförmigen Messpunkte die Anforderung NFA-2. Die geforderte Phasengenauigkeit von $\pm 2^\circ$ kann durch die DAC-basierte Methode somit nicht über den gesamten Parameterbereich, sondern nur innerhalb eines mittleren Frequenzbandes sichergestellt werden. Dies schränkt den Einstellbereich für die Phasenverschiebung ein. Innerhalb dieses eingeschränkten Bereichs wird die NFA-2 jedoch erfüllt.

Frequenzgenauigkeit (NFA-3):

Die Frequenzgenauigkeit der DAC-Kanäle erweist sich als sehr hoch und stabil. Die maximale prozentuale Abweichung liegt beim Sinussignal bei ca. 2,3 % und beim Rechtecksignal bei gerade einmal 0,84 %. Die geforderte Toleranz von ± 5 % wurde damit von allen Messwerten vollständig eingehalten. Abbildung 19 auf Seite 67 im Anhang stellt die Messergebnisse dar. Die Anforderung NFA-3 ist somit vollumfänglich erfüllt.

Anzumerken ist, dass die rechteckförmigen Signale nicht über denselben Frequenzbereich wie die sinusförmigen Signale vermessen werden konnten. Bei einer Frequenz von 1 Hz wies das Ausgangssignal zu Beginn jeder Halbperiode deutlich überhöhte Spannungswerte auf und flachte anschließend über die Halbperiode hinweg merklich ab, sodass am Oszilloskop kein rechteckförmiger Signalverlauf, sondern eher abklingende Spitzen zu erkennen waren. Abbildung 20 auf der Seite 68 im Anhang stellt genau dieses Spannungssignal dar. Eine zuverlässige Erfassung von Amplitude und Phase war unter diesen Bedingungen nicht möglich. Vermutlich ist dieses Verhalten auf den in Abschnitt 5.2 beschriebenen Koppelkondensator zurückzuführen, dessen Grenzfrequenz mit rund 1,59 Hz im Bereich der Signalfrequenz liegt. Aus diesem Grund wurde für das Rechtecksignal auf

eine Messung bei 1 Hz verzichtet.

Oberhalb von etwa 1 kHz konnte die Phasenverschiebung des Rechtecksignals ebenfalls nicht mehr sinnvoll abgelesen werden. Die Darstellung am Oszilloskop war in diesem Bereich nicht mehr stabil, sondern sprang fortlaufend hin und her, sodass weder die Phase noch die Amplitude zuverlässig bestimmt werden konnten.

Die Messreihe des rechteckförmigen Signals beschränkt sich daher auf den Bereich von 10 Hz bis 1 kHz. Beim sinusförmigen Signal trat ab einer Frequenz von etwa 2 kHz eine zunehmend treppenförmige Verzerrung des Signalverlaufs auf. Da die Stützwerte des Signals fortlaufend durch den Programmcode berechnet und an den DAC übergeben werden, nimmt die Anzahl der pro Periode ausgegebenen Stützwerte mit steigender Frequenz ab. Es liegt daher nahe, dass die zunehmende Treppenform auf die begrenzte Anzahl an Stützwerten pro Periode in diesem Frequenzbereich zurückzuführen ist. Abbildung 14b auf Seite 62 im Anhang zeigt den stufenförmigen Signalverlauf bei einer Frequenz von 2 kHz.

Reaktionstest (NFA-6):

Zur Validierung der Anforderung NFA-6 wurde mit Hilfe eines Logic Analyzers ein Reaktionstest durchgeführt. Der zugehörige Messaufbau ist in Abbildung 21 auf Seite 69 im Anhang dargestellt. Für die Messung wurde ein dedizierter GPIO-Pin des Masters verwendet, der auf High gesetzt wird, sobald das Display eine Berührung erkennt. Dieser Trigger-Pin, das DAC-Ausgangssignal sowie die beiden UART-Leitungen wurden an die Kanäle des Logic Analyzers angeschlossen. Die Zeitdifferenz zwischen der steigenden Flanke des Trigger-Pins und der daraus resultierenden Parameteränderung am DAC-Ausgang entspricht der internen Reaktionszeit des Systems. Die Ausgabe wurde Softwareseitig so angepasst, dass der DAC-Ausgang ein konstantes 1 V Signal auf Kanal 1 ausgibt.

Die Touch-Polling-Rate des Displays ist in der Konfigurationsdatei `lv_conf.h` (Zeile 83) auf 30 ms vordefiniert. Das Display prüft somit alle 30 ms, ob ein Touch-Ereignis aufgetreten ist. Zur Bestimmung der Reaktionszeit wird daher vom ungünstigsten Fall ausgegangen, bei dem die Berührung unmittelbar nach einem Abfragezyklus erfolgt und somit eine zusätzliche Latenz von 30 ms entsteht. Diese Polling-Rate ließe sich grundsätzlich erhöhen, um die Reaktionszeit weiter zu senken, allerdings zulasten der verfügbaren CPU-Leistung.

Tabelle 8 fasst die Ergebnisse von fünf Einzelmessungen zusammen. Die am Logic Analyzer gemessene Verarbeitungs- und Ausgabezeit (t_{Ausgang}) liegt in allen Fällen unter 2,1 ms. Zusammen mit der Worst-Case-Polling-Latenz von 30 ms (t_{Touch}) ergibt sich eine maximale Reaktionszeit von

Tabelle 8: Reaktionszeitmessung zur Überprüfung der Anforderung NFA-6 (DAC)

Messung	t_{Touch} [ms]	t_{Ausgang} [ms]	Reaktionszeit [ms]	NFA-6 erfüllt? ($\Delta t < 100$ ms)
1	30	2,05	32,05	Ja
2	30	1,70	31,70	Ja
3	30	1,75	31,75	Ja
4	30	1,86	31,86	Ja
5	30	1,73	31,73	Ja

rund 32 ms. Die geforderte Obergrenze von 100 ms wird damit deutlich unterschritten, sodass die Anforderung NFA-6 erfüllt ist. Abbildung 22 auf der Seite 70 im Anhang zeigt die Auswertung vom Logic Analyzer für einen der Reaktionstests.

6. Implementierungsmethode B: Signalgenerierung mittels Direct Digital Synthesis

6.1. Konzept und Komponentenauswahl

Wie in Kapitel 2.3 bereits angemerkt, gibt es heutzutage grundsätzlich zwei etablierte Möglichkeiten, die für die Generierung von Spannungssignalen genutzt werden. Neben der in Kapitel 5 beschriebenen Variante mittels interner Digital-Analog-Wandler gibt es zudem die Möglichkeit, die Signalgenerierung über Direct Digital Synthesis, beispielsweise über einen externen DDS-IC, umzusetzen. Wie bereits erwähnt, erfolgen bei der DDS die Signalgenerierung und die Digital-Analog-Wandlung meist in einem einzelnen dedizierten Baustein. Ist dies der Fall, spricht man von einem sogenannten vollständigen DDS-Modul. [11, S. 2] Der Grundaufbau eines DDS-Moduls besteht aus einem Phasenakkumulator, einer Möglichkeit, die Phasenvorgabe in eine Amplitude umzuwandeln und einem Digital-Analog-Wandler. [11, S. 1] Dieser Aufbau ist in Abbildung 6 dargestellt.

Ein DDS-Modul erzeugt ein Sinussignal mit einer bestimmten Frequenz. Die Frequenz ist dabei von zwei Parametern, der Frequenz der reference-clock und dem in Punkt 2.3 bereits erwähnten tuning-word, abhängig. Der Phasenakkumulator erzeugt bestimmte Phasenwerte, die als Adressen für eine Sinus-Lookup-Tabelle verwendet werden. Die Tabelle liefert zu jedem Phasenwert den entsprechenden digitalen Amplitudenwert eines Sinussignals, welcher anschließend durch einen DAC in eine analoge Spannung oder einen Strom umgewandelt wird. Die Ausgangsfrequenz des Signals wird durch das Phaseninkrement, welches wiederum durch das tuning-word bestimmt wird, festgelegt. Große Inkremente erzeugen dabei hohe und kleine Inkremente niedrige Frequenzen. [11, S. 1] Der Phasenakkumulator agiert innerhalb eines DDS-Moduls als Phasenkreis mit mehreren festgelegten Punkten (Abbildung 7). Jeder dieser definierten Punkte entspricht dabei einem bestimmten Punkt einer Sinusperiode. Während sich ein Vektor mit konstanter Geschwindigkeit um den Kreis bewegt, erzeugt der Sinus des jeweiligen Winkels des Vektors das entsprechende Ausgangssignal. Eine vollständige Umdrehung des Vektors entspricht dabei genau einer vollständigen Periode der Sinuswelle. Das tuning-word legt dabei die Schrittweite M auf dem Phasenkreis fest. Große Schrittweiten führen dazu, dass der Phasenakkumulator schneller überläuft und somit höhere Ausgangsfrequenzen erzeugt werden. Die Anzahl der möglichen Phasenpunkte wird durch die Auflösung des Phasenakkumulators n bestimmt und beeinflusst direkt die Frequenzauflösung des DDS-Systems. [11, S. 2]

Für die Umsetzung dieses Projekts wurde ein DDS-Modul auf Grundlage des programmierbaren

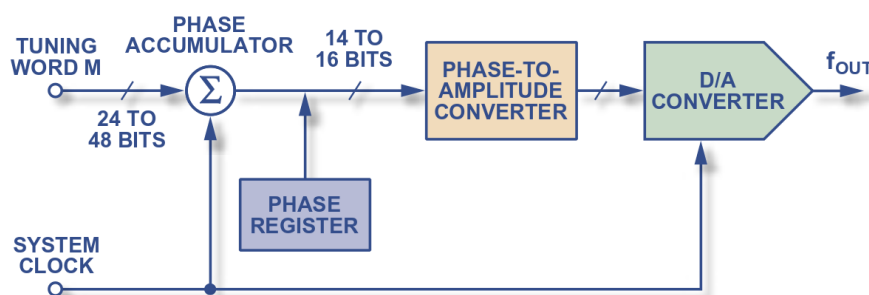


Abbildung 6: Bestandteile eines Direct Digital Synthesisers [11, S. 1]

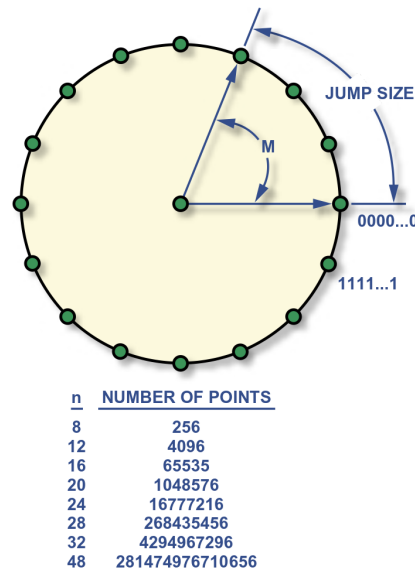


Abbildung 7: Digitaler Phasenkreis eines DDS-ICs [11, S. 2]

Wellenformgenerators AD9833 der Firma Analog Devices ausgewählt. Der AD9833 ist ein weitverbreiteter und kostengünstiger DDS-IC mit einer Ausgangsfrequenz von 0 bis 12,5 MHz und einer Auflösung von 28-Bit. [26, S. 1] Angesteuert wird der IC mittels einer 3-Draht-Schnittstelle über SPI, QSPI, MICROWIRE oder DSP. [26, S. 13] Das verwendete AD9833-Modul ist in Abbildung 8 zu sehen.

Der AD9833 eignet sich sowohl für die Generierung von sinus- als auch rechteckförmigen Spannungssignalen, wie unter FA-2 gefordert. Ausgestattet ist der IC mit zwei 28-Bit-Frequenzregistern (FREQ0 und FREQ1), zwei 16-Bit-Phasenregistern (PHASE0 und PHASE1) sowie einem 10-Bit-DAC, über welchen das analoge Spannungssignal ausgegeben wird. [26, S. 1] Ein funktionales Blockdiagramm des Aufbaus ist in Abbildung 23 auf der Seite 71 im Anhang zu finden. Insgesamt besitzt der AD9833 10 Pins, von denen jedoch nur sieben über die Anschlüsse des Moduls zur Verfügung stehen. Die Pins sowie die ihnen zugewiesenen Funktionen sind in Tabelle 9 aufgelistet.

Identisch zur obigen Beschreibung der Funktionsweise eines DDS-Moduls verwendet der AD9833 das im Frequenzregister gespeicherte Frequenzwort sowie den Takt des auf dem Modul vorhandenen 25 MHz Quarz, um die Ausgangsfrequenz des Spannungssignals zu verändern.

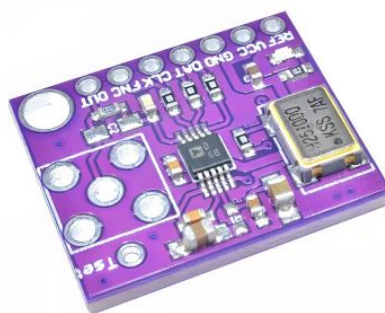


Abbildung 8: AD9833-Modul mit integriertem 25 MHz Quarz [27]

Berechnet wird die Ausgangsfrequenz dabei durch die Formel:

$$\frac{f_{MCLK}}{2^{28}} \cdot FREQREG \quad (8)$$

Dabei stehen f_{MCLK} für die Taktfrequenz des Quarzes und $FREQREG$ für den im ausgewählten Frequenzregister gespeicherten Wert. Dieses Signal wird um eine bestimmte Phase, die sich aus der nachfolgenden Gleichung berechnet, verschoben.

$$\frac{2 \cdot \pi}{4096} \cdot PHASEREG \quad (9)$$

$PHASEREG$ steht dabei für den Wert, der im ausgewählten Phasenregister gespeichert ist. [26, S. 15] Insgesamt besitzt der IC 15 Kontrollbits, welche unter anderem für die Auswahl der Datenregister, das Zurücksetzen des ICs, das Aktivieren des Sleep-Modus und das Umschalten des Ausgangssignals verwendet werden. Für die Auswahl der Frequenzregister werden Bit 15 und Bit 14 verwendet [(01)₂ steht dabei für FREQ0 und (10)₂ für FREQ1]. Für die Auswahl des Phasenregisters sind Bit 15, Bit 14 und Bit 13 verantwortlich [(110)₂ steht dabei für PHASE0 und (111)₂ für PHASE1]. Ist der FSYNC-Pin Low und entspricht Bit 13 dem Wert Eins, können die Daten in die Register getaktet werden. [26, S. 13-15] Für das Umschalten zwischen sinus- und rechteckförmiger Ausgangsspannung werden Bit 5, Bit 3 und Bit 1 verwendet. Durch den Zustand (0X0)₂, wobei X für einen beliebigen Wert steht, wird ein sinusförmiges Signal ausgegeben. Durch (110)₂ wird das Most Significant Bit (MSB) des DACs, was einem Rechtecksignal entspricht, am Ausgang ausge-

Tabelle 9: Pinbeschreibung des AD9833-Moduls [26, S. 6]

Pin	Verwendung
OUT	Spannungsausgang. An diesem Pin steht der analoge bzw. digitale Ausgang des AD9833 zur Verfügung.
GND	Masse
FSYNC	FSYNC dient als Synchronisationssignal für die Eingangsdaten. Wird FSYNC auf Low gesetzt, erkennt die interne Logik, dass ein neues Datenwort in das Gerät geladen werden soll.
SCLK	Serieller Takteingang. Die Daten werden mit jeder fallenden Flanke von SCLK in den AD9833 eingetaktet.
SDATA	Serieller Dateieingang. Über diesen Eingang wird das 16-Bit-Datenwort an den AD9833 übertragen.
REF	Über diesen Pin kann das Taktsignal der reference-clock, welches am MCLK-Pin anliegt, abgegriffen werden.
VCC	Positive Versorgungsspannung.

geben. Durch Bit 7 und Bit 6 kann der Chip in einen Sleep-Modus versetzt werden. [26, S. 16] Der IC ermöglicht es jedoch nicht, die Ausgangsspannung über die Kontrollbits zu verändern. Laut Datenblatt liefert der AD9833 bei der Ausgabe eines sinusförmigen Signals 0,65 V als maximale und 38 mV als minimale Ausgangsspannung. [26, S. 3] Da bei der Rechteckausgabe das MSB des DACs ausgegeben wird, entspricht die Ausgangsspannung ca. der Versorgungsspannung (2,3 V bis 5,5 V). [26, S. 1] Dabei ist zu erwähnen, dass die Ausgangsspannung neben dem ausgewählten Modus ebenfalls von der Frequenz des Ausgangssignals abhängt. Die Ausgangsspannung des AD9833 in Abhängigkeit der Ausgangsfrequenz und des ausgewählten Modus ist in Abbildung 24 auf Seite 72 im Anhang zu sehen. Zudem kann für die Umsetzung des Projektes nicht der vollständige Frequenzbereich des ICs verwendet werden, da die Verzerrung des Ausgangssignals mit zunehmender Ausgangsfrequenz immer weiter zunimmt. Dies ist in Abbildung 25 auf Seite 73 im Anhang zu sehen. Dadurch sinkt die maximale Ausgangsfrequenz auf ca. 1 MHz, da bei dieser Frequenz sowohl das sinusförmige als auch das rechteckförmige Spannungssignal weitestgehend unverzerrt bleiben. Dies würde weiterhin die unter NFA-4 gestellten Anforderungen an den einstellbaren Frequenzbereich erfüllen.

Um einen für den Laborversuch gemäß NFA-5 geeigneten Spannungsbereich bereitzustellen, müssen in beiden Betriebsvarianten identische Spannungswerte erreicht werden. Zudem soll der verwendete Slave eine flexible Anpassung der Ausgangsspannung ermöglichen. Um sowohl im Sinus- als auch im Rechteckbetrieb eine identische Ausgangsspannung zu erreichen, wird eine Verstärkerschaltung mittels eines Operationsverstärkers (OPV) verwendet. Dieser kann die Signale auf einen identischen und gleichzeitig besser nutzbaren Spannungswert verstärken. Um die Ausgangsspannung der Verstärkerschaltung anpassen zu können, wird die Eingangsspannung des OPVs über einen Spannungsteiler verändert. Um das Verhältnis des Spannungsteilers durch den Programmcode steuern zu können, wurden digitale Potentiometer, welche über eine serielle Schnittstelle angesteuert werden, verwendet.

Zu Beginn des Entwicklungsprozesses fiel die Entscheidung zunächst auf die digitalen Potentiometer DS1803-100 der Firma Analog Devices. Diese wurden jedoch im weiteren Entwicklungsprozess durch Potentiometer des Typs MCP42010 der Firma Microchip Technology ersetzt. Warum diese Entscheidung getroffen wurde, wird in den nachfolgenden Abschnitten erklärt. Grundsätzlich handelt es sich bei beiden ICs um kostengünstige und leicht verfügbare digitale Potentiometer mit 256 möglichen Einstellpositionen, von denen jeweils zwei pro IC verbaut sind (Pot-0 und Pot-1). [28, S. 1] [29, S. 1]

Unterschiede weisen sie jedoch in ihrer Ansteuerung und elektrischen Eigenschaften auf. Die Potentiometer des Typs DS1803-100 werden über zwei Leitungen mittels I²C angesteuert und haben einen Gesamtwiderstand von 100 k Ω . Über die Pins A0, A1 und A2 des ICs kann die Adresse im I²C-Bus festgelegt werden. Über die Datenleitung (SDA) werden dann mit jedem Takt der seriellen Clock am SCL-Pin die einzelnen Datenbits an die zwei 8-Bit-Register des ICs übertragen. Hat der IC alle Datenbits erhalten, bestätigt er auf der Datenleitung, dass die Übertragung erfolgreich war. Für die Ansteuerung des ICs wird zunächst das allgemeine Kontrollbyte (0101)₂ gefolgt von der Adresse des Chips und einer Null für das Schreiben oder einer Eins für das Lesen der Register übersendet. Als nächstes wird das zu beschreibende Potentiometer ausgewählt. Dabei steht

$(10101001)_2$ für Pot-0, $(10101010)_2$ für Pot-1 und $(10101111)_2$ für Pot-0 und Pot-1. Zuletzt wird das Datenbyte und somit der neue Wert des Potentiometers übertragen. [28, S. 1-10]

Die Potentiometer des Typs MCP42010 werden hingegen mittels SPI angesteuert. Angesprochen wird der IC in diesem Fall nicht über eine Busadresse, sondern über den Chip-Select-Pin (CS-Pin). Ist der CS-Pin Low, können die Datenbits über die Datenleitung am SI-Pin und das Taktsignal am SCK-Pin in das 16-Bit-Schieberegister geschrieben werden. Bei der Datenübertragung wird zunächst das Kontrollbyte übertragen. Über dieses Kontrollbyte kann bestimmt werden, ob der IC abgeschaltet $(XX10XX)_2$ oder Daten in das Register geschrieben werden sollen $(XX01XX)_2$. Der Buchstabe X steht dabei erneut für einen beliebigen Wert. Zudem wird über die Endung des Kontrollbytes das zu verändernde Potentiometer ausgewählt. Dabei steht die Endung $(01)_2$ für Pot-0, die Endung $(10)_2$ für Pot-1 und die Endung $(11)_2$ für Pot-0 und Pot-1. Im Gegensatz zu den DS1803-100 besitzen die MCP42010 einen Gesamtwiderstand von 10 k Ω . [29, S. 1-18]

Die elektrischen Eigenschaften der beiden Potentiometer weichen ebenfalls leicht voneinander ab. Die typische Versorgungsspannung der DS1803-100 Potentiometer liegt zwischen 3 V und 5 V, wohingegen die der MCP42010 zwischen 2,7 V und 5,5 V liegt. [28, S. 1] [29, S. 1] Im Gegensatz zu mechanischen Potentiometern sind beide ICs in ihrer minimalen und maximalen Spannung, die an den Kontakten anliegen kann, beschränkt. Bei den Potentiometern des Typs DS1803-100 beläuft sich die minimale Spannung auf $GND - 0,5$ V und die maximale Spannung auf $V_{CC} + 0,5$ V. [28, S. 6] Bei den Potentiometern des Typs MCP42010 liegt die minimale Spannung bei $GND - 0,6$ V und die maximale Spannung bei $V_{CC} + 1$ V. [29, S. 5] Somit sind beide Typen in der Lage, die Ausgangssignale des AD9833 weiterzuleiten und können für den Spannungsteiler vor der Verstärkerschaltung verwendet werden.

Für die Verstärkerschaltung selbst wird ein Operationsverstärker des Typs OPA1612 der Firma Texas Instruments verwendet. Bei diesem handelt es sich um einen kostengünstigen aber dennoch leistungsstarken High-Performance Bipolar-Input Audio OPV. [30, S. 1] Die elektrischen sowie nichtelektrischen Eigenschaften dieses Bauteils sind in Tabelle 10 aufgelistet.

Dieser OPV eignet sich durch seine hohe Bandbreite und Slew Rate in Kombination mit der geringen Rauschdichte und Verzerrung ideal, um hochfrequente Spannungssignale ohne einen

Tabelle 10: Elektrische und nichtelektrische Eigenschaften des OPA1612 der Firma Texas Instruments [30, S. 1-4]

Elektrische Eigenschaften	
Bandbreite (GBW)	40 MHz
Rauschdichte	1,1 nV/ $\sqrt{\text{Hz}}$ bei 1 kHz
Verzerrung	$1,5 \cdot 10^{-5} \%$ bei 1 kHz
Slew Rate	27 V/ μs
Versorgungsspannung	$\pm 2,25$ V bis ± 18 V
Versorgungsstrom	3,6 mA
Nichtelektrische Eigenschaften	
Anzahl an Verstärkern je IC	2
Anzahl an Kontakten	8

signifikanten Qualitätsverlust zu verstärken. Betrieben wird der OPV als ein nicht invertierender Verstärker. Eine derartige Verstärkerschaltung ist in der Abbildung 9 zu sehen. Dabei wird die am nicht invertierenden Eingang anliegende Spannung U_e um einen bestimmten Faktor V verstärkt und steht dann als verstärktes Spannungssignal U_a am Ausgang des OPVs zur Verfügung. Der Verstärkungsfaktor V berechnet sich dabei durch folgende Formel:

$$V = \frac{U_a}{U_e} = 1 + \frac{R_1}{R_2} \quad (10)$$

Durch die Veränderung des Verhältnisses zwischen den beiden Widerständen R_1 und R_2 kann nach dieser Formel also die Verstärkung des OPVs eingestellt werden. [25, S. 235]

Für das akustische und visuelle Feedback wird erneut ein passiver Buzzer sowie ein adressierbarer LED-Streifen des Typs WS2812B verwendet.

6.2. Schaltungsentwicklung

Nach der Auswahl der benötigten Komponenten für die Realisierung des Funktionsgenerators mittels DDS wird in diesem Abschnitt der eigentliche Schaltungsaufbau beschrieben. Hierbei wird der finale Schaltungsaufbau unter Verwendung der digitalen Potentiometer des Typs MCP42010 referenziert. Der vollständige Schaltplan ist in den Abbildungen 26 bis 29 auf den Seiten 74 bis 77 im Anhang zu finden.

Analog zur im Kapitel 5 beschriebenen Variante werden die Parameter der einzelnen Gruppen über den Master, also das Waveshare ESP32-S3 HMI, mittels einer microSD-Karte eingelesen und über eine UART-Schnittstelle an den Slave, also das ESP32 Devkit C V4, übertragen. Diese Verbindung ist auf Abbildung 26 auf der Seite 74 im Anhang zu erkennen. Der Master (U1) sowie der Slave (U2) sind über die jeweiligen UART-Anschlüsse der einzelnen Boards miteinander verbunden. Dabei ist die TX-Datenleitung des Masters (GPIO 43) mit der RX-Datenleitung des Slaves (GPIO 16) und die RX-Datenleitung des Masters (GPIO 44) mit der TX-Datenleitung des Slaves (GPIO 17) verbunden. Über ein 9 V bis 36 V auf 5,2 V DC-DC Buck Converter werden beide Komponenten mit Spannung versorgt. Mittels GPIO 15 steuert der Master den Buzzer, welcher ein akustisches Feedback für Eingaben auf dem Display ausgibt. Alle weiteren Komponenten werden

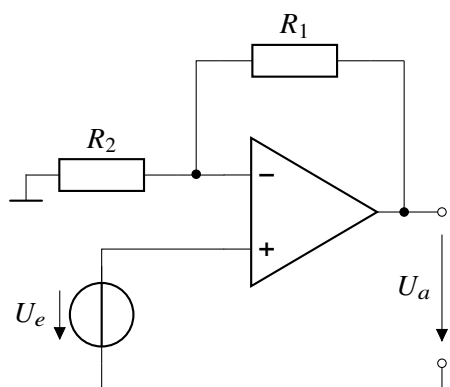


Abbildung 9: Nicht invertierende Verstärkerschaltung

einzig durch den Slave angesteuert und sind demnach nicht mit dem Master verbunden. So steuert der Slave beispielsweise den adressierbaren LED-Streifen, für das visuelle Feedback an den Nutzer, über GPIO 23 an. Da der Funktionsgenerator, wie in FA-1 gefordert, über zwei Ausgangskanäle verfügen soll, muss für jeden Kanal ein dedizierter AD9833-DDS-IC verwendet werden. Die Spannungsversorgung der beiden ICs erfolgt über den 3,3 V Pin des Slaves. Für die Ansteuerung der beiden AD9833-Module werden eine Datenleitung (SDATA), die mit GPIO 19 verbunden ist, eine Taktleitung (SCLK), welche mit GPIO 21 verbunden ist, sowie zwei Leitungen für die Chipauswahl (FSYNC1 und FSYNC2), die mit GPIO 18 und GPIO 5 verbunden sind, benötigt. Da sich beide ICs die gleiche Daten- und Taktleitung teilen, wird über die FSYNC-Pins bestimmt, welches der beiden Module mit Daten beschrieben werden soll. Ist der Zustand von FSYNC1 Low, so ist das AD9833-Modul des ersten Kanals ausgewählt und kann mit Daten beschrieben werden. Ist der Zustand von FSYNC2 Low, so ist das AD9833-Modul des zweiten Kanals ausgewählt und kann mit Daten beschrieben werden. Zusätzlich zu den einzelnen FSYNC-Pins für jedes Modul gibt es einen gemeinsamen FSYNC_SYNC Anschluss, der mit GPIO 22 des Slaves verbunden ist und für die Synchronisation der beiden Module verwendet wird. Ohne eine Synchronisation über den SYNC-Pin würden die beiden Module zu unterschiedlichen Zeitpunkten initialisiert und freigegeben werden, wodurch die beiden Ausgangssignale zeitlich verschoben wären. Dies hätte einen negativen Einfluss auf die Phasenbeziehung der beiden Signale zueinander und würde dazu führen, dass die beiden Spannungen stets um eine gewisse Phase verschoben wären. [11, S. 3-4]

Die genaue Verschaltung der AD9833-Module sowie der einzelnen FSYNC-Pins ist in Abbildung 27 auf Seite 75 im Anhang zu sehen. Wie bereits erwähnt, werden beide ICs sowohl mit der Datenleitung (SDA) als auch mit der Taktleitung (SCLK) verbunden. Da beide AD9833-Module jeweils nur über einen FSYNC-Anschluss verfügen, müssen die Leitungen für FSYNC1, FSYNC2 und FSYNC_SYNC so miteinander verschaltet werden, dass der IC des ersten Kanals sowohl durch FSYNC1 als auch durch FSYNC_SYNC und der IC des zweiten Kanals sowohl durch FSYNC2 als auch durch FSYNC_SYNC ausgewählt werden kann. Zur Umsetzung dieses Verhaltens werden die Dioden D1 bis D4 sowie die beiden Widerstände R3 und R4 verwendet. Die Widerstände R3 und R4 ziehen die FSYNC-Pins der AD9833-Module standardmäßig auf High. Wird nun beispielsweise FSYNC1 oder FSYNC2 auf Low gesetzt, liegt am jeweiligen FSYNC-Pin eine Spannung von ca. 0,7 V an, was dem Low-Zustand entspricht. Die Dioden D2 und D3 verhindern dabei, dass die weiterhin auf High liegende FSYNC_SYNC-Leitung die Pins zurück auf High zieht. Analog dazu werden beide FSYNC-Pins auf Low gezogen, wenn FSYNC_SYNC auf Low gesetzt wird. Die Dioden D1 und D4 verhindern hierbei, dass die einzelnen FSYNC-Leitungen (FSYNC1 und FSYNC2) ihre jeweiligen FSYNC-Pins wieder auf High ziehen.

Wie in der Abbildung zu erkennen ist, werden zudem die zwei REF-Pins, an denen das Taktsignal des auf jedem Modul vorhandenen 25 MHz-Quarz abgegriffen werden kann, miteinander verbunden. Da jeder Quarz eine Genauigkeit von ca. 50 ppm besitzt, kann es unter der Verwendung eines Quarzes pro Modul zu einem sogenannten Phasendrift kommen.[27] Dieser tritt auf, da der AD9833 auf Grundlage des Taktsignals des Quarzes die Ausgangsfrequenz modelliert. Weichen diese Taktsignale voneinander ab, geben die Module am Ausgang Signale mit unterschiedlichen Frequenzen aus. Dies sorgt dafür, dass die Signale bei der Betrachtung auf einem Oszilloskop langsam auseinanderlaufen, wodurch keine feste Phasenverschiebung erreicht werden kann. Der

Effekt nimmt mit steigender Frequenz immer weiter zu. Dieser Effekt ist in Abbildung 30 und 31 auf den Seiten 78 und 79 im Anhang zu erkennen. So liegt die Phasenverschiebung bei einer Ausgangsfrequenz von 50 Hz nach 5 Minuten bei ca. 16° , obwohl die beiden ICs zum Zeitpunkt $t = 0$ s keine Phasenverschiebung aufgewiesen haben. Bei einer Ausgangsfrequenz von 500 Hz steigt die Phasenverschiebung nach 5 Minuten bereits auf ca. 155° an. Um dieses Problem zu beheben, wird auf einem der beiden ICs der Quarz entfernt und die beiden REF-Pins miteinander verbunden. Dadurch teilen sich beide Module nun das gleiche Taktsignal und können synchron betrieben werden. Laut Hersteller ist es dabei besonders wichtig, dass die Abstände des Taktgebers und der DDS-Chips nahezu identisch sind, da große Unterschiede in den Abständen erneut zu einer unbeabsichtigten Phasenverschiebung führen könnten. [11, S. 3-4] Dies ist in Abbildung 10 dargestellt.

Der Output der beiden AD9833-Module wird über zwei einzelne Leitungen an zwei MCP42010 Potentiometer weitergeleitet. Die Verschaltung dieser Potentiometer ist in Abbildung 28 auf Seite 76 im Anhang zu sehen. Beide ICs sind dabei erneut an die gleiche Datenleitung (SDATA) und Taktleitung (SCLK) angeschlossen. Bei diesen Leitungen handelt es sich jedoch nicht um die Leitungen, welche für die Ansteuerung der DDS-ICs verwendet werden. Diese Trennung wurde vorgenommen, da die gemeinsame Nutzung identischer Daten- und Taktleitungen durch die AD9833-Module und die MCP42010-Potentiometer zu gegenseitigen Störungen führt, wodurch keine zuverlässige und gezielte Steuerung der einzelnen Bauteile möglich ist. Aus diesem Grund werden für die digitalen Potentiometer eine eigene Datenleitung auf GPIO 26 sowie eine eigene Taktleitung auf GPIO 25 vorgesehen. Analog zu den FSYNC-Pins der DDS-ICs erfolgt die Auswahl der Potentiometer über die beiden Chip-Select-Leitungen CS1 und CS2. CS1 steuert dabei das Potentiometer des ersten Kanals, während CS2 das Potentiometer des zweiten Kanals ansteuert. Die Leitungen sind dabei mit Pin 4 (CS1) beziehungsweise Pin 0 (CS2) des Slaves verbunden. Die Widerstände R5 und R6 fungieren als Pull-up-Widerstände und stellen sicher, dass die Low-aktiven CS-Pins der Potentiometer standardmäßig auf einem High-Pegel liegen, solange CS1 beziehungsweise CS2 nicht aktiv auf Low gezogen werden. Laut Datenblatt müssen der Reset- sowie Shutdown-Pin (Pins 11 und 12 des ICs) bei Nichtverwendung auf High gezogen werden. Dazu werden beide Pins mit der Versorgungsspannung (3,3 V) verbunden. [29, S. 12] Um aus den Potentiometern einen Spannungsteiler zu realisieren, mit dem die Eingangsspannung des Operationsverstärkers gesteuert werden kann, werden die Ausgangssignale der AD9833-Module

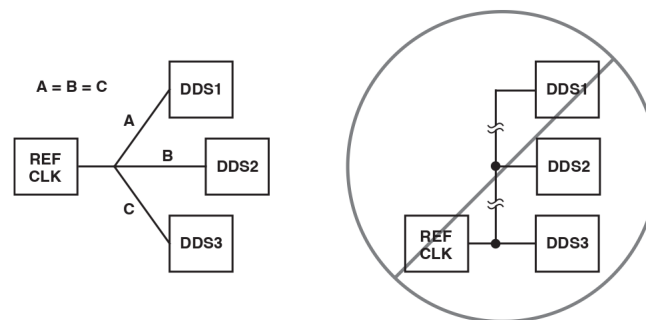


Abbildung 10: Referenztakt-Verteilung bei der Verwendung eines einzelnen Taktgebers für mehrere DDS-ICs [31, S. 1]

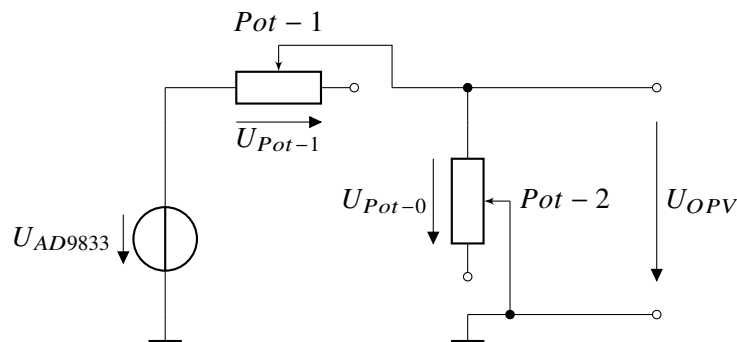


Abbildung 11: Vereinfachte Darstellung des Spannungsteilers mittels der digitalen Potentiometer MCP42010

jeweils mit dem B-Anschluss des zweiten Potentiometers (PB1) des ICs verbunden. Der Wiper des zweiten Potentiometers (PW1) wird anschließend mit dem B-Anschluss des ersten Potentiometers (PB0) verbunden, während der Wiper des ersten Potentiometers (PW0) mit GND verbunden wird. Der Eingang des Operationsverstärkers wird an der Verbindung zwischen PW1 und PB0 angeschlossen. Eine übersichtliche Darstellung des Spannungsteilers ist in Abbildung 11 zu sehen. Über das Verhältnis der Widerstände von Pot-0 und Pot-1 kann die Eingangsspannung des Operationsverstärkers eingestellt und somit die Amplitude des Ausgangssignals gesteuert werden. [25, S. 165]

Der Grund für die Nichtverwendung der digitalen Potentiometer des Typs DS1803-100 sind zwei konkrete bauteilspezifische Eigenschaften, welche bei der Entwicklung des Funktionsgenerators hervorgetreten sind. Da es sich bei digitalen Potentiometern nicht um klassische mechanische Bauteile handelt, besitzen sie im Gegensatz zu diesen eine bestimmte Grenzfrequenz. Digitale Potentiometer bestehen im Wesentlichen aus Widerständen und Schaltern. Der Widerstand entlang des Signalpfades bildet zusammen mit den parasitären Kapazitäten der Schalter, den Pin- und Platinenkapazitäten einen RC-Tiefpassfilter. Dieser Tiefpassfilter bestimmt die maximale Wechselstromfrequenz, die durch das Potentiometer geleitet werden kann, bevor das Ausgangssignal um mehr als 3 dB gedämpft wird. Dabei besitzen Potentiometer mit geringem Gesamtwiderstand eine hohe und Potentiometer mit einem großen Gesamtwiderstand eine geringe Grenzfrequenz. Für die Potentiometer des Typs DS1803-100 mit einem Gesamtwiderstand von ca. 100 k Ω liegt die Grenzfrequenz bei ca. 70 kHz. [32, S. 4] Dies würde bedeuten, dass der Spannungsteiler ab dieser Frequenz die Ausgangsspannung des OPVs nicht korrekt steuern kann. Zudem fiel bei Tests auf, dass die DS1803-100-Potentiometer bei höheren Frequenzen für nicht unerhebliche Verzerrungen des Ausgangssignals verantwortlich sind. Dies ist in Abbildung 32 auf Seite 80 im Anhang zu erkennen. Da die MCP42010 Potentiometer einen Gesamtwiderstand von ca. 10 k Ω besitzen, liegt die Grenzfrequenz laut Datenblatt bei ca. 1 MHz. Zudem tritt unter Verwendung dieser Potentiometer bei hohen Ausgangsfrequenzen keine nennenswerte Verzerrung des Spannungssignals auf.

Da das AD9833-Modul nur positive Spannungswerte ausgeben kann, muss das Signal vor der Verstärkerschaltung um den Nullpunkt angesteuert werden. Hierzu werden, wie in Abbildung 26 auf Seite 74 im Anhang zu sehen ist, zwei 100 nF Keramikkondensatoren C1 und C2 verwendet. Diese Koppelkondensatoren entfernen das Gleichtaktsignal und sorgen für ein nullsymmetrisches

Spannungssignal am Eingang des Operationsverstärkers. Die Widerstände R1 und R2 dienen als DC-Bias-Widerstände und stellen nach den AC-Koppelkondensatoren C1 und C2 einen definierten Gleichspannungspfad nach Masse bereit. Ohne diese Widerstände könnte kein stabiler Gleichspannungs-Arbeitspunkt an den Eingängen des Operationsverstärkers sichergestellt werden, was zu einem undefinierten Arbeitspunkt und einer Sättigung des Ausgangs führen könnte.

Die nullsymmetrischen Spannungssignale werden an die nicht invertierenden Kontakte des in Abbildung 29 auf Seite 77 im Anhang dargestellten Operationsverstärkers weitergeleitet. Damit ein in NFA-5 geforderter Spannungsbereich abgedeckt werden kann, wurde entschieden, das Spannungssignal auf einen Bereich von ± 10 V zu verstärken. Um diesen Spannungsbereich aussteuern zu können, wird der OPV mittels einer symmetrischen Versorgungsspannung von ± 15 V, die über Kontakte an den Laborarbeitsplätzen des Messtechniklabors abgegriffen wird, betrieben. Da die Kondensatoren C1 und C2 zusammen mit den Widerständen R1 und R2 einen RC-Hochpass bilden, muss der Verstärkungsfaktor V der Verstärkerschaltung in der Lage sein, das bei niedrigen Frequenzen gedämpfte Ausgangssignal des Spannungsteilers zu verstärken. Bei einer Frequenz von 1 Hz beträgt das Ausgangssignal im Sinusmodus ca. 287 mV (Top). Der OPV benötigt also einen Verstärkungsfaktor von ca. 34,84, um das Spannungssignal auf ± 10 V zu verstärken. Damit ein gewisser Sicherheitsabstand entsteht, wird durch die Dimensionierung der Widerstände R7 bis R10 ein Verstärkungsfaktor von ca. 37,36 festgelegt.

$$V = 1 + \frac{12 \text{ k}\Omega}{330 \text{ }\Omega} = 37,36 \quad (11)$$

Anhand der Slew Rate und der Bandbreite des OPVs lässt sich die maximal erreichbare Ausgangsfrequenz unter diesem Verstärkungsfaktor berechnen. Für die Sinusausgabe berechnet sich diese wie folgt: [33]

$$f_{\max} = \frac{\text{Slew Rate}}{2 \cdot \pi \cdot \text{Ausgangsspannung}} = \frac{27 \frac{\text{V}}{\mu\text{s}} \cdot 10^6}{2 \cdot \pi \cdot 10 \text{ V}} = 430 \text{ kHz} \quad (12)$$

Da bei einem Verstärkungsfaktor von 37,36 und einer Frequenz von 430 kHz eine Bandbreite von gerade einmal 16 MHz benötigt wird, ergibt sich die Grenzfrequenz einzig durch die Slew Rate des OPVs. Bei der Verstärkung des Rechtecksignals muss sich die Spannung am Ausgang des OPVs (V_a) bei einer Verstärkung auf ± 10 V um das Zweifache von V_a ändern. Dies wird nur mit der maximalen Anstiegsrate (Slew Rate) erreicht. Damit das Signal noch als rechteckig erkannt werden kann, sollte die Anstiegszeit kleiner als eine halbe Periodenzeit sein. Unter dem Grenzfall der halben Periodenzeit gilt: [33]

$$t_{\text{rise}} = \frac{2 \cdot V_a}{\text{Slew Rate}} \Rightarrow t_{\text{rise}} = \frac{T}{2} = \frac{1}{2 \cdot f} \quad (13)$$

Somit ergibt sich eine Grenzfrequenz $f_{\max}$ von:

$$f_{\max} = \frac{\text{Slew Rate}}{4 \cdot V_a} = \frac{27 \frac{\text{V}}{\mu\text{s}} \cdot 10^6}{4 \cdot 10 \text{ V}} = 675 \text{ kHz} \quad (14)$$

Für die eingestellte Verstärkung von 37,36 ergibt sich eine Grenzfrequenz von:

$$f_{max} = \frac{GBW}{V} = \frac{40 \text{ MHz}}{37,36} = 1,07 \text{ MHz} \quad (15)$$

Da ein Rechtecksignal jedoch nicht nur aus einer Grundschiwingung sondern aus einer Überlagerung von vielen Oberschwingungen besteht, müssen diese Schwingungen ebenfalls berücksichtigt werden. Ab der dritten Oberschwingung kann das Signal als rechteckförmig identifiziert werden, wodurch diese Oberschwingung ebenfalls durch den OPV verstärkt werden muss. [34, S. 128-130] Dadurch ergibt sich in Abhängigkeit der Bandbreite eine neue Grenzfrequenz von:

$$f_{max} = \frac{1,07 \text{ MHz}}{3} \approx 357 \text{ kHz} \quad (16)$$

Somit sollte der OPV in der Lage sein, bis zu einer Frequenz von 357 kHz ein erkennbares Rechtecksignal zu verstärken.

Nach dem OPV werden die Signale durch die 100 nF Keramikkondensatoren C3 und C4 (Abbildung 26 auf Seite 74 im Anhang) erneut um den Nullpunkt ausgesteuert. Hierbei werden keine zusätzlichen Bias-Widerstände benötigt, da im Inneren des Oszilloskops ein hochohmiger Widerstand mit Masse verbunden ist.

6.3. Ansteuerungslogik

Die Ansteuerung der einzelnen Komponenten und die damit verbundene Spannungsausgabe wurden, wie bei der in Kapitel 5 beschriebenen Variante, mittels der Programmiersprache C++ in der Arduino IDE (Version 2.3.X) umgesetzt. Für die Umsetzung wurden sieben dedizierte C++-Dateien, eine zentrale Ino-Datei und acht Header-Dateien verwendet. Die Auflistung der einzelnen Dateien sowie ihre Funktionen sind in Tabelle 11 zu finden.

Da die vollständige Beschreibung aller Funktionen, die im Programmcode implementiert wurden, den Rahmen dieser Ausarbeitung sprengen würde, wird sich im Folgenden auf die wichtigsten Funktionen zur Ansteuerung der AD9833-DDS-ICs, zur Ansteuerung der MCP42010-Potentiometer und die Funktionen zur Berechnung des Widerstandsverhältnisses beschränkt. Sobald der Funktionsgenerator mit Spannung versorgt wird, aktiviert sich das ESP32 DevKit und führt den Programmcode innerhalb der Funktion `setup()` aus. Dazu gehören die Initialisierung der seriellen Schnittstelle für die Ausgabe von Debug-Nachrichten, die Initialisierung des LED-Streifens, des NVS-Speichers, der Uart-Kommunikation, der AD9833-ICs und der MCP42010-ICs. Zudem wird die Anzahl aller im NVS gespeicherten Gruppen über den seriellen Monitor ausgegeben. Nachdem der Mikrocontroller diese Funktion vollständig durchlaufen hat, wird über die Funktion `loop()` dauerhaft überprüft, ob das HMI neue Daten über die UART-Schnittstelle übersendet. Der Inhalt beider Funktionen ist in Listing 4 auf Seite 81 im Anhang zu finden.

Für die Ansteuerung der AD9833-ICs sind im Wesentlichen fünf Funktionen verantwortlich: `UpdateRegister()`, `UpdateFreq()`, `UpdatePhase()`, `UpdateFreqWord()` und `SyncChips()`. Jeder IC besitzt zudem mehrere Variablen zum Abspeichern der aktuellen Frequenz, der aktuel-

len Phase und des aktuellen Ausgabemodus (Sinus oder Rechteck). Außerdem besitzt jeder IC eine Variable zum Speichern des aktiven Kontrollworts. Dieses Kontrollwort wird jeweils mit den gespeicherten Kommandos für das Aktivieren des Sleep-Modus (SLEEP), das Zurücksetzen des ICs (RESET) oder das Aktivieren der Sinus- oder Rechteckausgabe (SIN und SQU) logisch kombiniert, um die ICs in den gewünschten Betriebsmodus zu versetzen. Über die Funktion `UpdateRegister()` kann ein vollständiges 16-Bit-Datenwort, welches an die Funktion übergeben wird, an einen der beiden ICs gesendet werden. Welcher der beiden ICs angesprochen werden soll, wird dabei durch die Vorgabe des Chips beim Aufrufen der Funktion bestimmt. Je nach übergebenem Chip wird der entsprechende FSYNC-Pin für die Übertragung verwendet. Die Datenübertragung wird dabei durch einen High-Zustand des Taktsignals bei gleichzeitigem Low-Zustand von FSYNC gestartet und durch einen High-Zustand von FSYNC beendet. Die vollständige Funktion ist in Listing 5 auf Seite 82 im Anhang dargestellt.

Mittels der Funktionen `UpdateFreq()` und der Funktion `UpdatePhase()` (Listing 6 auf Seite 83 im Anhang) können die Frequenzen und die Phasenverschiebungen beider ICs gesteuert werden. Dazu werden die Phase und Frequenz sowie der zu steuernde Chip an die Funktion übergeben. Im Anschluss prüfen beide Funktionen, ob die übergebenen Werte korrekt bzw. einstellbar sind und berechnen im Anschluss das 28-bit-Frequenzwort bzw. das 12-bit-Phasenwort. Diese werden

Tabelle 11: Übersicht der Dateien für die Ansteuerung der Komponenten und Ausgabe der Spannungen des Slave-ESP32

Dateiname	Funktion
ESP32DevMod.ino	Hauptdatei. Verantwortlich für die Verknüpfung der einzelnen Unterdateien und das Ausführen der Funktionen.
AD9833.cpp und AD9833.h	Dateien für die Ansteuerung der AD9833-DDS-ICs.
MCP42010.cpp und MCP42010.h	Dateien für die Ansteuerung der MCP42010-Potentiometer.
ws2812b.cpp und ws2812b.h	Dateien für die Ansteuerung des adressierbaren LED-Streifens.
resistance.cpp und resistance.h	Dateien für die Berechnung der Widerstandsverhältnisse des Spannungsteilers.
dds_output.cpp und dds_output.h	Datei für die Steuerung der Spannungsausgabe, falls diese über das HMI gestartet oder gestoppt wird.
save_data.cpp und save_data.h	Dateien für das Speichern der Gruppenparameter im NVS.
signal_config.h	Datei für die Definition der Signalkonfiguration der UART-Übertragung.
uart.cpp und uart.h	Dateien für die Umsetzung der UART-Kommunikation zwischen dem HMI (Master) und dem ESP32 DevKit (Slave).

dann mittels `UpdateRegister()` an die ICs gesendet. Bei Testversuchen ist dabei aufgefallen, dass sowohl die Frequenz als auch die Phase unter Verwendung der Rechteckausgabe verdoppelt werden müssen. Am Ende der Funktion `UpdateFreq()` (Zeile 101 bis 108) wird zudem der Ausgabemodus (Sinus- oder Rechteckausgabe) festgelegt. Für die Synchronisation beider ICs durch den `FSYNC_SYNC`-Pin wird die Funktion `SyncChips()` in Kombination mit der Funktion `WriteFreqWord()` verwendet (Listing 7 auf Seite 85 im Anhang). Diese Funktionen erlauben es, im Gegensatz zur Funktion `UpdateFreq()`, beide ICs gleichzeitig mit dem identischen Frequenzwort zu beschreiben. Durch `SyncChips()` wird zudem garantiert, dass beide ICs, sollten sie sich im gleichen Ausgabemodus befinden, über den gemeinsamen `FSYNC`-Pin freigegeben werden und somit synchron laufen. Sollten sich die Modi unterscheiden, werden die ICs sequenziell so schnell wie möglich freigegeben, um die Phasenverschiebung möglichst gering zu halten.

Die Steuerung der Potentiometer des Typs MCP42010 wird weitestgehend mittels fünf Funktionen umgesetzt: `spi_write_byte()`, `mcp_send()`, `setPot0()`, `setPot1()` und `setBoth()`. Zudem werden die Positionen der Potentiometer beider ICs für Berechnungen in zwei Listen `positionPot0[]` und `positionPot1[]` gespeichert. Die Widerstände der Potentiometer im unteren und oberen Anschlag sind in den Listen `P0_255[]` und `P0_0[]` sowie `P1_255[]` und `P1_0[]` gespeichert. Über die Funktion `spi_write_byte()` kann ein vollständiges Byte, welches an die Funktion übergeben wird, über die SPI-Datenleitung übertragen werden. `mcp_send()` verwendet diese Funktion, um ein bestimmtes Kommando gefolgt von einem Wert an einen der beiden ICs, welcher beim Aufrufen der Funktion festgelegt wird, zu übertragen. Abhängig vom ausgewählten Chip wird dabei der entsprechende CS-Pin auf High oder Low gesetzt. Die Funktionen sind in Listing 8 auf Seite 87 im Anhang zu sehen. Mittels der Funktionen `setPot0()`, `setPot1()` und `setBoth()` können die Potentiometer der beiden ICs eingestellt werden. Dabei wird zuerst mittels `constrain` überprüft, ob der an die Funktion übergebene und einzustellende Wert zwischen 0 und 255 (8-bit) liegt. Anschließend wird mittels `mcp_send()` der neue Wert an den ausgewählten IC gesendet. Ist die Übertragung erfolgreich, wird der neue Wert auf dem seriellen Monitor ausgegeben. Falls nicht, wird eine Fehlermeldung ausgegeben. Diese Funktionen sind in Listing 9 auf Seite 88 im Anhang zu sehen.

Für die Berechnung der Widerstandsverhältnisse werden insgesamt drei Funktionen verwendet: `getVoltAmplifikation()`, `getInputVoltage()` und `calculateResistance()`. Da die verwendeten Widerstände in der Verstärkerschaltung produktionsbedingt leichte Abweichungen aufweisen, beträgt der Verstärkungsfaktor jedes Kanals nicht exakt 37,36. Über die Funktion `getVoltAmplifikation()` wird der gemessene Verstärkungsfaktor für den beim Aufruf der Funktion ausgewählten Kanal zurückgegeben. Wie bereits erwähnt, ist die Eingangsspannung des OPVs sowohl durch den RC-Hochpass als auch durch die Grenzfrequenz der Potentiometer und die Frequenzabhängigkeit der Ausgangsspannung des AD9833-Moduls nicht konstant. Daher wird über die Funktion `getInputVoltage()` die Eingangsspannung am OPV berechnet und zurückgegeben. Hierzu wird eine Lookup-Tabelle verwendet, in der die Ausgangsspannung des AD9833-Moduls mit einem 100 nF Kondensator bei unterschiedlichen Frequenzen gespeichert ist. Der Einfluss der Grenzfrequenz wird aufgrund der Komplexität ihrer Berechnung vernachlässigt. Anhand der eingestellten Frequenz wird dann die tatsächliche Eingangsspannung am OPV mittels der Lookup-Tabelle interpoliert. Beide Funktionen sind in Listing 10 auf Seite 90 im Anhang

zu finden. Mittels der Funktion `calculateResistance()` (Listing 11 auf Seite 92 im Anhang) werden die neuen Widerstandsverhältnisse des Spannungsteilers berechnet, um die vorgegebene Ausgangsspannung zu erreichen. Zu Beginn der Funktion werden die Eingangsspannung sowie der Verstärkungsfaktor des aktuellen Kanals bestimmt. Zudem wird die Realisierbarkeit der Zielspannung unter diesen Parametern geprüft. Kann die Zielspannung nicht erreicht werden, wird die Funktion ohne eine Veränderung der Widerstände verlassen. Befindet sich die Zielspannung im zulässigen Bereich, ermittelt die Funktion das erforderliche Widerstandsverhältnis zwischen den beiden Stellgliedern. Um eine maximale Auflösung zu gewährleisten, wird das erste digitale Potentiometer (Pot0) standardmäßig auf seinen Maximalwert fixiert, während die exakte Position für das zweite digitale Potentiometer (Pot1) berechnet wird. Sollte die neue Position von Pot1 den Grenzwert 255 erreichen, wird zudem die Position von Pot0 neu berechnet. Abschließend werden die neuen Widerstandswerte über den seriellen Monitor ausgegeben und die Potentiometer gesetzt. In der Datei `dds_output.cpp` werden die einzelnen Funktionen entsprechend der vorgegebenen Parameter der einzelnen Gruppen aufgerufen, um die geforderten Spannungssignale an den Ausgängen des Funktionsgenerators bereitzustellen.

6.4. Messaufbau, Testversuche und Genauigkeitsbewertung

Für die Überprüfung der nicht-funktionalen Anforderungen wurden, identisch zu der in Kapitel 5 beschriebenen Messreihe, verschiedene Messungen hinsichtlich der Frequenz-, Amplituden- und Phasengenauigkeit durchgeführt. Analog zur ersten Messreihe wurde für die Aufnahme und Bestimmung der Messwerte ein Keysight InfiniiVision 2000 X-Series Oszilloskop verwendet. Für die Überprüfung der nicht-funktionalen Anforderungen wurden die getesteten Signalformen (Sinus und Rechteck) in mehreren Frequenzbereichen unter verschiedenen Vorgaben zur Ausgangsspannung und Phasenverschiebung aufgezeichnet. Die Messung der Ausgangsspannung, der Frequenz und der Phasenverschiebung wurde durch die automatische Messfunktion des Oszilloskops unter Verwendung der DC-Kopplung durchgeführt.

Amplitudengenauigkeit (NFA-1):

Für die Bestimmung der Amplitudengenauigkeiten wurde die Ausgangsspannung der beiden Ausgangskanäle (Kanal 1 und Kanal 2) bei verschiedenen Frequenzen für die Vorgaben 10 V, 5 V, 2 V, 1 V und 0,5 V aufgezeichnet. Die Messung der Ausgangsspannung erfolgte dabei durch die automatische Spannungsmessung des Oszilloskops unter Verwendung der Top-Messung, welche den oberen Spannungswert ohne die Berücksichtigung von Spannungsspitzen misst. Als Frequenzbereich wurde sowohl für die Sinussignale als auch die Rechtecksignale der Bereich zwischen 10 Hz und 100 kHz ausgewählt. Die Ergebnisse der Messungen sind in Abbildung 33 bis 34 auf den Seiten 94 bis 95 im Anhang zu finden. Beim Betrachten der Abbildungen fällt auf, dass sowohl bei der Ausgabe von Sinus- als auch bei der Ausgabe von Rechtecksignalen bei einer Frequenz von 10 Hz ein Amplitudenfehler erreicht wird, welcher weit außerhalb des maximalen Grenzbereichs von 5 % liegt. Für die Sinusausgabe tauchen die beiden Kanäle erst bei einer Ausgangsfrequenz von 100 Hz in das Toleranzband ein. Beim zweiten Kanal ist bei dieser Frequenz besonders der Messpunkt bei 0,5 V auffällig, welcher mit einem Amplitudenfehler von 8,93 % erneut weit außerhalb des

Toleranzbands liegt. Hierbei könnte es sich jedoch um einen einfachen Messfehler handeln, da alle vorherigen Messpunkte der Frequenz innerhalb des Toleranzbands lagen.

Allgemein ist zu erkennen, dass die beiden Kanäle in beiden Ausgabemodi zueinander leichte Abweichungen aufweisen. So liegt der Messpunkt 10 V bei einer Frequenz von 1 kHz in der Rechteckausgabe bei Kanal 2 mit einem Wert von -3,73 % beispielsweise innerhalb und bei Kanal 1 mit einem Wert von -6,38 % außerhalb des Grenzbereichs. Bei der Rechteckausgabe fällt zudem auf, dass der relative Amplitudenfehler bei einer Vorgabe unterhalb von 2 V konstant außerhalb des Grenzbereichs liegt. Obwohl die ausgewählten Potentiometer eine angegebene Grenzfrequenz von 1 MHz besitzen und die berechnete Verstärkung für die Sinusausgabe bei 430 kHz und für die Rechteckausgabe bei 357 kHz liegt, fällt auf, dass das Sinussignal bei einer Frequenz von 100 kHz bereits gedämpft wird. Dies ist besonders bei der Spannungsvorgabe von 5 V zu erkennen. Bei der Rechteckausgabe ist bei einer Frequenz von 100 kHz und einer Spannungsvorgabe von 10 V zu erkennen, dass die Signale der beiden Kanäle leicht verzerrt sind und unterschiedlich schnelle Anstiegszeiten aufweisen (Abbildung 35 auf Seite 96 im Anhang). Somit unterscheiden sich die Pulsbreiten der Signale, was zu einer ungewollten Phasenverschiebung führt. Mit zunehmender Frequenz nimmt dieser Effekt immer weiter zu.

Insgesamt liegen von den 80 durchgeführten Messungen für die Sinusausgabe 69 innerhalb des Toleranzbereichs. Für die Rechteckausgabe liegen hingegen nur 43 von 80 Messungen innerhalb des 5 % Bereichs. Würde Man die Sinusausgabe auf eine Frequenz von ca. 1 kHz bis ca. 80 kHz begrenzen, lägen 50 von 50 Messungen innerhalb des Toleranzbereichs. Würde man die Rechteckausgabe auf eine Frequenz von ca. 10 kHz bis ca. 80 kHz beschränken, sowie alle Spannungsbereiche unterhalb von 2 V vermeiden, lägen 24 von 24 Messpunkten innerhalb des Toleranzbereichs. Möglicherweise könnten die genannten Frequenzbereiche etwas erweitert werden, jedoch erfüllen nur diese Frequenzbereiche unter Betrachtung der erhobenen Messwerte die Anforderung NFA-1.

Phasengenauigkeit (NFA-2):

Abbildung 36 auf Seite 97 im Anhang zeigt die Ergebnisse der Messung zur Phasengenauigkeit des DDS-Funktionsgenerators. Hierbei wurde die Phasenverschiebung zwischen Kanal 2 und Kanal 1 bei den Vorgaben 0°, 90° und 120° sowie einer Ausgangsspannung von 10 V bei unterschiedlichen Ausgangsfrequenzen für die Sinus- und Rechteckausgabe gemessen. Der Frequenzbereich wurde dabei auf 100 Hz bis 100 kHz festgelegt, da, wie aus den Messungen für die Amplitudengenauigkeit hervorgeht, dieser Bereich für eine mögliche Verwendung in Frage kommt. Die Phasenverschiebung wurde, wie in der Aufgabenstellung der Laborprüfung beschrieben, von Kanal 2 in Bezug auf Kanal 1 gemessen. In der Abbildung ist zu erkennen, dass alle aufgenommenen Messwerte sowohl für die Sinus- als auch die Rechteckausgabe innerhalb des Toleranzbereichs von $\pm 2^\circ$ liegen. Bei der Sinusausgabe wird ein maximaler Phasenfehler von $-1,91^\circ$ und ein minimaler Phasenfehler von $-0,05^\circ$ erreicht. Der mediane Phasenfehler für die Sinusausgabe liegt bei $-0,40^\circ$ und der durchschnittliche Phasenfehler bei rund $-0,43^\circ$. Für die Rechteckausgabe liegt der maximale Phasenfehler bei $0,80^\circ$ und der minimale Phasenfehler bei $0,00^\circ$. Insgesamt wird ein medianer Phasenfehler von $0,00^\circ$ und ein durchschnittlicher Phasenfehler von $0,03^\circ$ erreicht. Durch die bereits angesprochene Veränderung der Pulsbreite der beiden Kanäle nimmt der Phasenfehler bei der Rechteckausgabe mit steigender Frequenz immer weiter zu. So liegt der Phasenfehler bei einer Frequenz von 150 kHz

und einer Vorgabe von 120° beispielsweise bereits bei 45° . Bleibt die Frequenz jedoch auf den getesteten Bereich beschränkt, gilt die Anforderung NFA-2 als vollumfänglich erfüllt.

Frequenzgenauigkeit (NFA-3):

Für die Ermittlung der Frequenzgenauigkeit wurde der vollständige Frequenzbereich des AD9833-Moduls, bevor eine Verzerrung des Ausgangssignals erfolgt, betrachtet. Hierbei wurden logarithmisch Messwerte zwischen den Frequenzen 1 Hz und 1 MHz erhoben. Zusätzlich wurden Messungen bei den Frequenzen 500 kHz, 2 MHz, 3 MHz, 4 MHz und 5 MHz durchgeführt. Die Ergebnisse der Messungen sind in Abbildung 37 auf Seite 98 im Anhang zu sehen. Für die Sinusausgabe liegt der maximale Fehler bei 2,44 % und der minimale Fehler bei 0,00 %. Als durchschnittliche Abweichung ergibt sich ein Wert von 0,49 % und eine mediane Abweichung von 0,02 %. Für die Rechteckausgabe liegt der maximale Fehler bei 1,80 % und der minimale Fehler ebenfalls bei 0,00 %. Als durchschnittliche Abweichung ergibt sich ein Wert von 0,33 % und eine mediane Abweichung von 0,03 %. Da der durch die vorhergehenden Überprüfungen festgestellte maximale Frequenzbereich deutlich kleiner ist (maximal 100 Hz bis 100 kHz), sinkt der durchschnittliche sowie mediane Fehler für die Sinusausgabe auf 0,06 % bzw. 0,05 % und für die Rechteckausgabe auf 0,05 % bzw. 0,03 %. Somit gilt NFA-3 ebenfalls als vollumfänglich erfüllt.

Reaktionstest (NFA-6):

Zur Validierung der Anforderung NFA-6 wurde analog zu Kapitel 5 die interne Reaktionszeit des Funktionsgenerators mittels eines Logic Analyzers erfasst. Die Eingänge des Logic Analyzers wurden dabei mit dem Trigger-Pin des Masters (GPIO-15) und dem Ausgang des Operationsverstärkers (Kanal 2) verbunden (Abbildung 38 auf Seite 99 im Anhang). Da Kanal 2 durch den Programmcode nach Kanal 1 aktiviert wird, ergibt sich durch diesen Messaufbau die größtmögliche Zeitverzögerung. Die interne Reaktionszeit des Systems ergibt sich aus der Zeitdifferenz zwischen der steigenden Flanke des Trigger-Pins und der ersten positiven Flanke des Ausgangssignals. Zusätzlich zur internen Reaktionszeit muss erneut die Touch-Polling-Rate des HMIs berücksichtigt werden. Tabelle 12 zeigt die Ergebnisse von fünf Einzelmessungen der Reaktionszeit des DDS-Funktionsgenerators. Die am Logic Analyzer gemessene Verarbeitungs- und Ausgabezeit (t_{Ausgang}) liegt in allen Fällen über einer Zeit von 70 ms. Zusammen mit der Worst-Case-Polling-Latenz von 30 ms (t_{Touch}) ergibt sich eine Reaktionszeit von durchschnittlich 119,38 ms. Die geforderte Obergrenze von 100 ms wird damit deutlich überschritten, sodass die Anforderung NFA-6 nicht erfüllt ist.

Tabelle 12: Reaktionszeitmessung für das Einschalten der Ausgabe zur Überprüfung der Anforderung NFA-6 (DDS)

Messung	t_{Touch} [ms]	t_{Ausgang} [ms]	Reaktionszeit [ms]	NFA-6 erfüllt? ($\Delta t < 100$ ms)
1	30	90,6	120,6	Nein
2	30	89,5	119,5	Nein
3	30	90,5	120,5	Nein
4	30	86,6	116,8	Nein
5	30	89,5	119,5	Nein

7. Entwicklung der Benutzeroberfläche

7.1. Gestaltungsprinzipien

Die Bedienung des Funktionsgenerators erfolgt in beiden Implementierungsmethoden ausschließlich über die Benutzeroberfläche des HMIs. Da die einzelnen Arbeitsgruppen das Gerät nur kurz und mit unterschiedlicher Vorerfahrung bedienen, stehen eine geringe Einarbeitungszeit und eine möglichst selbsterklärende Bedienung im Vordergrund. Die grundlegenden Anforderungen an die Oberfläche wurden bereits in Abschnitt 3 in Form der funktionalen und nicht-funktionalen Anforderungen festgelegt. Aus ihnen leiten sich die folgenden Gestaltungsprinzipien ab.

Im Mittelpunkt steht die Reduktion auf die für den Versuch notwendigen Bedienschritte: das Auswählen der eigenen Arbeitsgruppe, das Wechseln zwischen den Gruppen (FA-8) sowie das Starten und Stoppen der Signalausgabe (FA-10). Damit diese Funktionen ohne Erklärung auffindbar sind, erfolgt die Navigation über dedizierte, eindeutig beschriftete Bedienelemente (FA-13). Eingaben sollen gemäß NFA-6 in unter 100 ms sichtbar quittiert werden, sodass die Bedienung als unmittelbar reaktionsschnell wahrgenommen wird. Für den Fall, dass Studierende Schwierigkeiten bei der Bedienung der Laborgeräte haben, stellt das HMI darüber hinaus verständlich aufbereitete Hilfestellungen bereit (FA-12, NFA-12), etwa zum Messen der Phasenverschiebung, der Amplitude oder der Frequenz mit dem Oszilloskop.

Eine zentrale Gestaltungsentscheidung besteht zudem darin, den Studierenden zu keinem Zeitpunkt die hinterlegten Sollwerte der Signalparameter anzuzeigen. Die Oberfläche beschränkt sich auf die Gruppenauswahl und die Ausgabesteuerung, wodurch der angestrebte Lerneffekt beim eigenständigen Messen der Signale erhalten bleibt und eine Manipulation der Messparameter ausgeschlossen wird.

7.2. Umsetzung mittels EEZ Studio und der LVGL-Library

Die softwaretechnische Entwicklung der Benutzeroberfläche erfolgte mit der integrierten Entwicklungsumgebung EEZ Studio (Version 0.26.2) in Verbindung mit der Grafikbibliothek LVGL (Version 8.4.0). Bei EEZ Studio handelt es sich um eine Open-Source-Entwicklungsumgebung, die ursprünglich für den Aufbau, die Steuerung und die Verwaltung modularer Mess- und Testaufbauten konzipiert wurde. Die Oberflächengestaltung erfolgt dabei überwiegend visuell. Bedienelemente werden per Drag-and-Drop platziert und über Eigenschaftsdialoge konfiguriert, wobei eine Echtzeit-Vorschau die Gestaltung unterstützt. Dieser Low-Code-Ansatz reduziert den Programieraufwand und beschleunigt den Arbeitsprozess. [35]

Für die Darstellung auf dem eingebetteten System greift EEZ Studio auf die Grafikbibliothek LVGL zurück, die gezielt für ressourcenbeschränkte Mikrocontroller ausgelegt ist und flüssige, grafisch ansprechende Oberflächen ermöglicht. Über die mitgelieferte Komponentenbibliothek stehen die für die Anwendung benötigten Bausteine, wie etwa Schaltflächen, Eingabefelder, Beschriftungen und Bildelemente, zur Verfügung. Aus diesen Elementen ließ sich eine an die in Abschnitt 3 definierten Anforderungen angepasste Bedienoberfläche für das HMI entwickeln. Die LVGL ver-

wendet für die Darstellung die Programmiersprache C. Diese Dateien können nahtlos in den für die Programmierung verwendeten C++-Code eingebunden werden. [35] [36]

Die umgesetzte grafische Benutzeroberfläche besteht aus neun einzelnen Seiten, zwischen denen über Schaltflächen auf den jeweiligen Seiten navigiert werden kann. Die einzelnen Seiten sind in den Abbildungen 39 bis 43 auf den Seiten 100 bis 103 im Anhang dargestellt.

Beim Starten des HMIs wird zuerst der Ladebildschirm angezeigt. Nach 500 ms wird automatisch auf den Hauptbildschirm gewechselt. Dieser besitzt eine Scroll-Liste für die Auswahl der Gruppen, zwei Buttons zum Starten und Stoppen der Spannungsausgabe, eine LED zur Visualisierung des Ausgabestatus sowie einen Button zum Laden der Gruppenparameter von der microSD-Karte. Darüber hinaus besitzt die Hauptseite in der oberen rechten Ecke eine Statusanzeige, die anzeigt, ob eine microSD-Karte erkannt wurde und welche Speichergröße die erkannte microSD-Karte aufweist. Die Veränderung der Anzeige wird dabei über die zwei Variablen `SD_Card_Inserted` und `SD_Card_Size` gesteuert. Die Zahl der über die Scroll-Liste ausgewählten und somit aktiven Gruppe kann über die Variable `Active_Group` abgefragt werden. Über den Button in der oberen linken Ecke gelangt man in die Navigationsansicht. Hier kann zwischen den einzelnen Hilfestellungen für das Messen der Amplitude, Frequenz und Phasenverschiebung sowie einer Übersicht der Grenzwerte, Informationen zur Formatierung der microSD-Karte und einer allgemeinen Beschreibung des Funktionsgenerators navigiert werden. Über den Zurück-Button in der oberen rechten Ecke gelangt man von jeder Unterseite zurück in die Navigationsansicht und von der Navigationsansicht zurück auf den Hauptbildschirm. Zusätzlich können auf dem Hauptbildschirm Fehlermeldungen in Form von Pop-ups angezeigt werden (Abbildung 44 auf Seite 104 im Anhang). Hierbei wird zwischen einer Fehlermeldung bei fehlender oder fehlerhafter CSV-Datei und einer Fehlermeldung bei einem Problem in der UART-Kommunikation mit dem Slave unterschieden.

Jeder Button ist mit einem eigenen Event-Handler ausgestattet, welcher beim Loslassen des Buttons einen bestimmten Programmcode, der manuell in eine C- oder C++-Datei mit dem Namen `actions.c` bzw. `actions.cpp` hinzugefügt werden muss, ausführt. Um die Einbindung des Programmcodes zu vereinfachen und zu vereinheitlichen, wurden die Event-Handler für die Funktionsgeneratoren in einer `actions.cpp` Datei umgesetzt. Exemplarische Beispiele für die verwendeten Event-Handler für die Buttons Start, Stop, Load Data und Hilfe sind in den Listings 12 bis 15 auf den Seiten 105 bis 108 im Anhang zu finden.

Da sich die beiden Implementierungsvarianten ausschließlich in der Ansteuerung der Komponenten durch den Slave unterscheiden, können die Benutzeroberfläche sowie deren Einbindung in den Programmcode unverändert für beide Varianten genutzt werden. Es ist lediglich erforderlich, die Grenzwerte in der entsprechenden Übersicht an die jeweilige Variante anzupassen.

8. Mechanisches Design

8.1. Konstruktion

Für die Umsetzung der mechanischen Konstruktion wurde das CAD-Programm Autodesk Fusion der Firma Autodesk in der Version 2702.1.X verwendet. Autodesk Fusion ist eine cloudbasierte Produktentwicklungsplattform, die Funktionen für Konstruktion, Fertigung, Simulation, Elektronikentwicklung und Datenmanagement in einer einzigen Umgebung vereint. Dadurch können Produktentwicklungsprozesse von der Konzeption bis zur Fertigung effizient und ohne den Einsatz zusätzlicher Softwarelösungen durchgeführt werden. [37] Zudem ermöglicht Fusion, durch einen großen Funktionsumfang und problemloses kollaboratives Arbeiten, einen strukturierten und produktiven Arbeitsprozess. Auch können externe 3D-Modelle, wie beispielsweise ein Modell des Waveshare Displays, problemlos in Projekte integriert werden. Dies ermöglicht es, das Gehäuse sehr genau an die verwendeten Komponenten anzupassen.

Für die Implementierungsmöglichkeiten wurden zwei Gehäuse entwickelt, die sich hauptsächlich durch ihre Abmessungen voneinander unterscheiden. Das Gehäuse für die Umsetzungsvariante mittels DAC ist in den Abbildungen 45 bis 49 auf den Seiten 109 bis 113 im Anhang dargestellt. Das Gehäuse für die Umsetzungsvariante mittels DDS ist in den Abbildungen 50 bis 54 auf den Seiten 114 bis 118 im Anhang dargestellt. Beide Gehäuse bestehen im Groben aus zwei Teilen: einem Deckel und einem Korpus. Im Deckel wird das HMI mittels vier Schrauben an dafür vorgesehenen Punkten befestigt. Damit das Display nicht beschädigt werden kann, schließt der Deckel an der Oberseite bündig mit diesem ab. Im Falle des DAC-Funktionsgenerators befindet sich im Deckel ebenfalls eine Aussparung für ein USB-C-Kabel, welches für die Stromversorgung verwendet wird. Über vier bzw. sechs Schrauben wird der Deckel mit dem restlichen Gehäuse verbunden.

Die Komponenten für die Spannungsausgabe sowie das akustische und visuelle Feedback werden im Korpus untergebracht. Die einzelnen Komponenten sind dabei auf Lochplatten aufgelötet. Für die DAC-Variante wird eine Lochplatte mit den Maßen 5x7 cm und für die DDS-Variante zwei Lochplatten mit den Maßen 5x7 cm sowie eine Lochplatte mit den Maßen 3x7 cm verwendet. Diese Platten werden mit jeweils vier Schrauben in den dafür vorgesehenen Befestigungspunkten mit dem Korpus verschraubt. Beim Gehäuse des DDS-Funktionsgenerators befindet sich neben den Lochplatten eine Aussparung für den DC-DC Buck Converter sowie eine Aussparung für das Kabel des Netzsteckers in der Wand des Korpus. Damit sich der Buck Converter nicht verschieben kann, ist er von einer Umrahmung umgeben. Unterhalb der Lochplatten befindet sich bei beiden Gehäusen ein viereckiger Rahmen, an welchem der LED-Streifen befestigt wird. In der Wand oberhalb der Lochplatten befinden sich bei beiden Gehäusen Löcher für Bananenbuchsen, über welche die Spannungssignale der Kanäle ausgegeben und im Falle des DDS-Funktionsgenerators der OPV mit Spannung versorgt wird. Unterhalb der Bananenbuchsen sind Aussparungen für Textplaketten zur Beschriftung der Anschlüsse integriert. An den Seiten beider Gehäuse befinden sich zudem Lüftungsschlitze, durch welche ein Überhitzen der Komponenten vermieden und der Ton des Buzzers von außen wahrgenommen werden kann.

Da der microSD-Kartenleser für das Einstecken und Entnehmen der microSD-Karte keinen besonders großen Raum zur Verfügung stellt, wäre dies ohne zusätzliche Hilfsmittel nicht ohne weiteres möglich. Aus diesem Grund wurde ein Halter entwickelt, in dem die microSD-Karte eingeklemmt wird. Dieser Halter kann durch eine seitliche Öffnung im Deckel eingeführt werden, wodurch das Einlegen und Entfernen der microSD-Karte deutlich vereinfacht wird.

8.2. Materialauswahl und Fertigung

Die beiden Gehäuse wurden mittels 3D-Druck angefertigt. Bei der Materialauswahl fiel die Entscheidung auf PLA (Polylactid), welches eines der am häufigsten genutzten Materialien im Bereich des 3D-Drucks ist. Es besitzt einen niedrigen Schmelzpunkt sowie einen geringen Verzug beim Druck und lässt sich dadurch zuverlässig und maßhaltig verarbeiten. [38] Beide Gehäuse wurden in schwarzem PLA gedruckt. Beim Korpus wurde darüber hinaus ein 5,5 mm breiter Abschnitt aus weißem PLA integriert, sodass das Licht des LED-Streifens von außen sichtbar ist.

Zur Befestigung der einzelnen Komponenten sowie der Gehäuseteile untereinander wurden Gewindeeinsätze (Heat-Inserts) verwendet. Diese ermöglichen es, Gewindemuttern thermisch in die Konstruktion einzusetzen, sodass belastbare und wiederverwendbare Schraubverbindungen realisiert werden können, ohne das vergleichsweise weiche PLA durch direktes Einschrauben zu beschädigen. Die Befestigungsschrauben wurden anschließend passend zum metrischen Gewinde der Einsätze sowie zu der durch die Konstruktion vorgegebenen Länge ausgewählt.

Bei der ersten Implementierungsmethode wurden acht Gewindeeinsätze des Typs M2x4x3,2 verwendet. Das HMI wurde an den vier Ecken des Deckels mit Schrauben des Typs M2,5x6 befestigt. Die Lochrasterplatine wurde mit vier Schrauben (M2x6) am Boden des Gehäuses verschraubt. Abschließend wurde der Deckel mit vier weiteren Schrauben (M2x16) mit dem unteren Gehäuseteil verbunden.

Für die zweite Implementierungsmethode wurden 18 Gewindeeinsätze des Typs M2x4x3,2 verwendet. Wie bei der ersten Methode wurde auch hier das HMI mit vier Schrauben (M2,5x6) am Deckel befestigt. Die Lochplatinen wurden mit insgesamt 12 Schrauben (M2x6) im Gehäuse verbaut. Zusätzlich wurde das Netzteil in das Gehäuse eingesetzt und mit Heißkleber fixiert. Damit die MCP42010-ICs leichter ausgetauscht werden können, wurden diese nicht direkt mit der Lochplatine verlötet, sondern in DIP-IC-Sockel, welche mit der Platine verlötet sind, eingesteckt. Abschließend wurde der Deckel mit sechs Schrauben (M2x16) mit dem Korpus verbunden.

Die verwendeten Bananenchucks wurden von der Innenseite des Korpus mittels Muttern befestigt. Um eine schnelle Identifikation der Anschlüsse auch ohne das Lesen der Beschriftungen zu ermöglichen, sind die Buchsen farblich gekennzeichnet. Dabei steht Schwarz für GND, Rot für Kanal 1 beziehungsweise +15 V und Blau für Kanal 2 beziehungsweise -15 V.

9. Fazit

9.1. Vergleich

In den vorangegangenen Kapiteln wurden mit der DAC-basierten Methode und der DDS-basierten Methode zwei verschiedene Ansätze zur Erzeugung der Spannungssignale umgesetzt und jeweils messtechnisch charakterisiert. Trotz ihrer unterschiedlichen Funktionsweisen werden beide Verfahren an denselben Anforderungen, die in den Abschnitten 3.2 und 3.3 für den in Abschnitt 3.1 beschriebenen Laborversuch definiert wurden, bewertet. Welcher der beiden Ansätze sich für den vorgesehenen Anwendungsfall besser eignet, lässt sich erst durch eine direkte Gegenüberstellung beider Methoden beurteilen.

Dieses Kapitel stellt die beiden Implementierungsmethoden daher anhand der zuvor erhobenen Messdaten (Abschnitte 5.4 und 6.4) sowie ergänzender Bewertungskriterien gegenüber. Im Vordergrund steht zunächst die Erfüllung der funktionalen und nicht-funktionalen Anforderungen, insbesondere hinsichtlich der Amplituden-, Phasen- und Frequenzgenauigkeit sowie der Reaktionszeit. Darüber hinaus werden mit dem nutzbaren Frequenz- und Amplitudenbereich, der Schaltungs- und Softwarekomplexität sowie den Bauteilkosten weitere Kriterien betrachtet, die für die praktische Eignung der jeweiligen Methode entscheidend sind. Auf Basis dieser Gegenüberstellung wird abschließend eine begründete Bewertung beider Ansätze vorgenommen.

Die Gegenüberstellung in Tabelle 13 zeigt, dass beide Methoden den Großteil der FA und NFA erfüllen. Beide Varianten sind in der Lage Spannungssignale mit unterschiedlichen Frequenzen, Amplituden und Phasenverschiebungen auf zwei Kanälen auszugeben. Zudem können beide Funktionsgeneratoren über das HMI bzw. die Benutzeroberfläche gesteuert werden. Die Parameter der Spannungssignale können bei beiden Methoden über eine microSD-Karte eingelesen und verarbeitet werden. Durch die entwickelten Chassis besitzen beide Varianten zudem ein labortaugliches Gehäuse. Beide Varianten können durch eine externe Stromversorgung unterbrechungsfrei betrieben werden und speichern die Gruppenparameter beim Ausschalten in einem Zwischenspeicher, sodass sie bei Inbetriebnahme nicht erneut geladen werden müssen. Beide Benutzeroberflächen zeigen Hardwarefehler über dedizierte Fehlermeldungen auf dem Display an und besitzen verständliche Hilfestellungen für die zu verwendenden Laborgeräte.

Variante A erfüllt FA-5 nur teilweise, da eine definierte Phasenverschiebung aufgrund des Auseinanderlaufens der Signale über einen längeren Zeitraum nicht aufrechterhalten werden kann. NFA-1 und NFA-2 gelten als erfüllt, jedoch nur unter Eingrenzung der Frequenz- und oder Spannungsbereiche.

Variante B erfüllt NFA-6 nicht, da die Gesamtreaktionszeit des Funktionsgenerators die vorgegebene Zeit von 100 ms überschreitet. NFA-1 und NFA-2 gelten als erfüllt. Identisch zu Variante A gilt dies jedoch erneut nur unter Einschränkung des Spannungsbereichs und oder Einschränkung des Frequenzbereichs. Eine genaue Auflistung der Begrenzungen im Zusammenhang mit den nicht-funktionalen Anforderungen ist in Tabelle 14 zu finden.

Was die Schaltungs- und Softwarekomplexität betrifft, weist die Implementierungsmethode A deutliche Vorteile auf. Für die Erzeugung des Spannungssignals werden nur der zweite Mikrocontroller

Tabelle 13: Gegenüberstellung der beiden Implementierungsmethoden anhand der funktionalen und nicht-funktionalen Anforderungen

Anforderung	Methode A: DAC	Methode B: DDS
funktionale Anforderungen		
FA-1	+	+
FA-2	+	+
FA-3	+	+
FA-4	+	+
FA-5	o	+
FA-6	+	+
FA-7	+	+
FA-8	+	+
FA-9	+	+
FA-10	+	+
FA-11	+	+
FA-12	+	+
FA-13	+	+
Nicht-funktionale Anforderungen		
NFA-1	+	+
NFA-2	+	+
NFA-3	+	+
NFA-4	+	+
NFA-5	+	+
NFA-6	+	-
NFA-7	+	+
NFA-8	+	+
NFA-9	+	+
NFA-10	+	+
NFA-11	+	+
NFA-12	+	+

+ = Anforderung vollständig erfüllt

o = Anforderung teilweise erfüllt

- = Anforderung nicht erfüllt

sowie zwei Kondensatoren benötigt, wodurch die Komplexität der Schaltung sehr gering ausfällt. Insgesamt verwendet die Implementierungsmethode A 31 Komponenten. Im Gegensatz dazu verwendet die Implementierungsmethode B 81 einzelne Komponenten, wodurch der Gesamtaufbau deutlich komplexer ausfällt. Zudem muss auf einem der AD9833-ICs der 25 MHz Quarz entfernt werden, wodurch der Zugang zu einer Heißluftlötstation oder etwas Vergleichbarem benötigt wird.

Aufgrund der unterschiedlichen Anzahl an Komponenten besitzen die beiden Funktionsgeneratoren ebenfalls unterschiedliche Abmessungen. Variante A zeichnet sich dabei durch seine sehr kompakten Abmessungen von 6 cm Breite, 8 cm Tiefe und 5,32 cm Höhe aus. Im Gegensatz dazu belaufen sich die Abmessungen von Implementierungsmethode B auf 17,84 cm Breite, 8 cm Tiefe und 5,37 cm Höhe. Somit ist das Gehäuse des zweiten Funktionsgenerators rund dreimal breiter.

Durch die in Variante B verwendeten Komponenten sowie die Steuerung der Ausgangsspannung

Tabelle 14: Auflistung der Begrenzungen im Zusammenhang mit den nicht-funktionalen Anforderungen NFA-1 und NFA-2

Kriterium	Methode A: DAC	Methode B: DDS
Amplitudenfehler $\leq \pm 5\%$ (NFA-1)	Sinus: $100 \text{ Hz} \leq f \leq 2 \text{ kHz}$ $500 \text{ mV} < V_{out}$ Rechteck: $10 \text{ Hz} \leq f \leq 1 \text{ kHz}$ $300 \text{ mV} < V_{out} < 1,1 \text{ V}$	Sinus: $1 \text{ kHz} \leq f \leq 80 \text{ kHz}$ Rechteck: $10 \text{ kHz} \leq f \leq 80 \text{ kHz}$ $2 \text{ V} \leq V_{out}$
Phasenfehler $\leq \pm 2^\circ$ (NFA-2)	Sinus: $100 \text{ Hz} \leq f \leq 500 \text{ Hz}$ $300 \text{ mV} < V_{out} < 1 \text{ V}$ Rechteck: $10 \text{ Hz} \leq f \leq 100 \text{ Hz}$	$f < 150 \text{ kHz}$

des Funktionsgenerators mithilfe eines Spannungsteilers besteht zudem eine erhöhte Softwarekomplexität. Neben den Funktionen für die Kommunikation mittels UART und das Einlesen der microSD-Karte, welche für beide Implementierungsmethoden identisch sind, erhöhen die einzelnen Funktionen zur Ansteuerung der Komponenten über das entsprechende Busprotokoll sowie die Funktionen für die Berechnung des Widerstandsverhältnisses des Spannungsteilers die Softwarekomplexität nicht unerheblich.

Auch der Kostenaufwand der beiden Implementierungsmethoden weist drastische Unterschiede auf. Eine Übersicht der Kosten der beiden Implementierungsmethoden ist in Tabelle 17 auf Seite 119 im Anhang zu finden. Dabei fällt auf, dass die Implementierungsmethode B mit Materialkosten von 97,29 € mehr als doppelt so teuer ist wie die Methode A. Neben der größten Kostenposition, dem ESP32-S3 2.8-Zoll Display Development Board mit 25,50 €, welche bei beiden Funktionsgeneratoren zu finden ist, besitzt die Implementierungsmethode A mit dem ESP32 DevKit C V4 nur eine weitere kostenintensive Position. Methode B besitzt zusätzlich zur Kostenposition des ESP32 DevKit C V4 die Positionen der AD9833-, MCP42010- und OPA1612-ICs. Diese Komponenten alleine sind mit einem Preis von 48,81 € für ca. 46 % der Kosten verantwortlich.

9.2. Bewertung und Empfehlung

Anhand der im vorherigen Abschnitt dargestellten Gesichtspunkte hinsichtlich der Erfüllung der funktionalen und nicht-funktionalen Anforderungen, der Schaltungs- und Softwarekomplexität sowie des Kostenvergleichs lässt sich eine eindeutige Bewertung der beiden Implementierungsmethoden vornehmen und eine entsprechende Empfehlung ableiten.

Grundsätzlich hat sich gezeigt, dass sich beide Varianten der Signalerzeugung (DAC und DDS) für die Umsetzung eines zweikanaligen Funktionsgenerators eignen. Beide Funktionsgeneratoren erfüllen nahezu alle gestellten funktionalen und nicht-funktionalen Anforderungen und könnten für die Laborprüfung im Modul Messtechnik ET2013 verwendet werden.

Anhand der Tabelle 14 ist jedoch eindeutig zu erkennen, dass sowohl der Frequenz- als auch der Spannungsbereich bei der Implementierungsmethode A sehr stark begrenzt sind. Unter den ge-

nannten Beschränkungen kann die Sinusausgabe nur im Frequenzbereich von 100 Hz bis 500 Hz bei einer Ausgangsspannung von 300 mV bis 1 V und die Rechteckausgabe in einem Frequenzbereich von 10 Hz bis 100 Hz betrieben werden. Für die Verwendung des Geräts in einem Laborversuch mit 30 Arbeitsgruppen wäre es dennoch möglich, einen Parametersatz zu erstellen, in dem alle Gruppen unterschiedliche Messsignale erhalten.

Bei der Implementierungsmethode B liegt der Frequenzbereich für die Sinusausgabe zwischen 1 kHz und 80 kHz. Für die Rechteckausgabe erstreckt sich der Frequenzbereich bei einer Ausgangsspannung von $V_{out} \geq 2 \text{ V}$ von 10 kHz bis 80 kHz. Somit besitzt Variante B bezogen auf die Breite des nutzbaren Frequenzbereichs bei der Sinusausgabe einen etwa 198-fach größeren Frequenzbereich als Variante A. Bei der Rechteckausgabe erhöht sich dieser Faktor auf etwa 778.

Trotz der größeren Abmessungen des DDS-Funktionsgenerators ist dieser weiterhin ausreichend kompakt, um problemlos auf den Labortischen eingesetzt zu werden. Implementierungsmethode B weist jedoch auch Nachteile auf. So wird die Anforderung NFA-6 mit einer Reaktionszeit von mehr als 100 ms nicht erfüllt. Zwar überschreitet die maximale Reaktionszeit von 120,6 ms den geforderten Wert nur geringfügig, dennoch könnte dies bereits zu einer für den Nutzer wahrnehmbaren Verzögerung zwischen Ein- und Ausgabe führen. Darüber hinaus liegen die Materialkosten der Variante B mit 97,29 € deutlich über denen der Implementierungsmethode A.

Unter Berücksichtigung aller genannten Gesichtspunkte ist abschließend festzuhalten, dass die Implementierungsmethode B trotz der höheren Kosten, der größeren Schaltungs- und Softwarekomplexität sowie der größeren Abmessungen besser für den Einsatz im Modul Messtechnik I ET2013 geeignet ist. Der DDS-Funktionsgenerator bietet einen deutlich größeren Spannungs- und Frequenzbereich und erreicht höhere Genauigkeiten als die Variante A.

Der DAC-Funktionsgenerator stellt hingegen eine geeignete Lösung für Anwendungen mit geringeren Anforderungen an Genauigkeit und Funktionsumfang dar, bei denen ein besonders kompaktes Gerät erforderlich ist. Zudem zeichnet er sich durch seine geringe Schaltungs- und Softwarekomplexität sowie die niedrigen Materialkosten aus, wodurch ein Nachbau mit geringem Aufwand möglich ist. In nahezu allen technischen Bewertungskriterien ist er dem DDS-Funktionsgenerator jedoch unterlegen.

Der vollständige Quellcode, die Schaltpläne sowie die ergänzende technische Dokumentation beider Implementierungsmethoden sind im zugehörigen GitHub-Repository unter der folgenden URL öffentlich einsehbar:

<https://github.com/Kenn-ing/Studienprojekt-I-Konzeption-und-Entwicklung-eines-Zweikanal-Funktionsgenerators>

Quellenverzeichnis

- [1] Hochschule für Wirtschaft und Recht Berlin (Hrsg.): Porträt, <https://www.hwr-berlin.de/hwr-berlin/ueber-uns/portraet>, 2026 (zuletzt abgerufen am 24.06.2026).
- [2] Hochschule für Wirtschaft und Recht Berlin (Hrsg.): Studiengangsbeschreibung für den Dualen Studiengang Industrielle Elektrotechnik, <https://moodle.hwr-berlin.de/mod/resource/view.php?id=1133469>, 21.09.2023 (zuletzt abgerufen am 24.06.2026).
- [3] Furtado, Luis Fernando Ferreira (Professur für Elektrotechnik/Automatisierungstechnik an der Hochschule für Wirtschaft und Recht Berlin): Interview zu den Rahmenbedingungen und Zielen des Studienprojekts, 21.01.2026, Hochschule für Wirtschaft und Recht Berlin Alt-Friedrichsfelde 60.
- [4] Stiftung Radiomuseum Luzern (Hrsg.): Funktionsgenerator HM-8030-4, https://www.radiomuseum.org/r/hameg_funktionsgenerator_hm_803.html# (zuletzt abgerufen am 24.06.2026).
- [5] COPA-DATA GmbH (Hrsg.): E-Book zur Mensch-Maschine-Schnittstelle, <https://go.copadata.com/de/free-hmi-ebook> (zuletzt abgerufen am 24.06.2026).
- [6] Berliner Wasserbetriebe AöR (Hrsg.): Ausführungsbeschreibung Abwasserpumpwerke, <https://aquanet.berlinwasser.de/standard?id=a59fbddc-6ef1-40e6-b09f-fe963deeb043> (internes Dokument), April 2023 (zuletzt abgerufen am 24.06.2026).
- [7] Bernstein, Herbert: Mikrocontroller - Grundlagen der Hard- und Software der Mikrocontroller ATtiny2313, ATtiny26 ATmega32 und ATmega3282, Springer Vieweg (Hrsg.), 3. Auflage, 2026 (zuletzt abgerufen am 24.06.2026).
- [8] Hercog, Darko; Lerher, Tone; Truntič, Mitja; Težak, Oto: Design and Implementation of ESP32-Based IoT Devices, <https://www.mdpi.com/1424-8220/23/15/6739>, MDPI Journals (Hrsg.), 23.07.2023 (zuletzt abgerufen am 24.06.2026).
- [9] Espressif Systems (Hrsg.): About Espressif, <https://www.espressif.com/en/company/about-espressif> (zuletzt abgerufen am 24.06.2026).
- [10] Espressif Systems (Hrsg.): Modules, <https://www.espressif.com/en/products/modules> (zuletzt abgerufen am 24.06.2026).
- [11] Murphy, Eva; Slattery, Colm: All About Direct Digital Synthesis, Analog Dialogue, Vol. 38 (Nr. 08), Analog Devices Inc. (Hrsg.), August 2004, <https://www.analog.com/media/en/analog-dialogue/volume-38/number-3/articles/all-about-direct-digital-synthesis.pdf> (zuletzt abgerufen am 24.06.2026).
- [12] Espressif Systems (Hrsg.): ESP32 Series Datasheet, Version 5.2, https://documentation.espressif.com/esp32_datasheet_en.pdf (zuletzt abgerufen am 24.06.2026).
- [13] Furtado, Luis Fernando Ferreira (Hrsg.): Laborarbeit ET2013 Messtechnik I – Labor Messtechnik, 03.12.2025 (zuletzt abgerufen am 24.06.2026).

- [14] Furtado, Luis Fernando Ferreira (Hrsg.): Laborarbeit (Aufgabe 1) - JG25 ET2013 Messtechnik I – Labor Messtechnik, 2025 (zuletzt abgerufen am 24.06.2026).
- [15] Berrybase (Hrsg.): Datenblatt zu Whaveshare ESP32-S3-Touch-LCD-2.8, <https://www.berrybase.de/product-datasheet/0194271e99387062a6342fa5528c82d6/create> (zuletzt abgerufen am 24.06.2026).
- [16] Waveshare (Hrsg.): ESP32-S3-Touch-LCD-2.8, <https://www.waveshare.com/esp32-s3-touch-lcd-2.8.htm> (zuletzt abgerufen am 24.06.2026).
- [17] Espressif Systems (Hrsg.): ESP32-WROOM-32 Datasheet, Version 3.6, https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf (zuletzt abgerufen am 24.06.2026).
- [18] Espressif Systems (Hrsg.): ESP-32 NodeMCU Development Board DevKit C V4, <https://www.az-delivery.de/cdn/shop/products/esp-32-dev-kit-c-v4-895689.jpg?v=1679398546&width=1200> (zuletzt abgerufen am 24.06.2026).
- [19] Waveshare (Hrsg.): Wiki zum ESP32-S3-Touch-LCD-2.8, <https://docs.waveshare.com/ESP32-S3-Touch-LCD-2.8> (zuletzt abgerufen am 24.06.2026).
- [20] Espressif Systems (Hrsg.): ESP32 Technical Reference Manual, Version 5.7, https://documentation.espressif.com/esp32_technical_reference_manual_en.pdf (zuletzt abgerufen am 24.06.2026).
- [21] Espressif Systems (Hrsg.): ESP32 Datenblatt (ESP32-D0WD-V3), Version 5.2, https://documentation.espressif.com/esp32_datasheet_en.pdf#cd-peri-pin-config (zuletzt abgerufen am 24.06.2026).
- [22] Espressif Systems (Hrsg.): Überblick zu Digital To Analog Converter (DAC), <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/dac.html> (zuletzt abgerufen am 24.06.2026).
- [23] Keysight Technologies Inc. (Hrsg.): Keysight InfiniiVision 2000 X-Series Oszilloskope - Benutzerhandbuch, 13. Ausgabe, Handbuchteilenummer 75015-97143, August 2025 (zuletzt abgerufen am 24.06.2026).
- [24] Murata Manufacturing Co. Ltd. (Hrsg.): What are impedance/ESR frequency characteristics in capacitors?, <https://article.murata.com/en-us/article/impedance-esr-frequency-characteristics-in-capacitors> (zuletzt abgerufen am 24.06.2026).
- [25] Göbel, Holger: Einführung in die Halbleiter-Schaltungstechnik, Springer Vieweg (Hrsg.), 6. aktualisierte Auflage, 2019 (zuletzt abgerufen am 24.06.2026).
- [26] Analog Devices (Hrsg.): Low Power, 12.65 mW, 2.3 V to 5.5 V, Programmable Waveform Generator, <https://www.analog.com/media/en/technical-documentation/datasheets/ad9833.pdf>, 2019 (zuletzt abgerufen am 24.06.2026).

- [27] anzado GmbH (Hrsg.): AD9833 Signalgenerator Modul, <https://www.roboter-bausatz.de/p/ad9833-signalgenerator-modul> (zuletzt abgerufen am 24.06.2026).
- [28] Deltas Semiconductor; Maxim Integrated: DS1803 Addressable Dual Digital Potentiometer, <https://www.analog.com/media/en/technical-documentation/data-sheets/DS1803.pdf>, Analog Devices (Hrsg.) (zuletzt abgerufen am 24.06.2026).
- [29] Microchip Technology: MCP41XXX/42XXX Single/Dual Digital Potentiometer with SPI Interface, https://cdn-reichelt.de/documents/datenblatt/A200/MCP42010_MIC.pdf, reichelt elektronik (Hrsg.), 2003 (zuletzt abgerufen am 24.06.2026).
- [30] Texas Instruments: OPA161x SoundPlus High-Performance, Bipolar-Input Audio Operational Amplifiers, https://www.ti.com/lit/ds/symlink/opa1612.pdf?ts=1779781533360&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252Fde-de%252FOPA1612, 2014 (zuletzt abgerufen am 24.06.2026).
- [31] Brandon, David: AN-587 APPLICATION NOTE - Synchronizing Multiple AD9850/AD9851 DDS-Based Synthesizers, <https://www.analog.com/media/en/technical-documentation/application-notes/an-587.pdf>, Analog Devices (Hrsg.), 2002 (zuletzt abgerufen am 24.06.2026).
- [32] Analog Devices (Hrsg.): Choosing the Correct digiPOT for Your Application, https://www.analog.com/media/en/news-marketing-collateral/product-selection-guide/choosing_the_correct_digipot.pdf, 2014 (zuletzt abgerufen am 24.06.2026).
- [33] ROHM Semiconductor GmbH (Hrsg.): Typical Parameters of Opamps Amplification Factor and Voltage Gain, <https://techweb.rohm.com/product/opamps-arithmetic-circuits/opamps/23786/> (zuletzt abgerufen am 24.06.2026).
- [34] Tervo, Richard J.: Practical Signals Theory with MATLAB Applications, https://www.google.de/books/edition/Practical_Signals_Theory_with_MATLAB_App/dQBeAAAQBAJ?hl=de&gbpv=1&dq=square+signal+fourier+series&pg=PA128&printsec=frontcover, John Wiley & Sons (Hrsg.), 2013 (zuletzt abgerufen am 24.06.2026).
- [35] OpenELAB Technology Ltd. (Hrsg.): EEZ Studio: Open-Source-Plattform für modulares Testen, <https://openelab.io/de/blogs/learn/eez-studio-an-open-source-tool-for-modular-test-and-measurement?srsId=AfmB0oq2Q4taiAQBWYxQB7EUu20fdKpgNuV2eWc63evtv32gQ33pUw0k> (zuletzt abgerufen am 24.06.2026).
- [36] LVGL Ltd. (Hrsg.): Introduction - LVGL Documentation, <https://lvgl.io/docs/open/introduction> (zuletzt abgerufen am 24.06.2026).
- [37] Autodesk (Hrsg.): Autodesk Fusion: nicht einfach nur CAD, sondern die Zukunft der Konstruktion und Fertigung, <https://www.autodesk.com/de/products/fusion-360/overview> (zuletzt abgerufen am 24.06.2026).

- [38] Material 4 Print GmbH & Co. KG: Informationen zum PLA Filament (Polylactide), <https://www.material4print.de/blogs/materialien/informationen-zum-pla-filament-poly lactide?srsltid=AfmB0oqENEOqoBnlU-1Vw0-QUY5nlYSp4o6nmgkqegFb3aEHAfoIj-0J> (zuletzt abgerufen am 24.06.2026).

Anhang

A1 Gruppenparameter des Laborversuchs im Modul Messtechnik I (ET2013)

Tabelle 15: Parameter des sinusförmigen Spannungssignals der Arbeitsgruppen 1 bis 4 für die Laborarbeit im Modul Messtechnik I (ET2013) [14, S. 1]

Gruppe	Kanal 1										Kanal 2										Phase			
	T_1 [s]	f_1 [Hz]	ω_1 [rad/s]	$\ddot{u}_1$ [V]	$\dot{u}_1$ [V]	u_{e1} [V]	U_{rms1} [V]	T_2 [s]	f_2 [Hz]	ω_2 [rad/s]	$\ddot{u}_2$ [V]	$\dot{u}_2$ [V]	u_{e2} [V]	U_{rms2} [V]	Δt [s]	φ [°]	φ [rad]	φ [°]	φ [rad]	φ [°]	φ [rad]	φ [°]	φ [rad]	φ [°]
1	1,54E-04	6500	40840,7	0	2,5	-2,5	5	1,7677	1,54E-04	6500	40840,7	-2	0,5	-4,5	5	2,6693	5,13E-05	120	2,094	0,333				
2	1,04E-04	9600	60318,6	0	4,5	-4,5	9	3,1819	1,04E-04	9600	60318,6	-1	-0,5	-1,5	1	1,0606	1,45E-06	5	0,087	0,014				
3	1,85E-04	5400	33929,2	0	0,5	-0,5	1	0,3535	1,85E-04	5400	33929,2	-2	1	-5	6	2,9154	3,60E-05	70	1,222	0,194				
4	1,56E-04	6400	40212,4	0	0,5	-0,5	1	0,3535	1,56E-04	6400	40212,4	2	5	-1	6	2,9154	3,47E-05	80	1,396	0,222				

Phasenverschiebungen (Δt , φ [°], φ [rad], φ [°]) jeweils von Kanal 2 bezogen auf Kanal 1.

Tabelle 16: Parameter des rechteckförmigen Spannungssignals der Arbeitsgruppen 1 bis 4 für die Laborarbeit im Modul Messtechnik I (ET2013) [14, S. 2]

Gruppe	T[s]	f[Hz]	ω [rad/s]	$\ddot{u}$ to $\hat{u}$ [V]	0 to $\hat{u}$ [V]	$\hat{u}$ [V]	$\ddot{u}$ [V]	U_e [V]	U_{rms} [V]	τ [s]	$D = \tau / T$ [%]
1	1,75E-04	5700	35814,2	8	4	12	4	8	4,87E+00	2,81E-05	16
2	2,17E-04	4600	28902,7	4	2	6	2	4	3,07E+00	9,78E-05	45
3	1,89E-04	5300	33300,9	5	2	8	2	6	3,49E+00	9,62E-05	51
4	1,72E-04	5800	36442,5	8	4	12	4	8	7,34E+00	0,000136	79

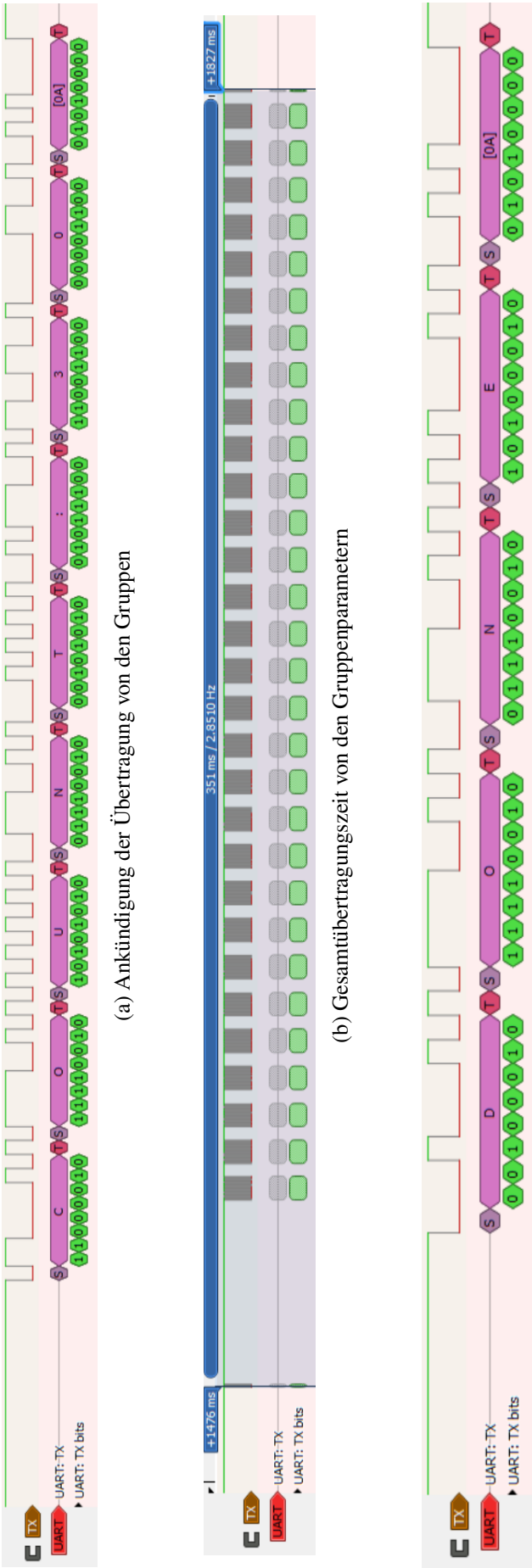
A2 UART-Kommunikation zwischen Master und Slave

Listing 1: CSV-Datei parsen und die Parameter intern im Puffer zwischenspeichern (csv.cpp, Zeilen 56-95)

```
56  /**
57   * Parst eine CSV-Zeile und befüllt eine SignalGroup.
58   * Reihenfolge: Gruppennummer, A1, F1, P1, S1, A2, F2, P2, S2
59   */
60  static void csv_parse_line(const char *line_buffer, SignalGroup &group) {
61      const char *pointer = line_buffer;
62      char value_buffer[32];
63
64      memset(&group, 0, sizeof(group)); // Gruppe mit 0 initialisieren
65
66      // Spalte 1: Gruppen-ID
67      csv_read_value(pointer, value_buffer);
68      group.group_id = (uint8_t)atoi(value_buffer);
69
70      // CH1 Parameter: Spalte 2 bis 5
71      csv_read_value(pointer, value_buffer);
72      group.ch1.amplitude_mV = atof(value_buffer); // Amplitude in mV
73
74      csv_read_value(pointer, value_buffer);
75      group.ch1.frequency_Hz = atof(value_buffer); // Frequenz in Hz
76
77      csv_read_value(pointer, value_buffer);
78      group.ch1.phase_deg = atof(value_buffer); // Phasenwinkel in Grad
79
80      csv_read_value(pointer, value_buffer);
81      group.ch1.signalType = atoi(value_buffer); // Signalform
82
83      // CH2 Parameter: Spalte 6 bis 9
84      csv_read_value(pointer, value_buffer);
85      group.ch2.amplitude_mV = atof(value_buffer); // Amplitude in mV
86
87      csv_read_value(pointer, value_buffer);
88      group.ch2.frequency_Hz = atof(value_buffer); // Frequenz in Hz
89
90      csv_read_value(pointer, value_buffer);
91      group.ch2.phase_deg = atof(value_buffer); // Phasenwinkel in Grad
92
93      csv_read_value(pointer, value_buffer);
94      group.ch2.signalType = atoi(value_buffer); // Signalform
95  }
```

Listing 2: Parser für eine einzelne Gruppenzeile (uart.cpp, Zeilen 32-59)

```
32 static bool parse_group(const String& line, SignalGroup& out) {
33     unsigned int id;
34     float a1, f1, p1, a2, f2, p2;
35     uint8_t s1, s2;
36
37     int scanned = sscanf(line.c_str(),
38         "G:%u,A1:%f,F1:%f,P1:%f,S1:%hhu,A2:%f,F2:%f,P2:%f,S2:%hhu",
39         &id, &a1, &f1, &p1, &s1, &a2, &f2, &p2, &s2);
40     // Alle 9 Felder aus dem String lesen
41     if (scanned != 9) {
42         Serial.printf("Parse-Fehler: nur %d/9 Felder (%s)\n",
43             scanned, line.c_str());
44         return false;
45     }
46
47     // Gelesene Werte in die SignalGroup schreiben
48     out.group_id = (uint8_t)id;
49     out.ch1.amplitude_mV = a1;
50     out.ch1.frequency_Hz = f1;
51     out.ch1.phase_deg = p1;
52     out.ch1.signalType = s1;
53     out.ch2.amplitude_mV = a2;
54     out.ch2.frequency_Hz = f2;
55     out.ch2.phase_deg = p2;
56     out.ch2.signalType = s2;
57
58     return true;
59 }
```



The schematic diagram illustrates a functional generator circuit. It features an ESP32-S3 microcontroller (U2) and an LM358P op-amp (U1). The circuit is powered by a 5V supply. A buzzer (BZ1) is connected to the microcontroller. Two 100nF capacitors (C1, C2) are used for timing. An LED (D1) is connected to the microcontroller. The schematic shows the internal connections of the components and the microcontroller pins used.

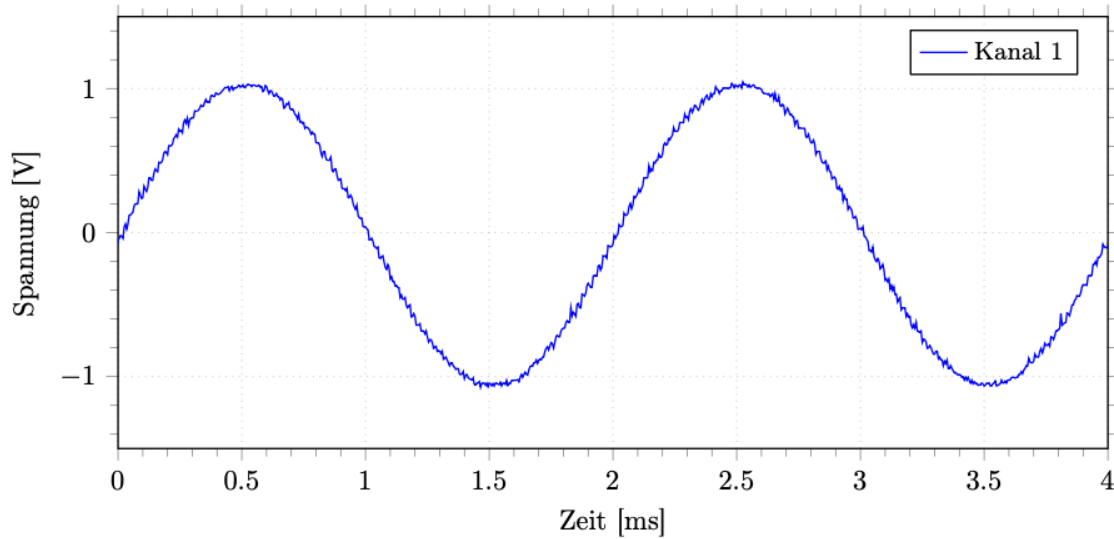
Component Pin Connections:

- U1 (LM358P):**
 - 1: LMA_IN/RF
 - 4: CHIP_PU/RESET
 - 21: GPIO15/ADC2_CH4/XTAL_32K_P
 - 22: GPIO16/ADC2_CH5/XTAL_32K_N
 - 53: XTAL_N
 - 54: XTAL_P
 - 44: MTCK-/TAG/GPIO39
 - 45: WTD0-/TAG/GPIO40
 - 47: WTD1-/TAG/GPIO41
 - 48: WTD2-/TAG/GPIO42
 - 28: SPLCS1/GPIO26
 - 30: SPIHD/GPIO27
 - 31: SPIWP/GPIO28
 - 32: SPICS0/GPIO29
 - 33: SPICLK_P/GPIO30
 - 34: SPID/GPIO31
 - 35: SPID/GPIO32
 - 36: SPID/GPIO33
 - 37: GPIO34
 - 38: GPIO35
 - 39: GPIO36
 - 40: GPIO37
- U2 (ESP32-S3):**
 - 3V3: EN
 - VP: VP
 - 34: GPIO34
 - 35: GPIO35
 - 36: GPIO36
 - 37: GPIO37
 - 38: GPIO38
 - 39: GPIO39
 - 40: GPIO40
 - 41: GPIO41
 - 42: GPIO42
 - 43: GPIO43
 - 44: GPIO44
 - 45: GPIO45
 - 46: GPIO46
 - 47: GPIO47
 - 48: GPIO48
 - 49: GPIO49
 - 50: GPIO50
 - 51: GPIO51
 - 52: GPIO52
 - 53: GPIO53
 - 54: GPIO54
 - 55: GPIO55
 - 56: GPIO56
 - 57: GPIO57
 - 58: GPIO58
 - 59: GPIO59
 - 60: GPIO60
 - 61: GPIO61
 - 62: GPIO62
 - 63: GPIO63
 - 64: GPIO64
 - 65: GPIO65
 - 66: GPIO66
 - 67: GPIO67
 - 68: GPIO68
 - 69: GPIO69
 - 70: GPIO70
 - 71: GPIO71
 - 72: GPIO72
 - 73: GPIO73
 - 74: GPIO74
 - 75: GPIO75
 - 76: GPIO76
 - 77: GPIO77
 - 78: GPIO78
 - 79: GPIO79
 - 80: GPIO80
 - 81: GPIO81
 - 82: GPIO82
 - 83: GPIO83
 - 84: GPIO84
 - 85: GPIO85
 - 86: GPIO86
 - 87: GPIO87
 - 88: GPIO88
 - 89: GPIO89
 - 90: GPIO90
 - 91: GPIO91
 - 92: GPIO92
 - 93: GPIO93
 - 94: GPIO94
 - 95: GPIO95
 - 96: GPIO96
 - 97: GPIO97
 - 98: GPIO98
 - 99: GPIO99
 - 100: GPIO100
 - 101: GPIO101
 - 102: GPIO102
 - 103: GPIO103
 - 104: GPIO104
 - 105: GPIO105
 - 106: GPIO106
 - 107: GPIO107
 - 108: GPIO108
 - 109: GPIO109
 - 110: GPIO110
 - 111: GPIO111
 - 112: GPIO112
 - 113: GPIO113
 - 114: GPIO114
 - 115: GPIO115
 - 116: GPIO116
 - 117: GPIO117
 - 118: GPIO118
 - 119: GPIO119
 - 120: GPIO120
 - 121: GPIO121
 - 122: GPIO122
 - 123: GPIO123
 - 124: GPIO124
 - 125: GPIO125
 - 126: GPIO126
 - 127: GPIO127
 - 128: GPIO128
 - 129: GPIO129
 - 130: GPIO130
 - 131: GPIO131
 - 132: GPIO132
 - 133: GPIO133
 - 134: GPIO134
 - 135: GPIO135
 - 136: GPIO136
 - 137: GPIO137
 - 138: GPIO138
 - 139: GPIO139
 - 140: GPIO140
 - 141: GPIO141
 - 142: GPIO142
 - 143: GPIO143
 - 144: GPIO144
 - 145: GPIO145
 - 146: GPIO146
 - 147: GPIO147
 - 148: GPIO148
 - 149: GPIO149
 - 150: GPIO150
 - 151: GPIO151
 - 152: GPIO152
 - 153: GPIO153
 - 154: GPIO154
 - 155: GPIO155
 - 156: GPIO156
 - 157: GPIO157
 - 158: GPIO158
 - 159: GPIO159
 - 160: GPIO160
 - 161: GPIO161
 - 162: GPIO162
 - 163: GPIO163
 - 164: GPIO164
 - 165: GPIO165
 - 166: GPIO166
 - 167: GPIO167
 - 168: GPIO168
 - 169: GPIO169
 - 170: GPIO170
 - 171: GPIO171
 - 172: GPIO172
 - 173: GPIO173
 - 174: GPIO174
 - 175: GPIO175
 - 176: GPIO176
 - 177: GPIO177
 - 178: GPIO178
 - 179: GPIO179
 - 180: GPIO180
 - 181: GPIO181
 - 182: GPIO182
 - 183: GPIO183
 - 184: GPIO184
 - 185: GPIO185
 - 186: GPIO186
 - 187: GPIO187
 - 188: GPIO188
 - 189: GPIO189
 - 190: GPIO190
 - 191: GPIO191
 - 192: GPIO192
 - 193: GPIO193
 - 194: GPIO194
 - 195: GPIO195
 - 196: GPIO196
 - 197: GPIO197
 - 198: GPIO198
 - 199: GPIO199
 - 200: GPIO200

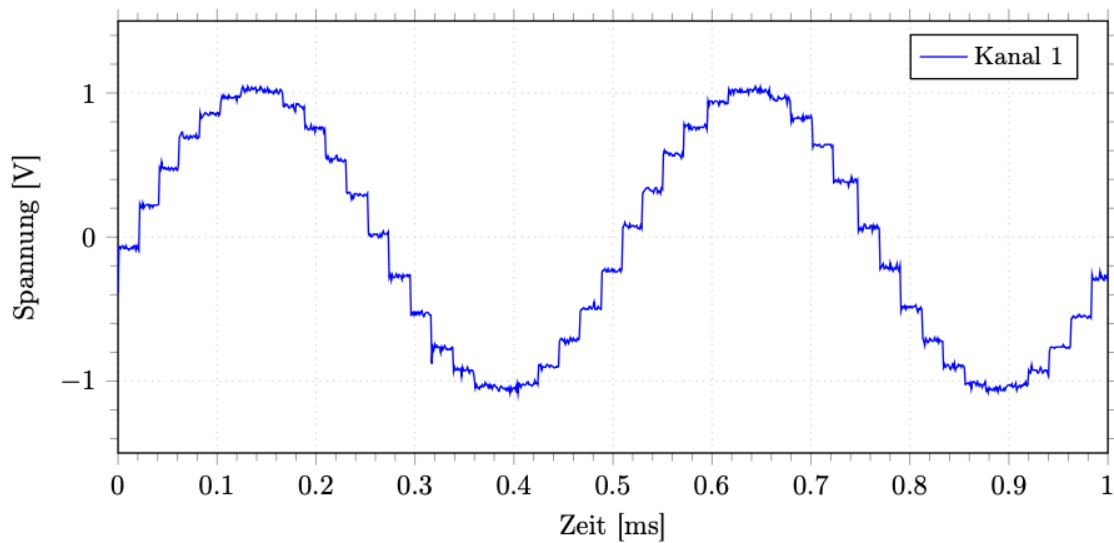
Abbildung 13: Schaltplan der DAC-basierten Signalerzeugung (Methode A)

Listing 3: Inhalt der Funktion `dac_output_loop()` (`dac_output.cpp`, Zeilen 84-126)

```
84  /**
85   * Hauptschleife der DAC-Ausgabe wird kontinuierlich in loop() aufgerufen.
86   * Berechnet die aktuellen DAC-Werte anhand der Phasenakkumulatoren.
87   * Phasenakkumulator-Methode: kein Overflow-Problem, beide Kanäle bleiben
      synchron.
88   */
89  void dac_output_loop() {
90      if (!aktiv) return;
91
92      // Zeitdifferenz seit letztem Aufruf berechnen
93      uint32_t now = micros();
94      uint32_t dtUs = now - lastMicros;
95      lastMicros = now;
96      float dt = dtUs * 1e-6f; // Umrechnung in Sekunden
97
98      // Phasenakkumulatoren aktualisieren
99      phi_acc1 += 2.0f * PI * f_CH1 * dt;
100     phi_acc2 += 2.0f * PI * f_CH2 * dt;
101
102     // Im Bereich 0..2pi halten damit die Werte nicht unendlich wachsen
103     if (phi_acc1 >= 2.0f * PI) phi_acc1 -= 2.0f * PI;
104     if (phi_acc2 >= 2.0f * PI) phi_acc2 -= 2.0f * PI;
105
106     float center = MAX_HALF_COUNTS;
107     float wave1, wave2;
108
109     // CH1 Sinus oder Rechteck
110     if (waveMode1 == 0) {
111         wave1 = sinf(phi_acc1); // Sinussignal
112     } else {
113         wave1 = (sinf(phi_acc1) >= 0.0f) ? 1.0f : -1.0f; // Rechtecksignal
114     }
115
116     // CH2 Sinus oder Rechteck
117     if (waveMode2 == 0) {
118         wave2 = sinf(phi_acc2); // Sinussignal
119     } else {
120         wave2 = (sinf(phi_acc2) >= 0.0f) ? 1.0f : -1.0f; // Rechtecksignal
121     }
122
123     // DAC-Werte ausgeben begrenzt auf 0..255
124     dacWrite(DAC_PIN_CH1, constrain((int)roundf(center + amp1 * wave1), 0, 255));
125     dacWrite(DAC_PIN_CH2, constrain((int)roundf(center + amp2 * wave2), 0, 255));
126 }
```

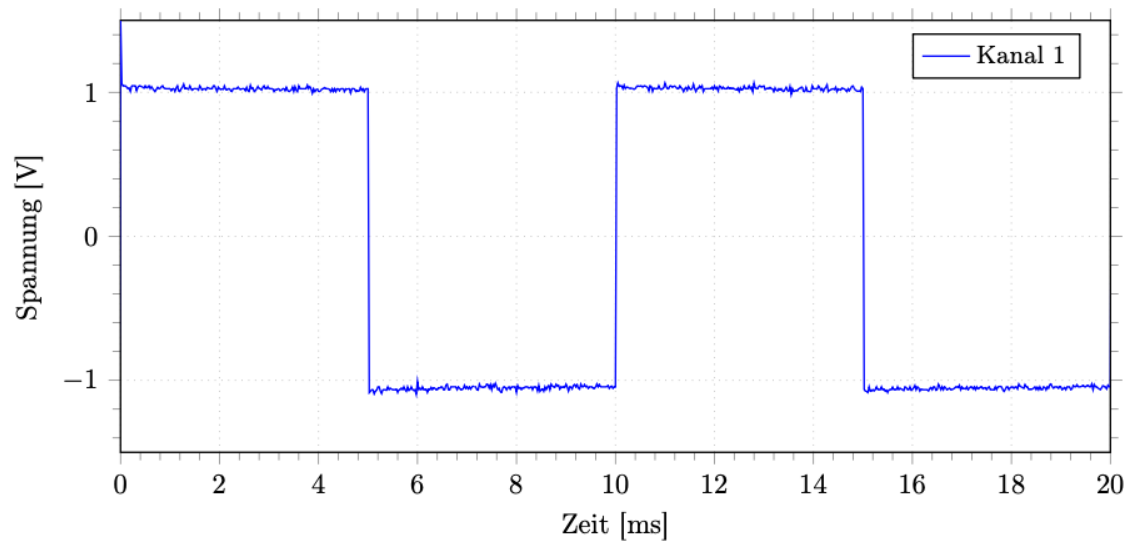


(a) Ausgangssignal (Sinus) bei einer Frequenz von 500 Hz

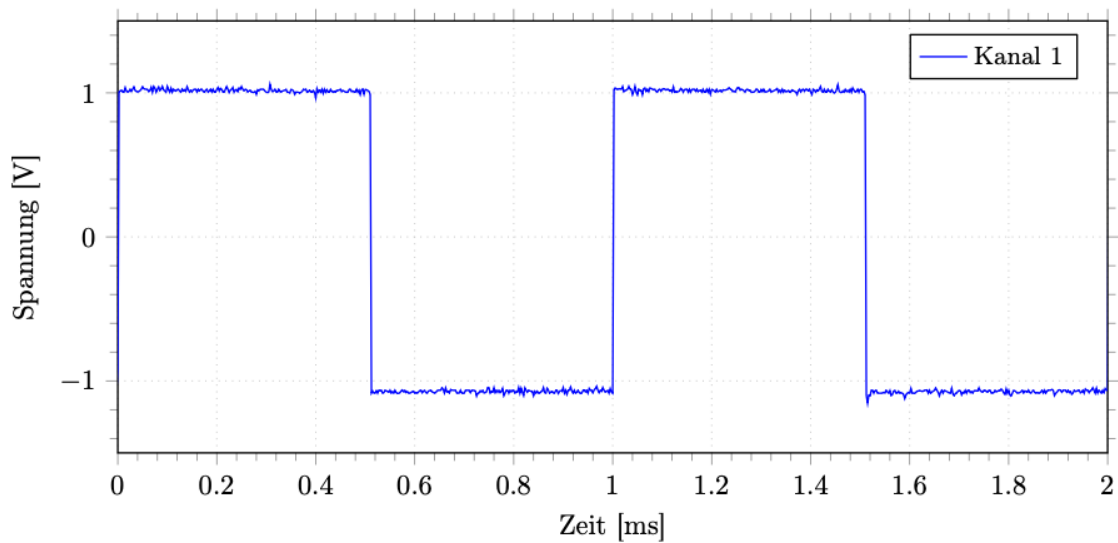


(b) Ausgangssignal (Sinus) bei einer Frequenz von 2 kHz

Abbildung 14: Ausgangssignale der DAC Pins mit verschiedener Frequenz und einer 1 V Amplitude (Sinus)



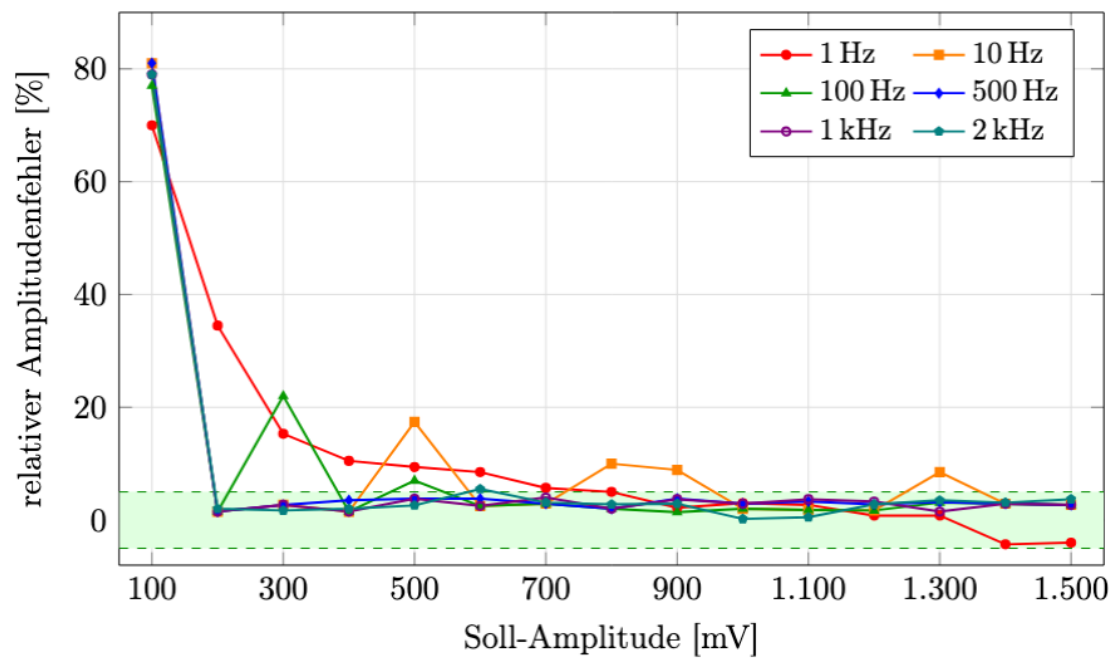
(a) Ausgangssignal (Rechteck) bei einer Frequenz von 100 Hz



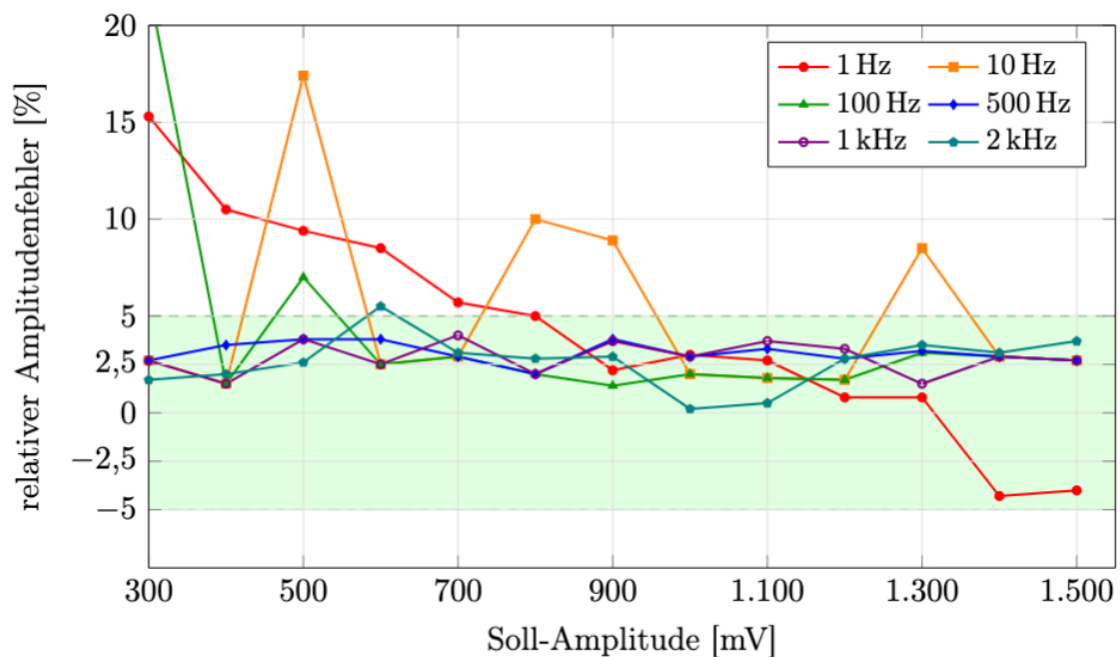
(b) Ausgangssignal (Rechteck) bei einer Frequenz von 1 kHz

Abbildung 15: Ausgangssignale der DAC Pins mit verschiedener Frequenz und einer 1 V Amplitude (Rechteck)

A4 Messergebnisse der Implementierungsmethode A (DAC)



(a) Relativer Amplitudenfehler der DAC-Signalerzeugung (Sinus)



(b) Relativer Amplitudenfehler der DAC-Signalerzeugung (Sinus) ab 300 mV

Abbildung 16: Relativer Amplitudenfehler der DAC-Signalerzeugung (Sinus) in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen

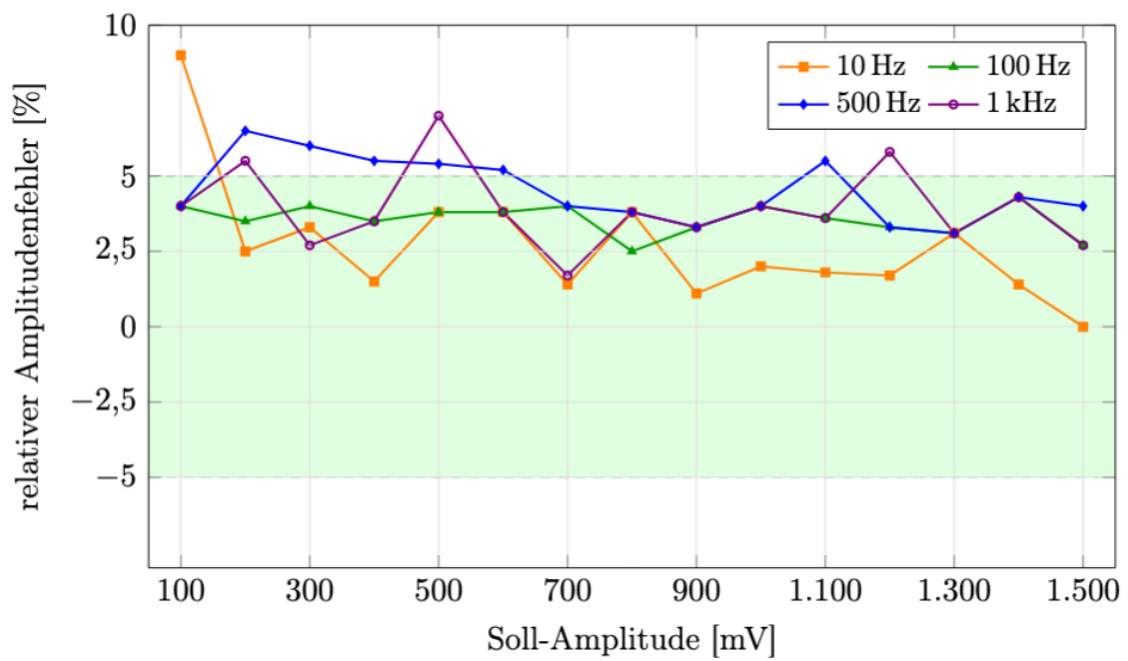
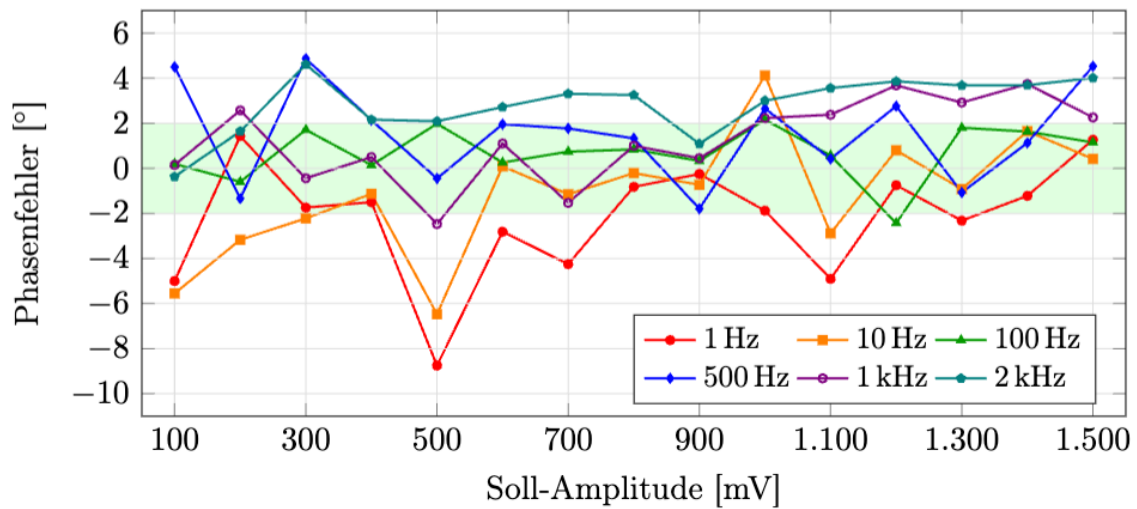
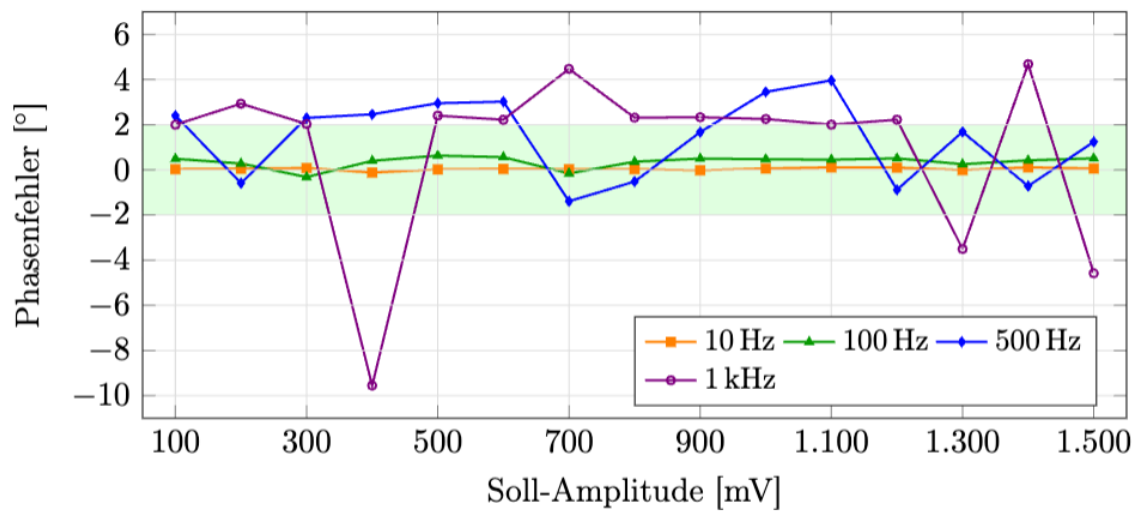


Abbildung 17: Relativer Amplitudenfehler der DAC-Signalerzeugung (Rechteck) in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen

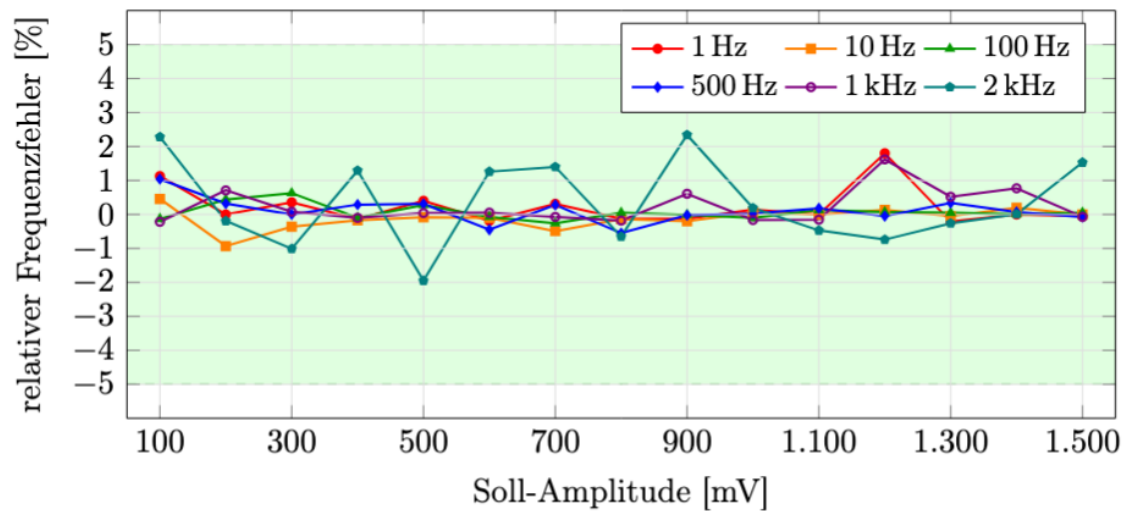


(a) Phasenfehler in Abhängigkeit der Ausgangsspannung für alle getesteten Frequenzstufen (Sinus)

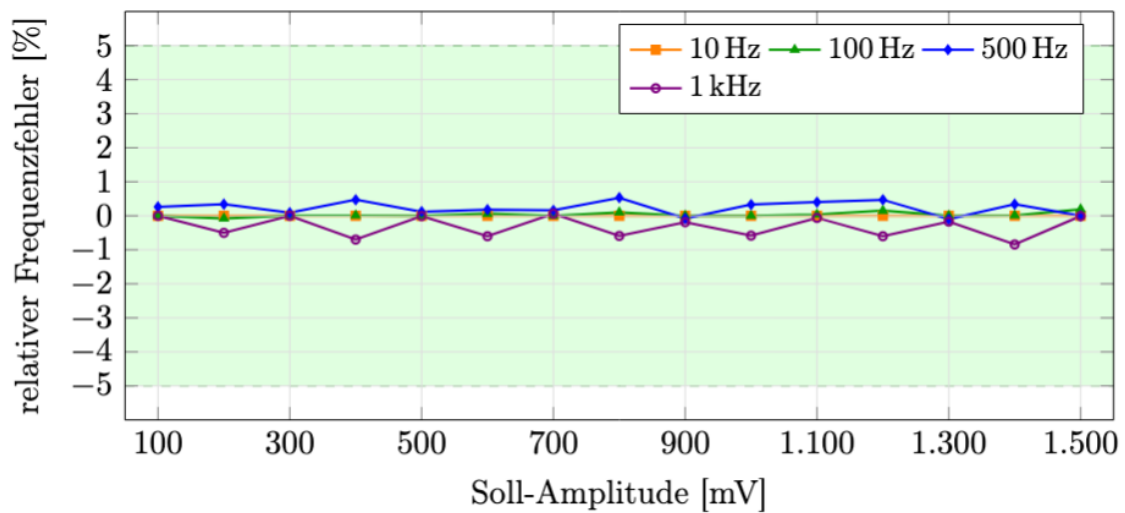


(b) Phasenfehler in Abhängigkeit der Ausgangsspannung für alle getesteten Frequenzstufen (Rechteck)

Abbildung 18: Phasenfehler der DAC-Signalerzeugung in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen



(a) Relativer Frequenzfehler der DAC-Signalerzeugung (Sinus) in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen



(b) Relativer Frequenzfehler der DAC-Signalerzeugung (Rechteck) in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen

Abbildung 19: Relativer Frequenzfehler in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen

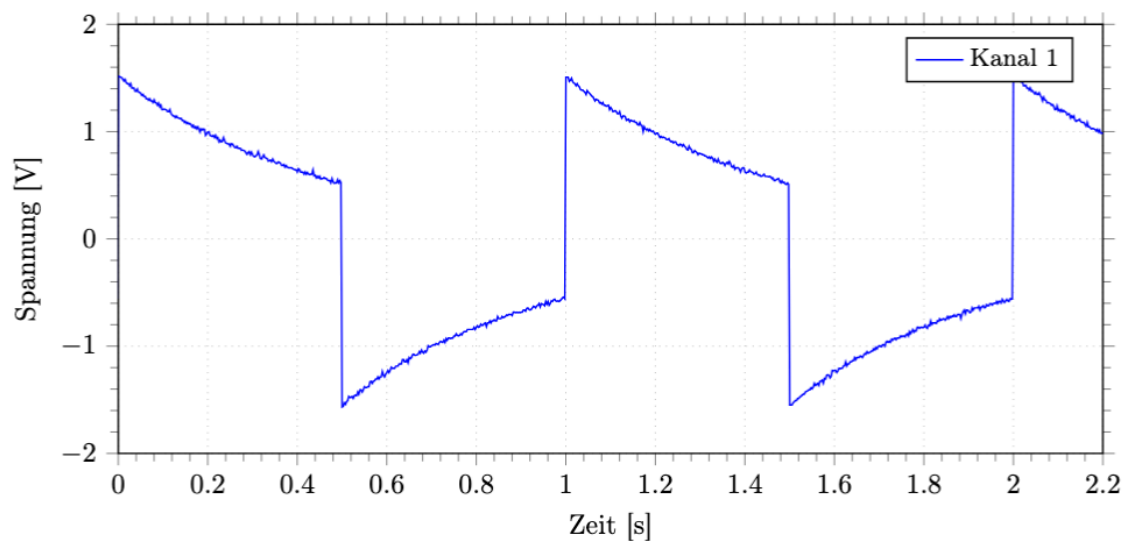
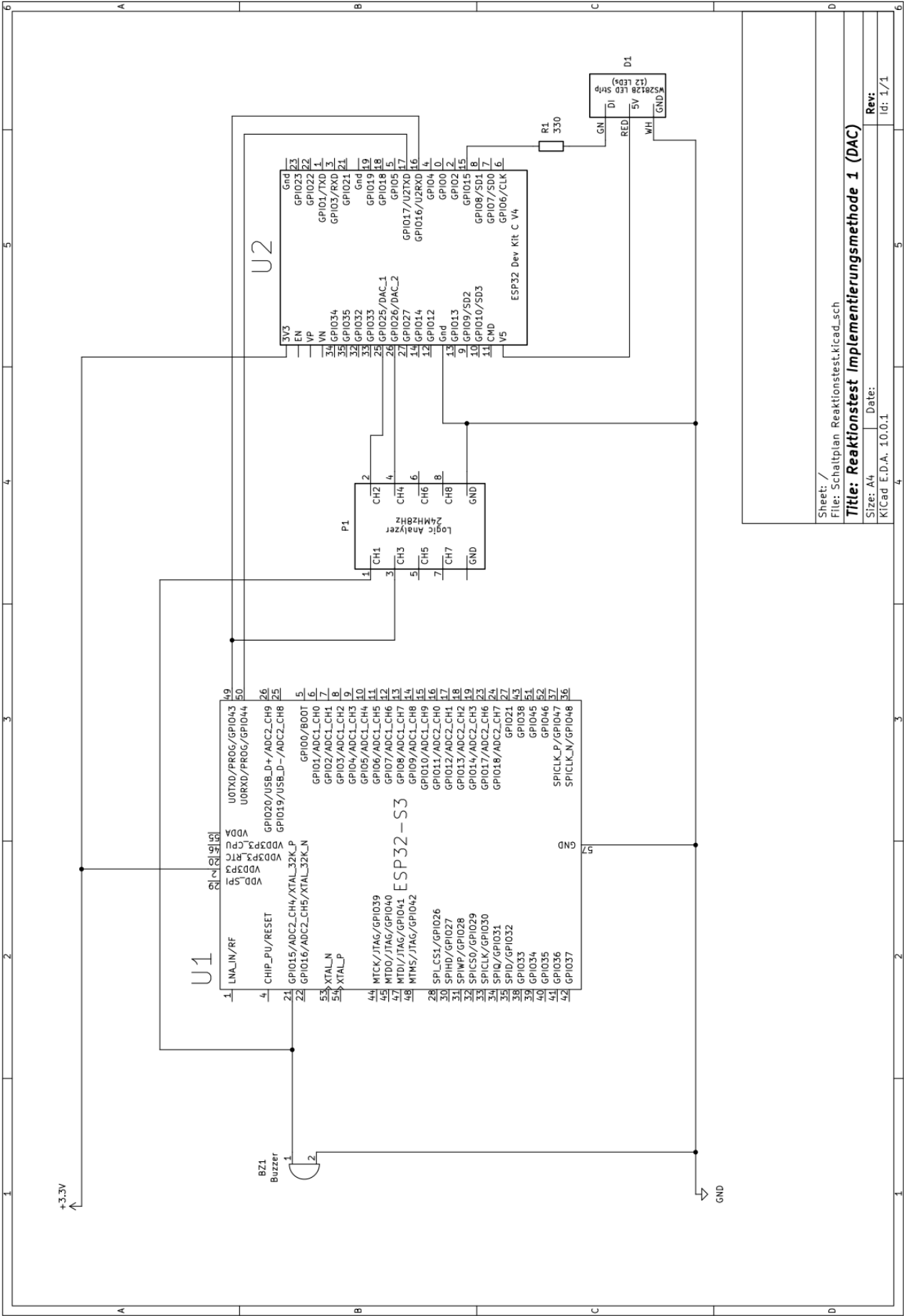


Abbildung 20: Ausgangssignal (Rechteck) des DAC-Funktionsgenerators bei einer Frequenz von 1 Hz und einer Amplitude von 1 V



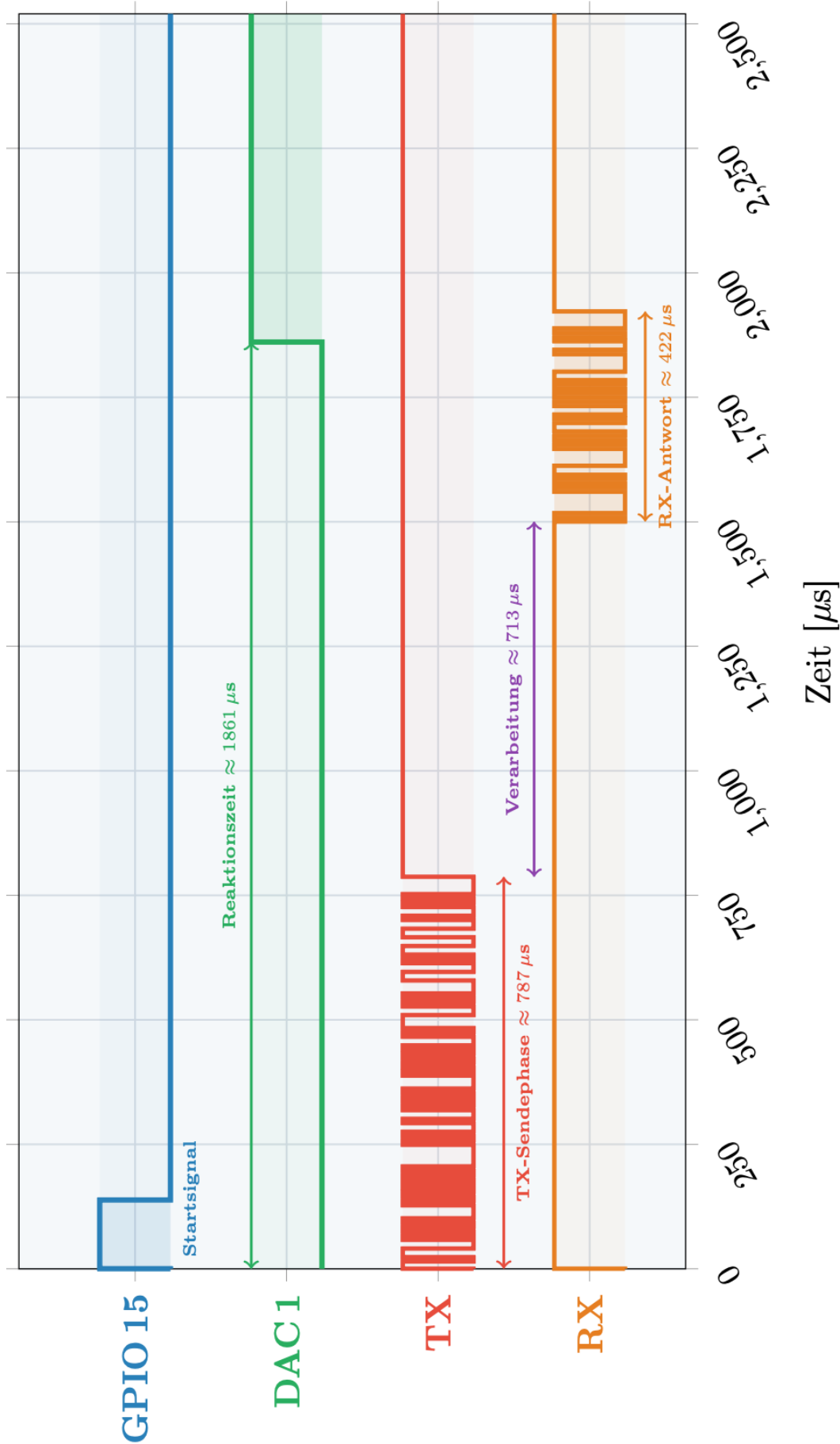


Abbildung 22: Reaktionsgeschwindigkeit vom DAC

A5 Implementierungsmethode B (DDS)

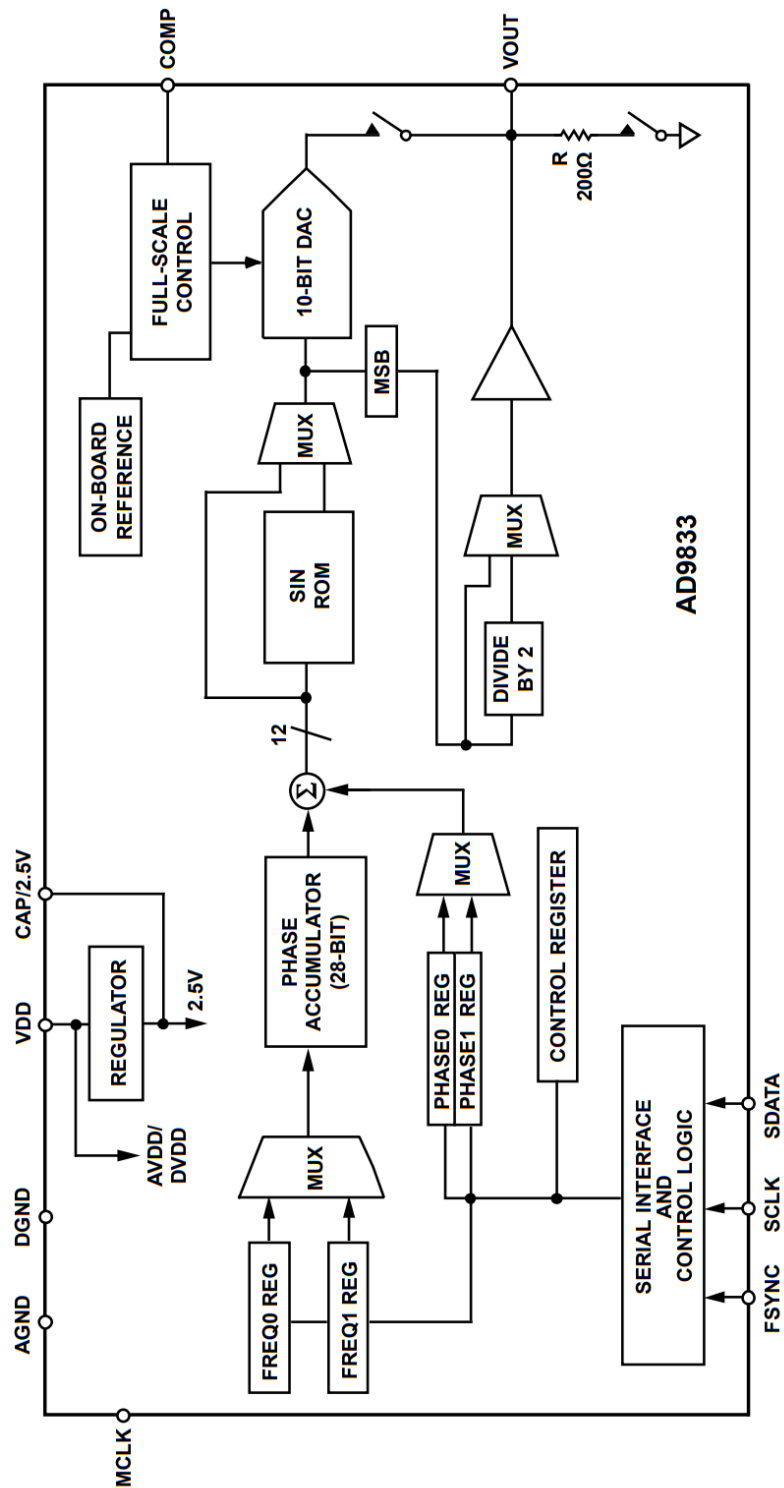


Abbildung 23: Funktionales Blockdiagramm des AD9833 von Analog Devices [26, S. 1]

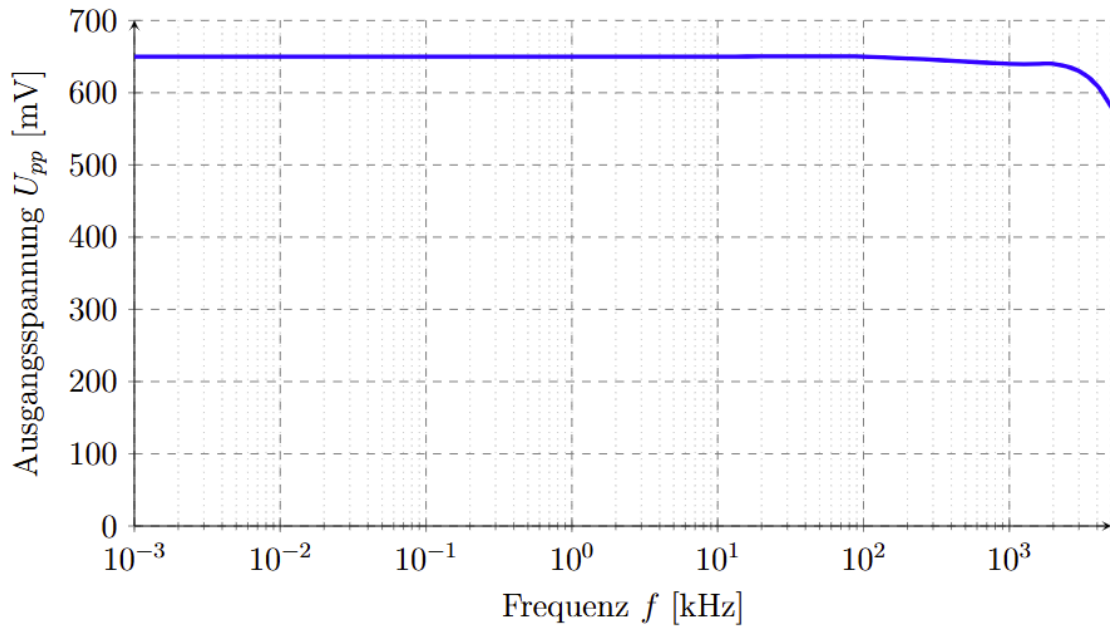
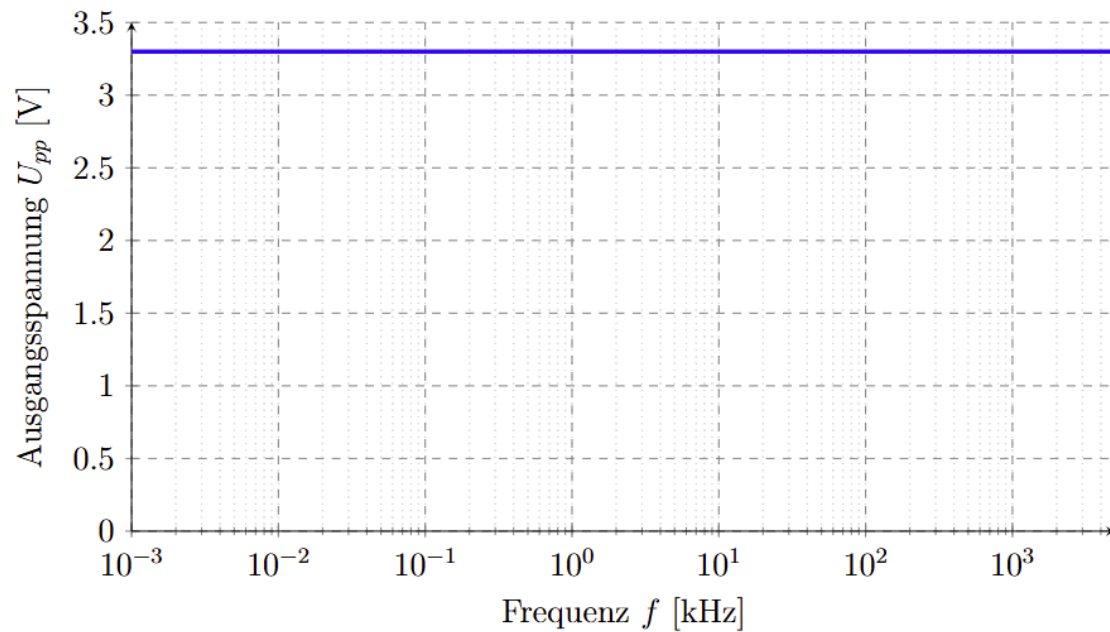
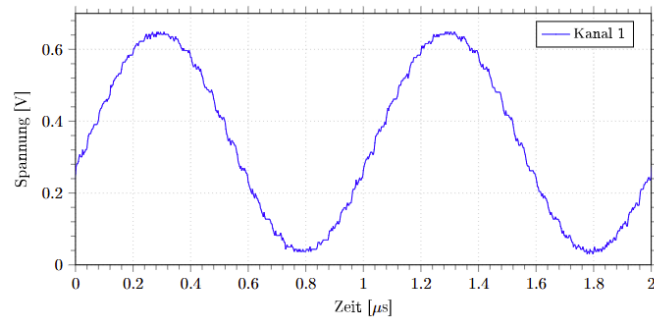
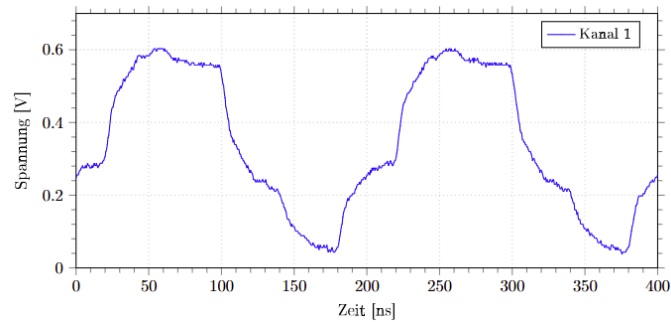
(a) Ausgangsspannung (U_{pp}) (Sinus) in Abhängigkeit der Frequenz(b) Ausgangsspannung (U_{pp}) (Rechteck) in Abhängigkeit der Frequenz

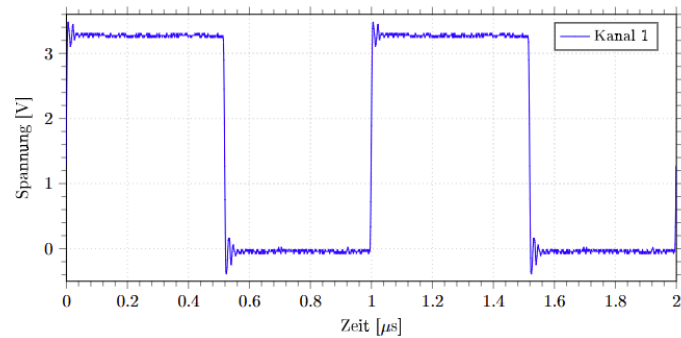
Abbildung 24: Ausgangsspannung des AD9833-Moduls bei sinus- und rechteckförmiger Ausgangsspannung und einer Versorgungsspannung von 3,3 V in Abhängigkeit der Ausgangsfrequenz



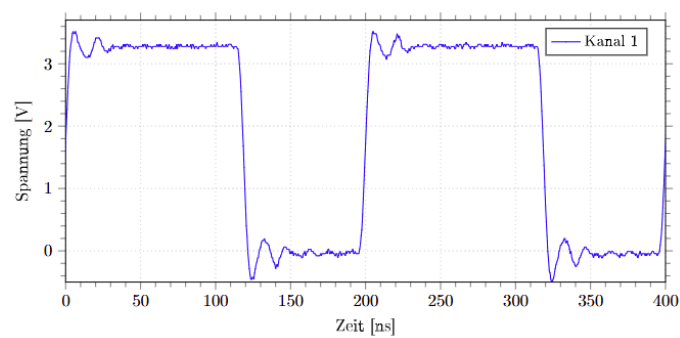
(a) Ausgangssignal (Sinus) bei einer Frequenz von 1 MHz



(b) Ausgangssignal (Sinus) bei einer Frequenz von 5 MHz



(c) Ausgangssignal (Rechteck) bei einer Frequenz von 1 MHz



(d) Ausgangssignal (Rechteck) bei einer Frequenz von 5 MHz

Abbildung 25: Ausgangssignale des AD9833-Moduls in Abhängigkeit der Ausgangsfrequenz und des Ausgabemodus

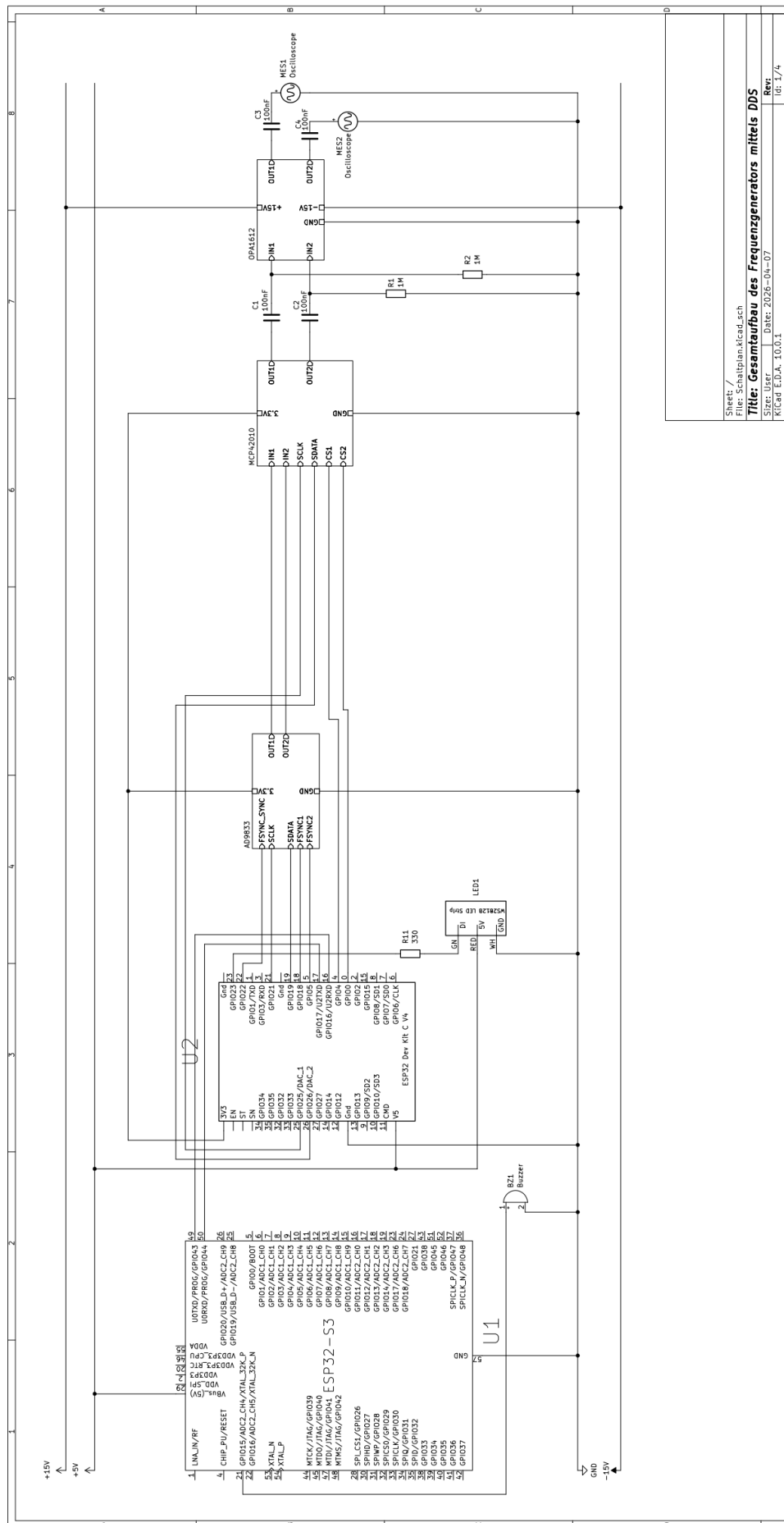


Abbildung 26: Gesamtschaltplan des Funktionsgenerators unter der Verwendung der DDS

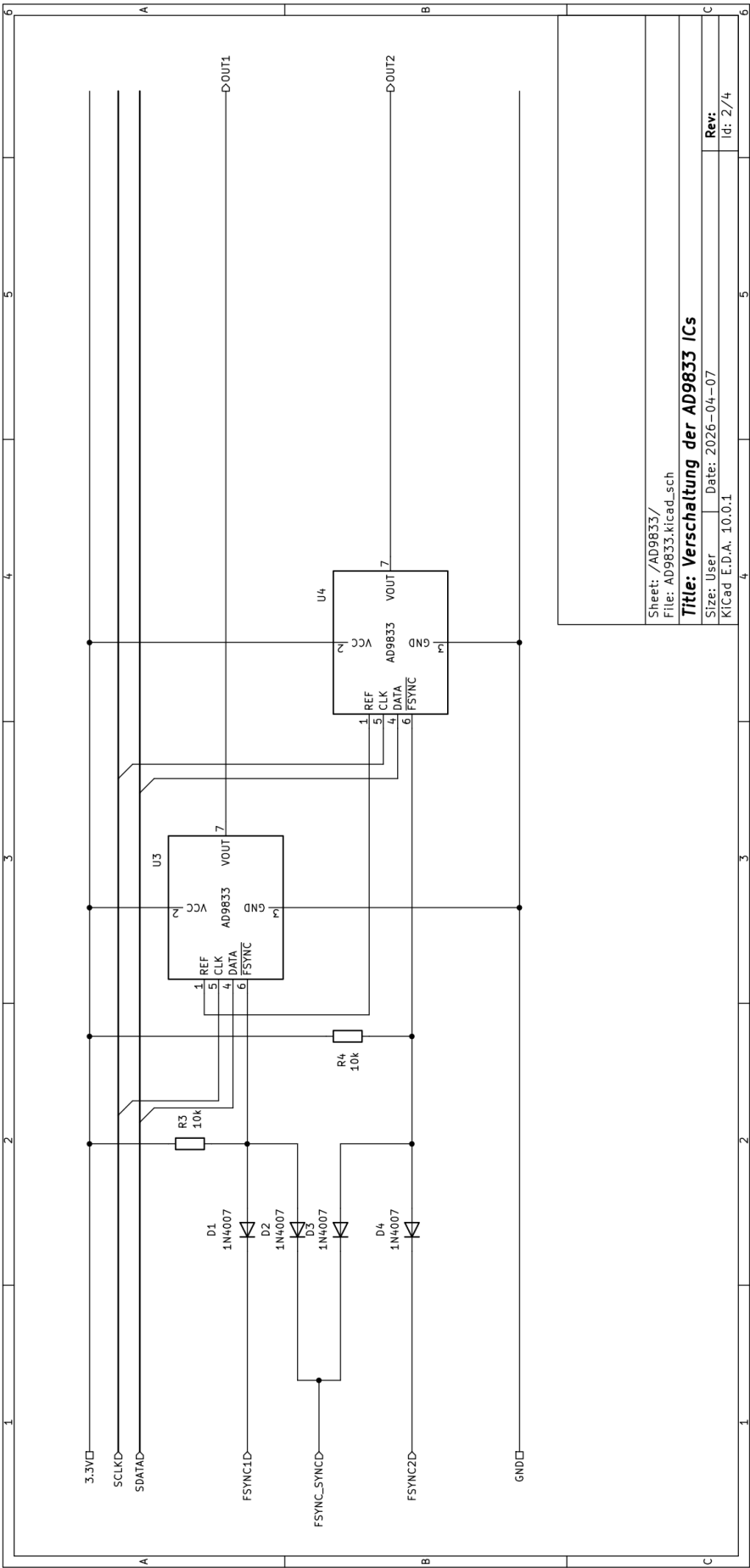


Abbildung 27: Verschaltung der zwei AD9833-DDS-ICs

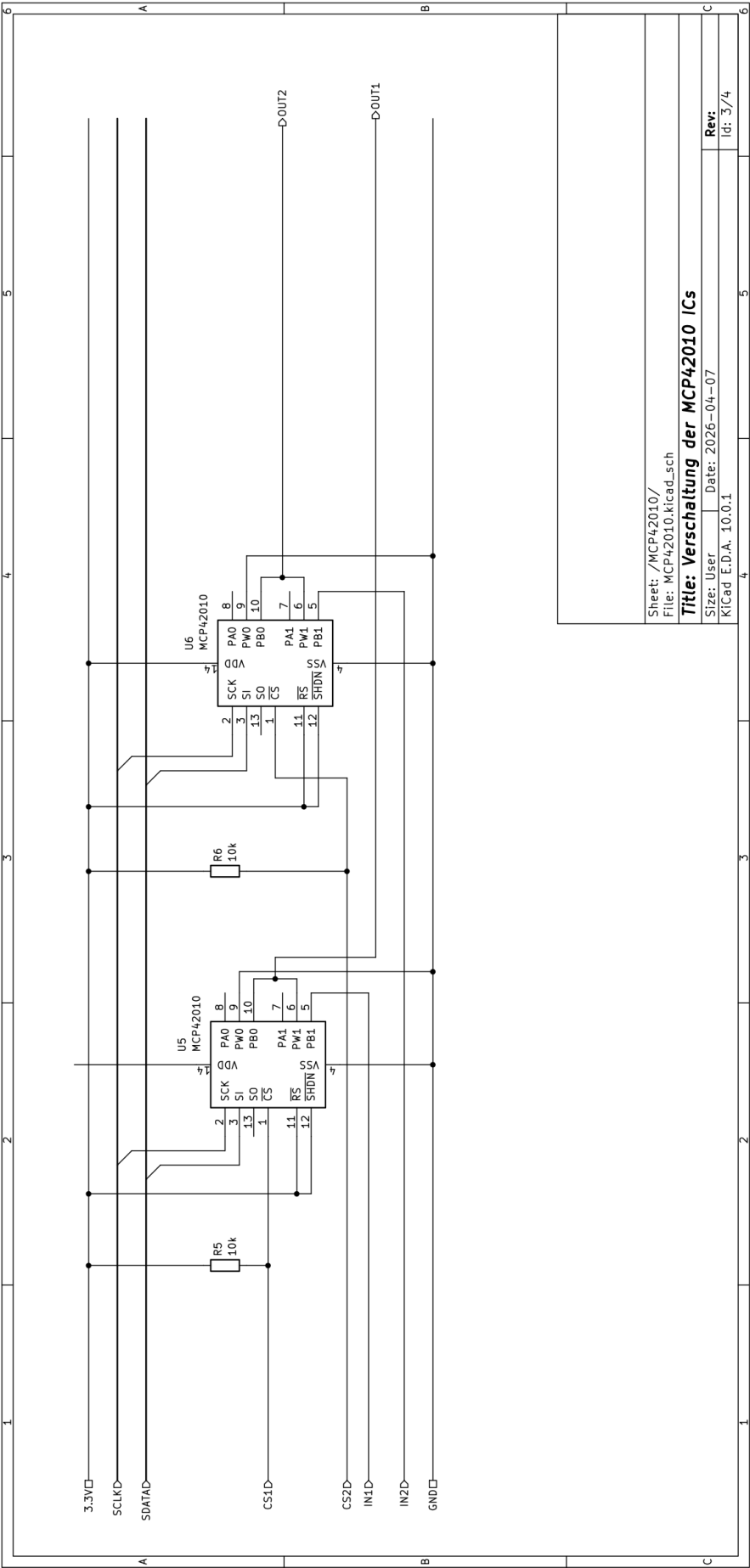


Abbildung 28: Verschaltung der zwei digitalen Potentiometer des Typs MCP42010

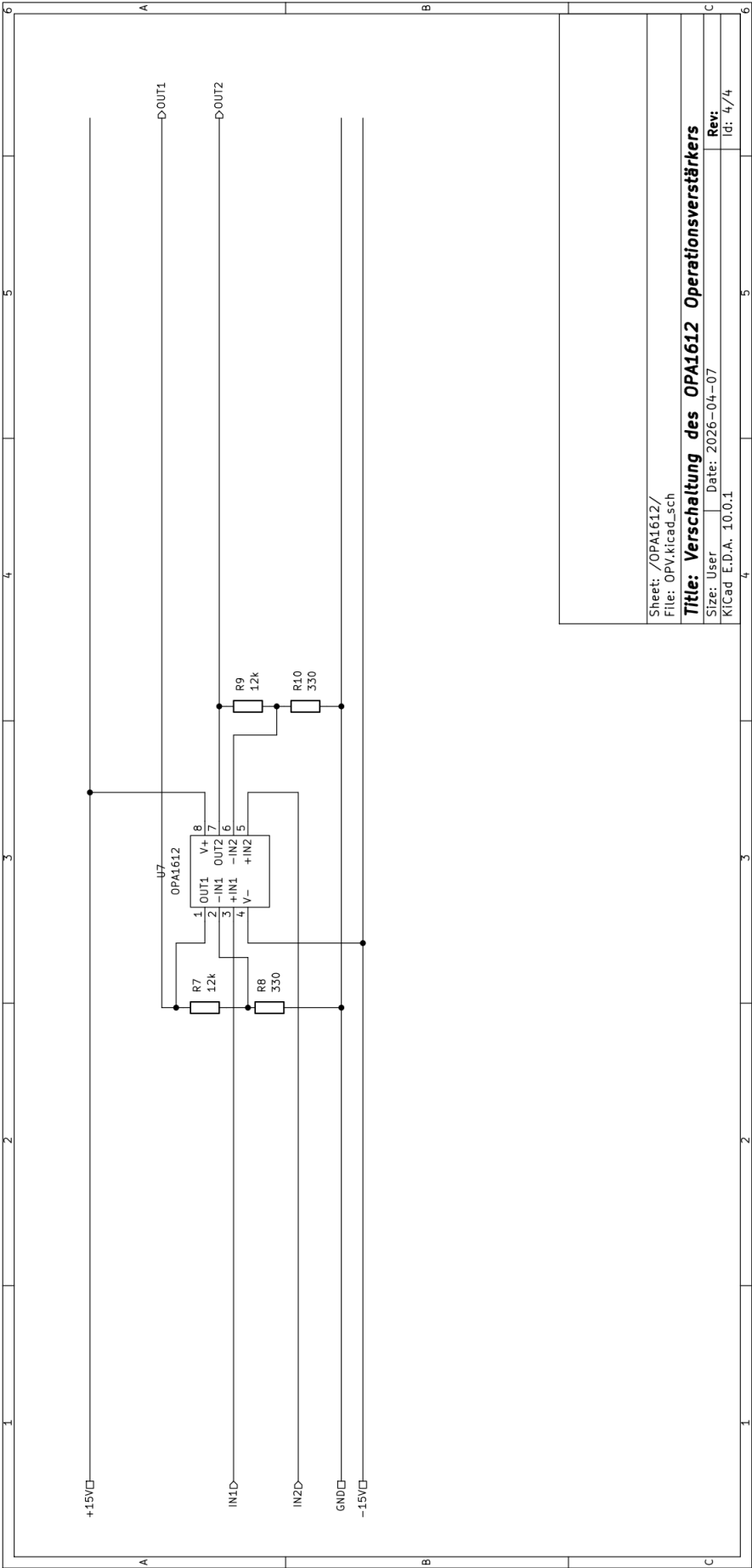
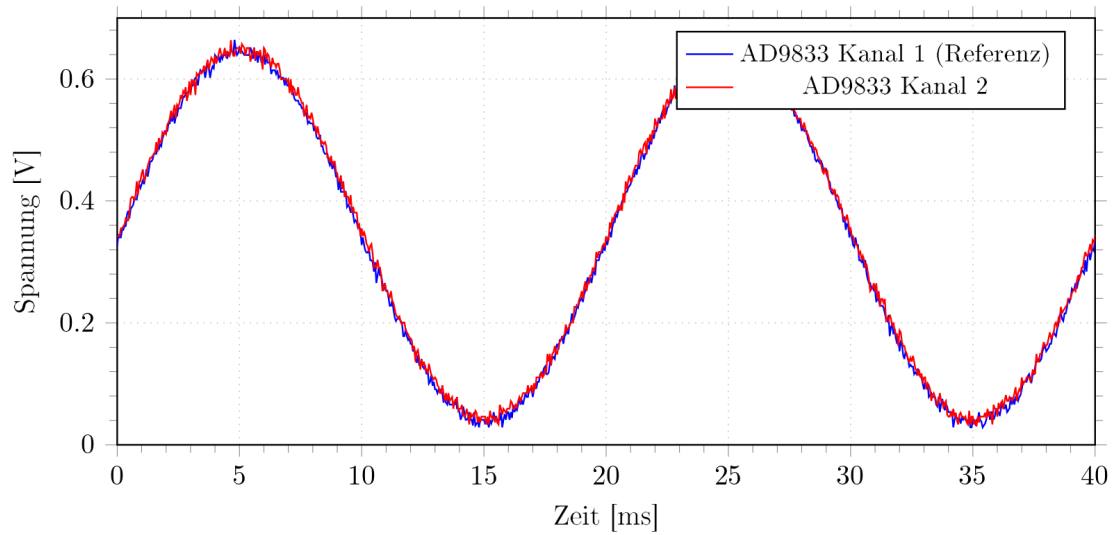
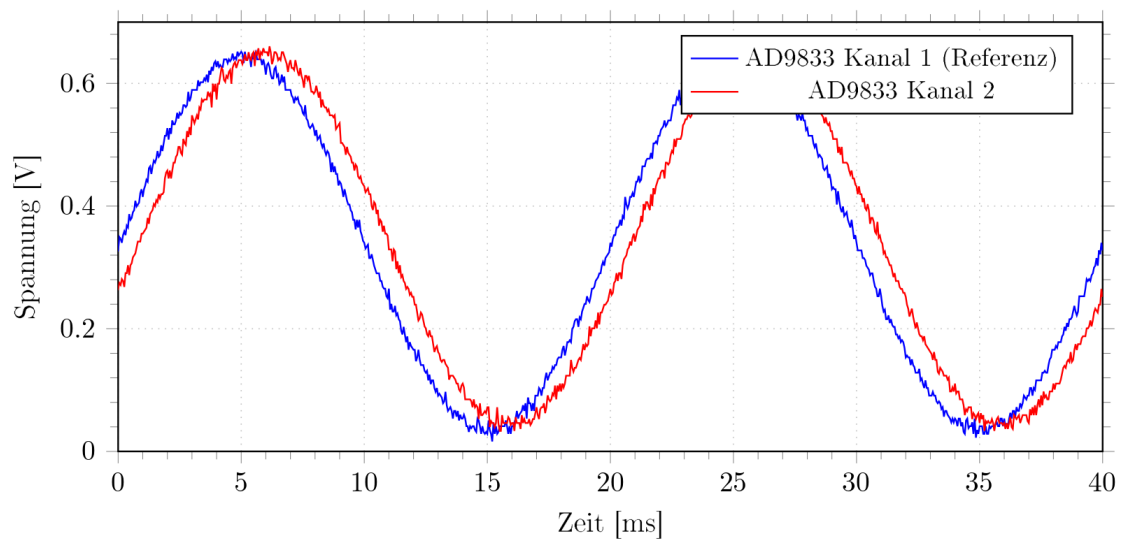


Abbildung 29: Verschaltung des Operationsverstärkers OPA1612

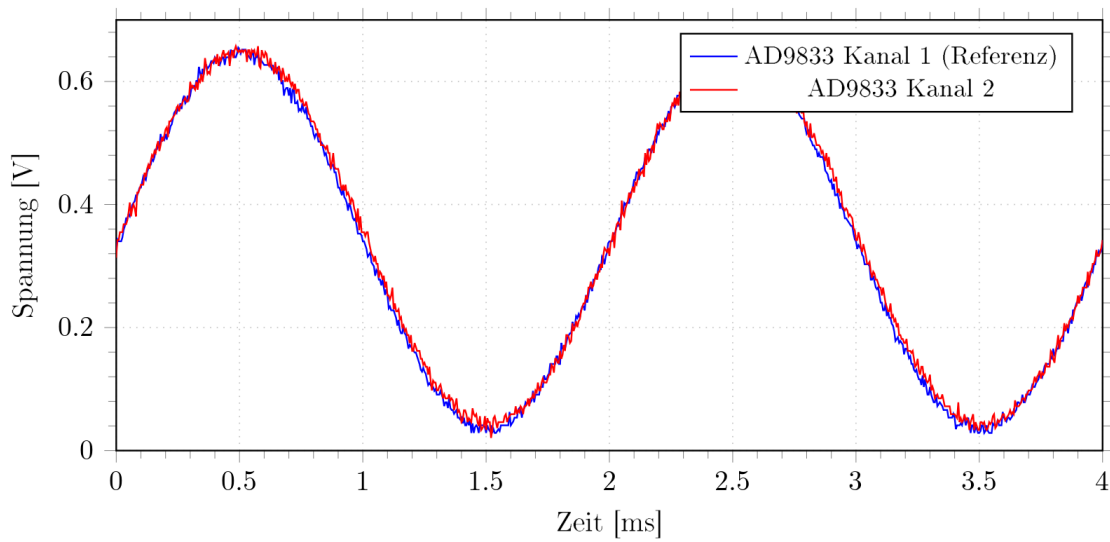


(a) Phasenverschiebung bei einer Frequenz von 50 Hz zum Zeitpunkt $t = 0$ s

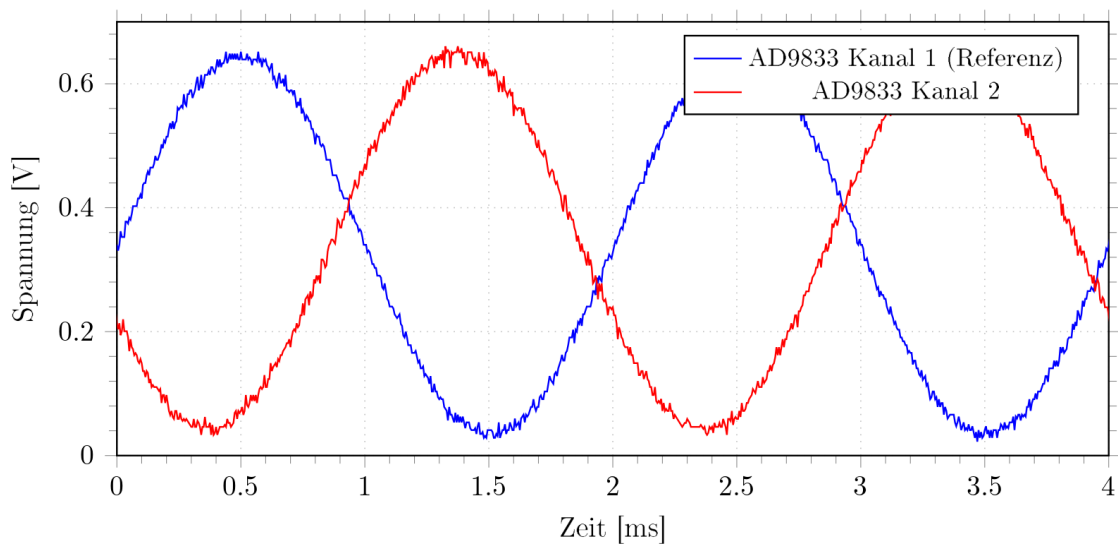


(b) Phasenverschiebung bei einer Frequenz von 50 Hz zum Zeitpunkt $t = 5$ Minuten

Abbildung 30: Phasenverschiebung der beiden AD9833-Module bei Verwendung unterschiedlicher Taktsignale und einer Ausgangsfrequenz von 50 Hz



(a) Phasenverschiebung bei einer Frequenz von 500 Hz zum Zeitpunkt $t = 0$ s



(b) Phasenverschiebung bei einer Frequenz von 500 Hz zum Zeitpunkt $t = 5$ Minuten

Abbildung 31: Phasenverschiebung der beiden AD9833-Module bei Verwendung unterschiedlicher Taktsignale und einer Ausgangsfrequenz von 500 Hz

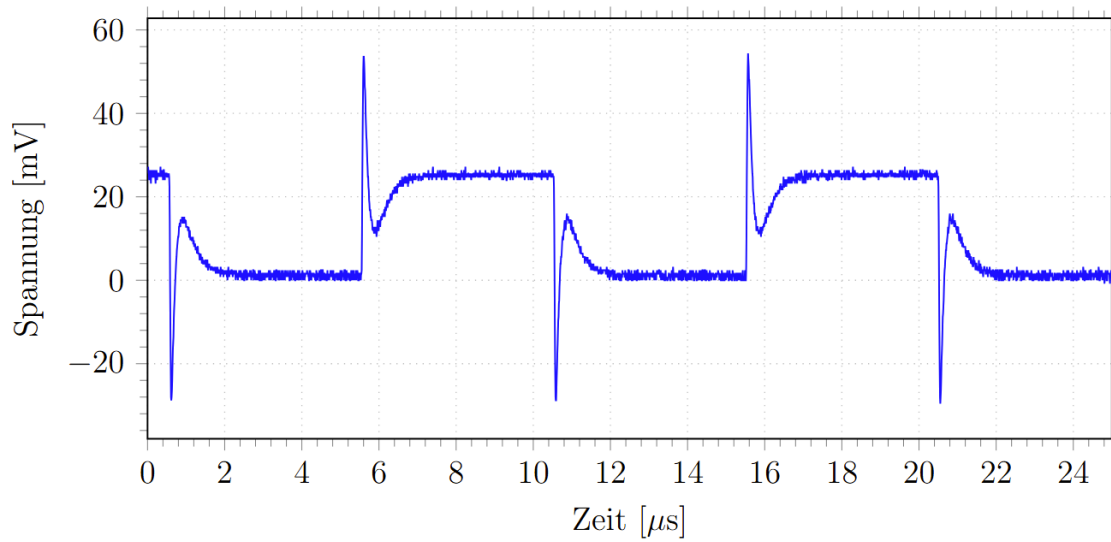


Abbildung 32: Ausgangssignal des Spannungsteilers unter Verwendung des DS1803-100-Potentiometers bei einer Frequenz von 100 kHz und einer Spannungsvorgabe von 1 V

Listing 4: Inhalt der Funktionen setup() und loop() (ESP32DevMod.ino, Zeilen 15-39)

```
15 static SignalGroup gruppen[MAX_GROUPS];
16 static uint8_t    anzahl_gruppen = 0;
17
18 void setup() {
19     Serial.begin(115200);
20     delay(500);
21     ws2812b_init();
22     Serial.println("ESP32 develop gestartet");
23
24     nvs_init();
25     uart_init();
26     AD9833_Init();
27     MCP42010_Init();
28
29     // Ausgabe der vorhandenen Gruppen im Serial Monitor
30     anzahl_gruppen = nvs_load_all(gruppen);
31     if (anzahl_gruppen > 0) {
32         Serial.printf("%u Gruppen aus NVS geladen, warte auf Startsignal.\n",
33             anzahl_gruppen);
34     }
35
36 void loop() {
37     // Warten auf Übertragung neuer Gruppen
38     uart_loop();
39 }
```

Listing 5: Inhalt der Funktion UpdateRegister() (AD9833.cpp, Zeilen 35-67)

```
35  /**
36   * Funktion zum Senden eines 16-Bit Datenworts an den AD9833.
37   * Die Daten werden MSB-first übertragen, beginnend mit einem aktiven FSYNC (
      LOW) und endend mit einem inaktiven FSYNC (HIGH).
38   */
39  void UpdateRegister(uint16_t data, int chip) {
40      // Abfrage, welcher FSYNC-Pin für den jeweiligen Chip verwendet werden soll
41      uint8_t fsPin = getFSPin(chip);
42
43      // Startbedingung SCLK=High, FSYNC=LOW
44      digitalWrite(PIN_SCLK, HIGH);
45      delayMicroseconds(T_DELAY_US);
46
47      digitalWrite(fsPin, LOW); // FSYNC active LOW. Übertragung starten
48      delayMicroseconds(T_DELAY_US);
49
50      // Daten MSB-first übertragen
51      for (int bit = 15; bit >= 0; --bit) {
52          // set data
53          if ((data >> bit) & 0x1) digitalWrite(PIN_SD, HIGH);
54          else digitalWrite(PIN_SD, LOW);
55
56
57          delayMicroseconds(T_DELAY_US);
58          digitalWrite(PIN_SCLK, LOW); // fallende Flanke
59          delayMicroseconds(T_DELAY_US);
60          digitalWrite(PIN_SCLK, HIGH); // steigende Flanke
61          delayMicroseconds(T_DELAY_US);
62      }
63
64      // Übertragung beenden (FSYNC=HIGH)
65      digitalWrite(fsPin, HIGH);
66      delayMicroseconds(T_DELAY_US);
67  }
```

Listing 6: Inhalt der Funktionen UpdateFreq() und UpdatePhase() (AD9833.cpp, Zeilen 69-129)

```

69  /**
70   * Funktion zum Aktualisieren der Frequenz des AD9833.
71   * Die Funktion berechnet das 28-bit FREQWORD basierend auf der gewünschten
       Frequenz und der Masterclock, teilt es in LSB und MSB auf und sendet die
       entsprechenden Register-Updates.
72   */
73  void UpdateFreq(float freqHz, int chip) {
74      // Schutzbereich
75      if (freqHz < 0.0f) freqHz = 0.0f;
76      if (freqHz > float(MAX_FREQ)) freqHz = float(MAX_FREQ); // max. 12.5 MHz
77
78      // Wenn Rechteckspannung ausgegeben werden soll, muss die Frequenz verdoppelt
       werden
79      if (mode[chip] == 1) freqHz = freqHz * 2;
80
81      // Berechnung 28-bit FREQWORD: round((freq * 2^28) / MCLK)
82      double freqWordDb1 = ((double)freqHz * (double)(1ULL << 28)) / (double)
       MCLK_HZ;
83
84      uint32_t freqWord = (uint32_t)llround(freqWordDb1); // Runden
85
86      // teile in 14 LSB / 14 MSB
87      uint16_t LSB14 = freqWord & 0x3FFF; // Bits 13..0
88      uint16_t MSB14 = (freqWord >> 14) & 0x3FFF; // Bits 27..14
89
90      // Adresse für FREQ0 (D15D14 = 01) -> 0x4000 | 14bitdata
91      uint16_t wordLSB = 0x4000 | LSB14;
92      uint16_t wordMSB = 0x4000 | MSB14;
93
94      // Reset=1
95      UpdateRegister(activeControl[chip] |= RESET, chip);
96
97      // LSB dann MSB übertragen
98      UpdateRegister(wordLSB, chip);
99      UpdateRegister(wordMSB, chip);
100
101      // Sinus-Wellenform (B28=0, RESET=0, Mode=0)
102      if (mode[chip] == 0) {
103          UpdateRegister((activeControl[chip] &= ~RESET) & ~SQU, chip); // Sinus-
       Wellenform
104      }
105      // Rechteck-Wellenform (B28=0, RESET=0, MODE=1)
106      else if (mode[chip] == 1) {
107          UpdateRegister((activeControl[chip] &= ~RESET) |= SQU, chip); // Rechteck-
       Wellenform
108      }
109  }
110
111  /**
112   * Funktion zum Aktualisieren der Phase des AD9833.

```

```
113  * Die Funktion berechnet das 12-bit PHASEWORD basierend auf der gewünschten
      Phase in Grad, begrenzt den Wertebereich auf 0-360 Grad und sendet das
      entsprechende Register-Update.
114  */
115  void UpdatePhase(float phaseDeg, int chip) {
116      // Wertebereich begrenzen
117      if (phaseDeg < 0.0f) phaseDeg = 0.0f;
118      if (phaseDeg > 360.0f) phaseDeg = 360.0f;
119
120      if (mode[chip] == 1) phaseDeg = phaseDeg * 2;
121
122      // 12-bit Phase Word berechnen
123      uint16_t phaseWord = (uint16_t)((phaseDeg * 4096.0f / 360.0f) + 0.5f);
124      phaseWord &= 0xFFFF;
125
126      // PHASE0 schreiben
127      uint16_t reg = 0xC000 | phaseWord;
128      UpdateRegister(reg, chip);
129  }
```

Listing 7: Inhalt der Funktionen WriteFreqWord() und SyncChips() (AD9833.cpp, Zeilen 145-209)

```

145  /**
146   * Funktion zum Schreiben eines Frequenzworts (28-bit) für die Synchronisation
      beider Chips.
147   */
148  void WriteFreqWord(float freqHz, int chip) {
149      // Schutzbereich identisch zur oberen Funktion UpdateFreq
150      if (freqHz < 0.0f) freqHz = 0.0f;
151      if (freqHz > float(MAX_FREQ)) freqHz = float(MAX_FREQ);
152      if (mode[chip] == 1) freqHz = freqHz * 2;
153
154      // Berechnung 28-bit FREQWORD: round((freq * 2^28) / MCLK)
155      double freqWordDb1 = ((double)freqHz * (double)(1ULL << 28)) / (double)
          MCLK_HZ;
156      uint32_t freqWord = (uint32_t)llround(freqWordDb1);
157      uint16_t wordLSB = 0x4000 | (freqWord & 0x3FFF);
158      uint16_t wordMSB = 0x4000 | ((freqWord >> 14) & 0x3FFF);
159
160      // Updaten der beiden Register
161      UpdateRegister(wordLSB, chip);
162      UpdateRegister(wordMSB, chip);
163  }
164
165  /**
166   * Funktion zum Synchronisieren beider AD9833-Chips.
167   * Diese Funktion setzt beide Chips gleichzeitig zurück und überträgt die
      gleichen Frequenzdaten, um sicherzustellen, dass beide Chips synchron
      laufen.
168   */
169  void SyncChips() {
170      // Beide einzeln mit RESET=1 und Frequenz übertragen
171      UpdateRegister(activeControl[0] |= RESET, 0);
172      UpdateRegister(activeControl[1] |= RESET, 1);
173      // Frequenzwörter übertragen
174      WriteFreqWord(f[0], 0);
175      WriteFreqWord(f[1], 1);
176
177      // activeControl für Modus anpassen
178      for (int chip = 0; chip < 2; chip++) {
179          activeControl[chip] &= ~RESET;
180          if (mode[chip] == 1) activeControl[chip] |= SQU;
181          else activeControl[chip] &= ~SQU;
182      }
183
184
185      // Über den gemeinsamen Sync-Pin beide Chips gleichzeitig mit den gleichen
      Daten beschreiben, damit sie beide synchron laufen
186      if (activeControl[0] == activeControl[1]) {
187          digitalWrite(PIN_SCLK, HIGH);
188          delayMicroseconds(1);
189          digitalWrite(PIN_SYNC, LOW); // Übertragung starten

```

```
190     delayMicroseconds(1);
191
192     // 16-Bit Control-Word seriell übertragen (MSB zuerst)
193     for (int bit = 15; bit >= 0; --bit) {
194         digitalWrite(PIN_SD, (activeControl[0] >> bit) & 0x1 ? HIGH : LOW);
195         delayMicroseconds(1);
196         digitalWrite(PIN_SCLK, LOW);
197         delayMicroseconds(1);
198         digitalWrite(PIN_SCLK, HIGH);
199         delayMicroseconds(1);
200     }
201     // Übertragung beenden
202     digitalWrite(PIN_SYNC, HIGH);
203 }
204 // Unterschiedliche Modi, dann sequenziell so schnell wie möglich freigeben
205 else {
206     UpdateRegister(activeControl[0], 0);
207     UpdateRegister(activeControl[1], 1);
208 }
209 }
```

Listing 8: Inhalt der Funktionen `spi_write_byte()` und `mcp_send()` (MCP42010.cpp, Zeilen 35-66)

```
35  /**
36   * Funktion zum senden eines vollständigen bytes über das SPI-Interface
37   */
38  void spi_write_byte(uint8_t data){
39      for (int bit = 7; bit >= 0; --bit){
40          digitalWrite(PIN_SI, (data >> bit) & 0x1 ? HIGH : LOW);
41          delayMicroseconds(1);
42          digitalWrite(PIN_SCK, LOW);
43          delayMicroseconds(1);
44          digitalWrite(PIN_SCK, HIGH);
45          delayMicroseconds(1);
46      }
47  }
48
49  /**
50   * Funktion zum senden eines vollständigen Kommandos an den MCP42010 (ausgewä
      hlter Chip, Command, Wert)
51   */
52  bool mcp_send(int chip, uint8_t command, uint8_t value){
53      // Abfragen, welcher CS-Pin verwendet werden muss
54      int cs_pin = get_cs_pin(chip);
55
56      digitalWrite(PIN_SCK, HIGH);
57      delayMicroseconds(1);
58      digitalWrite(cs_pin, LOW); // Übertragung starten
59      delayMicroseconds(1);
60      spi_write_byte(command);
61      spi_write_byte(value);
62      digitalWrite(cs_pin, HIGH); // Übertragung beenden
63      delayMicroseconds(1);
64
65      return true;
66  }
```

Listing 9: Inhalt der Funktionen setPot0(), setPot1() und setBoth() (MCP42010.cpp, Zeilen 82-136)

```
82  /**
83   * Funktion zum Setzen des Potentiometerwerts für Potentiometer 0 eines
      bestimmten Chips.
84   */
85  void setPot0(int value, int chip) {
86      value = constrain(value, 0, 255);    // Sicherstellen, dass der Wert im gü
      ltigen Bereich liegt
87      positionPot0[chip] = value;          // aktualisieren der position von Pot0
88
89      bool ok = mcp_send(chip, CMD_POT0, (uint8_t)value); // Neuen Wert an die
      Adresse des ICs für das Potentiometer 0 senden
90
91      // Wenn die Übertragung erfolgreich war, den Widerstandswert berechnen und
      ausgeben, andernfalls eine Fehlermeldung ausgeben
92      if (ok) {
93          float resistance = getRes(0, chip);
94          Serial.printf("Pot0 von Chip %d gesetzt: %d/255  (~%.2f kΩ von %.2f kΩ)\n"
      , chip, value, resistance, P0_255[chip]);
95      } else {
96          Serial.println("Fehler!");
97      }
98  }
99
100 /**
101  * Funktion zum Setzen des Potentiometerwerts für Potentiometer 1 eines
      bestimmten Chips.
102  */
103  void setPot1(int value, int chip) {
104      value = constrain(value, 0, 255); // Sicherstellen, dass der Wert im gültigen
      Bereich liegt
105      positionPot1[chip] = value;        // aktualisieren der position von Pot1
106
107      bool ok = mcp_send(chip, CMD_POT1, (uint8_t)value); // Neuen Wert an die
      Adresse des ICs für das Potentiometer 1 senden
108
109      // Wenn die Übertragung erfolgreich war, den Widerstandswert berechnen und
      ausgeben, andernfalls eine Fehlermeldung ausgeben
110      if (ok) {
111          float resistance = getRes(1, chip);
112          Serial.printf("Pot1 von Chip %d gesetzt: %d/255  (~%.2f kΩ von %.2f kΩ)\n"
      , chip, value, resistance, P1_255[chip]);
113      } else {
114          Serial.println("Fehler!");
115      }
116  }
117
118 /**
119  * Funktion zum Setzen beider Potentiometer gleichzeitig auf denselben Wert für
      einen bestimmten Chip.
120  */
```

```
121 void setBoth(int value, int chip) {
122     value = constrain(value, 0, 255); // Sicherstellen, dass der Wert im gültigen
        Bereich liegt
123     positionPot0[chip] = value;      // aktualisieren der position von Pot0
124     positionPot1[chip] = value;      // aktualisieren der position von Pot1
125
126     bool ok = mcp_send(chip, CMD_BOTH, (uint8_t)value); // Neuen Wert an die
        Adresse des ICs für beide Potentiometer gleichzeitig senden
127
128     // Wenn die Übertragung erfolgreich war, den Widerstandswert berechnen und
        ausgeben, andernfalls eine Fehlermeldung ausgeben
129     if (ok) {
130         float resistanceP0 = getRes(0, chip);
131         float resistanceP1 = getRes(1, chip);
132         Serial.printf("Pot0 + Pot1 von Chip %d gesetzt: %d/255 (P0: ~%.2f kΩ von
            %.2f kΩ; P1: ~%.2f kΩ von %.2f kΩ)\n", chip, value, resistanceP0,
            P0_255[chip], resistanceP1, P1_255[chip]);
133     } else {
134         Serial.println("Fehler!");
135     }
136 }
```

Listing 10: Inhalt der Funktionen `getVoltAmplification()` und `getInputVoltage()` sowie die Lookup-Tabellen für die Berechnung der Eingangsspannung (`resistance.cpp`, Zeilen 15-62)

```

15 //Lookup-Tabellen für die Berechnung der benötigten Widerstände basierend auf
    der Frequenz des AD9833 Chips im Sinusmodus (Ausgangsspannung des AD9833
    ist im Sinusmodus frequenzabhängig).
16 static const float freqTable[] = {1.0f,          10.0f,          100.0f,
    1000.0f,          10000.0f,          100000.0f,          500000.0f,
    700000.0f,          1000000.0f,          1500000.0f,          1800000.0f,          2000000.0f
    };
17 static const float voltTable[] = {0.287,          0.298f,          0.298f,
    0.298f,          0.298,          0.298f,          0.298f,          0.298f,
    0.298f,          0.298f,          0.273f,          0.273f};
18 static const int tableSize = 12;
19
20 // Konstante Ausgangsspannung im Rechteckmodus (weitestgehend unabhängig von
    der Frequenz)
21 static const float voltSqu = 1.65f;
22
23
24 /**
25  * Funktion zum Ausgeben der Verstärkung des OPVs basierend auf der Chip-Nummer
    (0 oder 1).
26  */
27 float getVoltAmplification(int chip){
28     if(chip==0){
29         return 37.2;
30     }
31     if(chip==1){
32         return 37.5;
33     }
34 }
35
36 /**
37  * Funktion zum Berechnen der Eingangsspannung der Potentiometer basierend auf
    der aktuellen Frequenz und dem Modus des AD9833 Chips.
38  * Im Sinusmodus ist die Ausgangsspannung des AD9833 frequenzabhängig, weshalb
    eine Interpolation in der freqTable/voltTable durchgeführt wird. Im
    Rechteckmodus wird eine konstante Spannung angenommen.
39  */
40 float getInputVoltage(float freqHz, int chip){
41
42     // Überprüfen, in welchem Modus der AD9833 betrieben wird. Wird ein
    Rechtecksignal ausgegeben, wird die konstante Spannung zurückgegeben.
43     if (mode[chip]==1){
44         return voltSqu;
45     }
46     // Im Sinusmodus wird die Spannung anhand der Frequenz interpoliert. Für
    Frequenzen außerhalb der Tabelle werden die Grenzwerte zurückgegeben.
47     else {
48         // Grenzfälle: Wenn die Frequenz außerhalb der Tabelle liegt, den
    jeweils nächsten Wert zurückgeben
49         if (freqHz <= freqTable[0]) return voltTable[0];

```

```
50     if (freqHz >= freqTable[tableSize-1]) return voltTable[tableSize-1];
51
52     // Interpolation: Den Bereich in der Tabelle finden, in dem die
53     // Frequenz liegt, und dann logarithmisch interpolieren
54     for (int i = 0; i < tableSize - 1; i++) {
55         if (freqHz >= freqTable[i] && freqHz < freqTable[i + 1]) {
56             // Logarithmische Position zwischen den zwei Stützpunkten
57             // berechnen
58             float t = log(freqHz / freqTable[i]) / log(freqTable[i+1] /
59                 freqTable[i]);
60             return voltTable[i] + t * (voltTable[i+1] - voltTable[i]);
61         }
62     }
```

Listing 11: Inhalt der Funktion calculateResistance() (resistance.cpp, Zeilen 64-126)

```

64  /**
65   * Funktion zum Berechnen der Widerstände für die Potentiometer basierend auf
        der Zielspannung, der aktuellen Frequenz und dem Modus des AD9833 Chips.
66   * Die Funktion berechnet die benötigten Widerstandswerte für P0 und P1, um die
        gewünschte Ausgangsspannung zu erreichen, und setzt die Potentiometer
        entsprechend.
67   */
68  void calculateResistance(float targetVoltage, int chip) {
69      // Abfrage der aktuellen Eingangsspannung der Potentiometer basierend auf
        der Frequenz und dem Modus des AD9833 Chips
70      float inputVoltage = getInputVoltage(f[chip], chip);
71
72      // Abfrage der Verstärkung des OPVs basierend auf der Chip-Nummer
73      float voltAmplification = getvoltAmplification(chip);
74
75      // Berechnung der maximalen und minimalen Ausgangsspannung, die mit den
        Potentiometern erreicht werden kann, um die Zielspannung auf
        Plausibilität zu prüfen
76      float maxVout = (P0_255[chip] / (P0_255[chip] + P1_0[chip])) * inputVoltage
        * voltAmplification;
77      float minVout = (P0_0[chip] / (P0_0[chip] + P1_255[chip])) * inputVoltage
        * voltAmplification;
78
79      // Wenn die Zielspannung außerhalb des Bereichs liegt, der mit den
        Potentiometern erreicht werden kann, eine Fehlermeldung ausgeben und
        die Funktion verlassen
80      if (targetVoltage > maxVout) {
81          Serial.printf("Fehler: Maximum ist %.3f V\n", maxVout);
82          return;
83      }
84      if (targetVoltage < minVout) {
85          Serial.printf("Fehler: Minimum ist %.3f V\n", minVout);
86          return;
87      }
88
89      // Potentiometer 0 auf 255 setzen (maximaler Widerstand), um die benötigte
        Position von Potentiometer 1 zu berechnen.
90      int newPot0 = 255;
91
92      // Division der Zielspannung durch die Verstärkung des OPVs, um die benö
        tigte Eingangsspannung zu berechnen.
93      float realTarget = targetVoltage / voltAmplification;
94
95      float ratio = (inputVoltage - realTarget) / realTarget; // Verhältnis P1/P0,
        das benötigt wird, um die Zielspannung zu erreichen
96
97      float targetP1kOhm = ratio * P0_255[chip]; // Berechnung des benötigten
        Widerstands für P1, um die Zielspannung zu erreichen
98
99      // P1 Position aus Widerstandswert berechnen: pos = (R - P1_0) / (P1_255 -
        P1_0) * 255

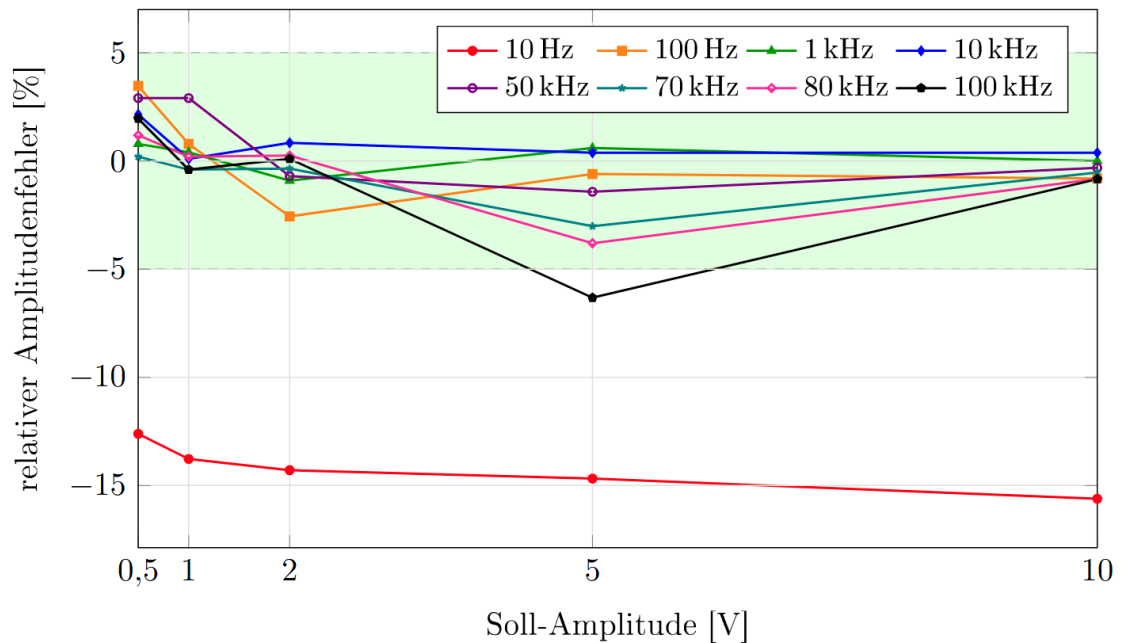
```

```

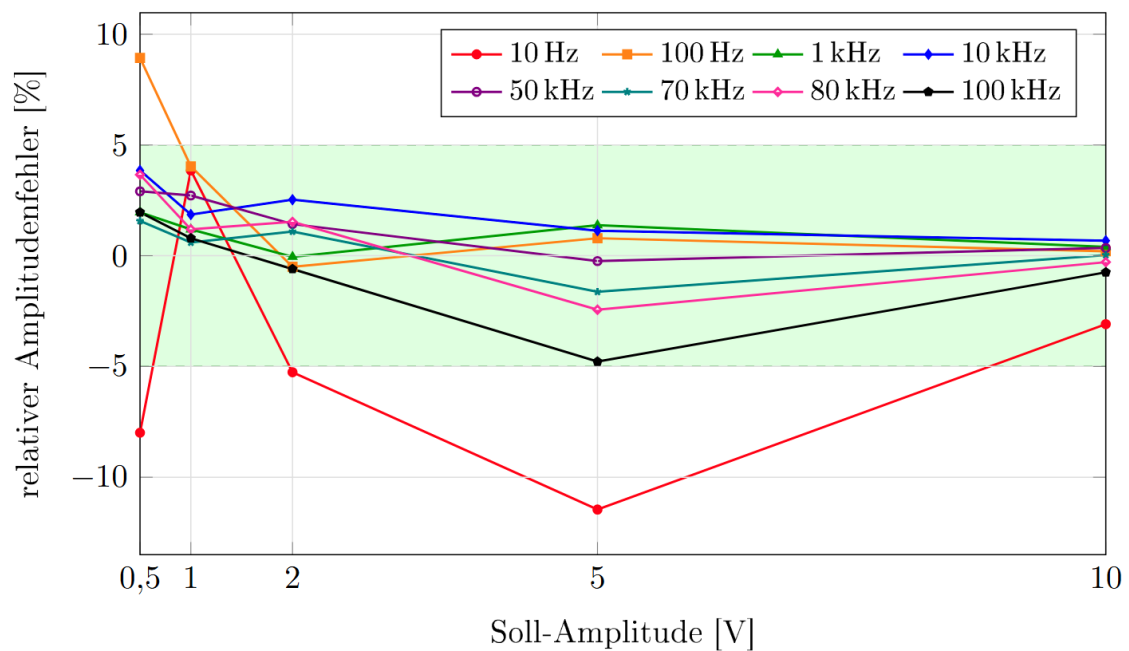
100     int newPot1 = (int)((targetP1kOhm - P1_0[chip]) / (P1_255[chip] - P1_0[chip]
101         ]) * 255.0f + 0.5f); //+0.5f für korrektes Runden zum nächsten Integer
102
103     newPot1 = constrain(newPot1, 0, 255); // Sicherstellen, dass der Wert im gü
104         ltigen Bereich liegt
105
106     // Falls P1 durch constrain begrenzt wurde, P0 nachrechnen
107     if (newPot1 == 255) {
108         float actualP1_kOhm = P1_255[chip];
109         float targetP0kOhm = actualP1_kOhm / ratio;
110         newPot0 = (int)((targetP0kOhm - P0_0[chip]) / (P0_255[chip] - P0_0[chip]
111             ]) * 255.0f + 0.5f);
112         newPot0 = constrain(newPot0, 1, 255); // mind. 1, damit nicht mit 0
113             dividiert wird
114     }
115
116     // Tatsächliche Ausgangsspannung berechnen und ausgeben
117     float actualP0_kOhm = (newPot0 / 255.0f * (P0_255[chip] - P0_0[chip])) +
118         P0_0[chip];
119     float actualP1_kOhm = (newPot1 / 255.0f * (P1_255[chip] - P1_0[chip])) +
120         P1_0[chip];
121     float actualVout = inputVoltage * (actualP0_kOhm / (actualP0_kOhm +
122         actualP1_kOhm)) * voltAmplification;
123
124     Serial.println("-----");
125     Serial.printf("  Zielspannung:      %.3f V\n", targetVoltage);
126     Serial.printf("  Erreichte Spannung: %.3f V\n", actualVout);
127     Serial.printf("  Abweichung:      %.3f V\n", actualVout - targetVoltage);
128     Serial.printf("  Pot0 für IC %d: %3d/255  (~%.3f kΩ)\n", chip, newPot0,
129         actualP0_kOhm);
130     Serial.printf("  Pot1 für IC %d: %3d/255  (~%.3f kΩ)\n", chip, newPot1,
131         actualP1_kOhm);
132     Serial.println("-----");
133
134     setPot0(newPot0, chip);
135     setPot1(newPot1, chip);
136 }

```

A6 Messergebnisse der Implementierungsmethode B (DDS)

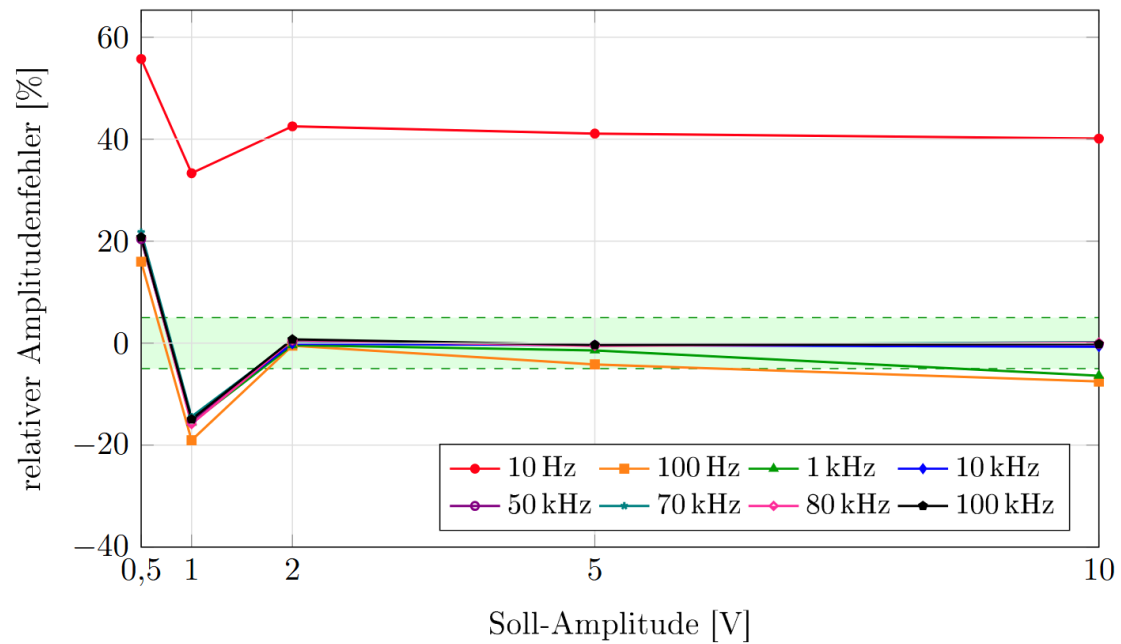


(a) Relativer Amplitudenfehler des DDS-Funktionsgenerators (Sinus) Kanal 1

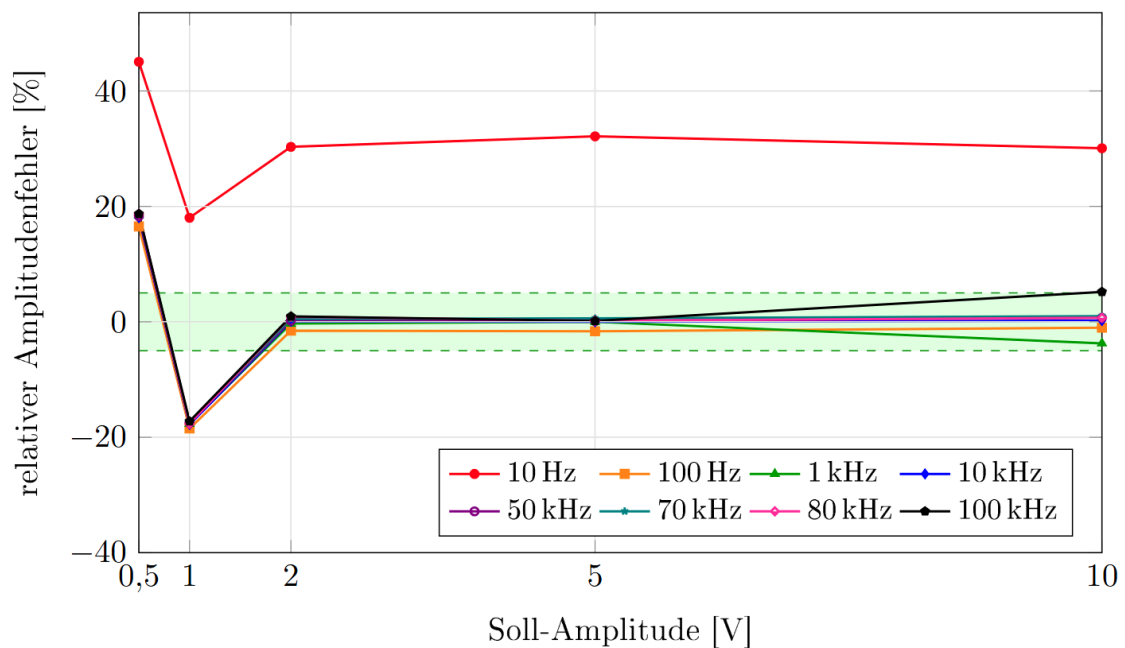


(b) Relativer Amplitudenfehler des DDS-Funktionsgenerators (Sinus) Kanal 2

Abbildung 33: Relativer Amplitudenfehler des DDS-Funktionsgenerators (Sinus) in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen

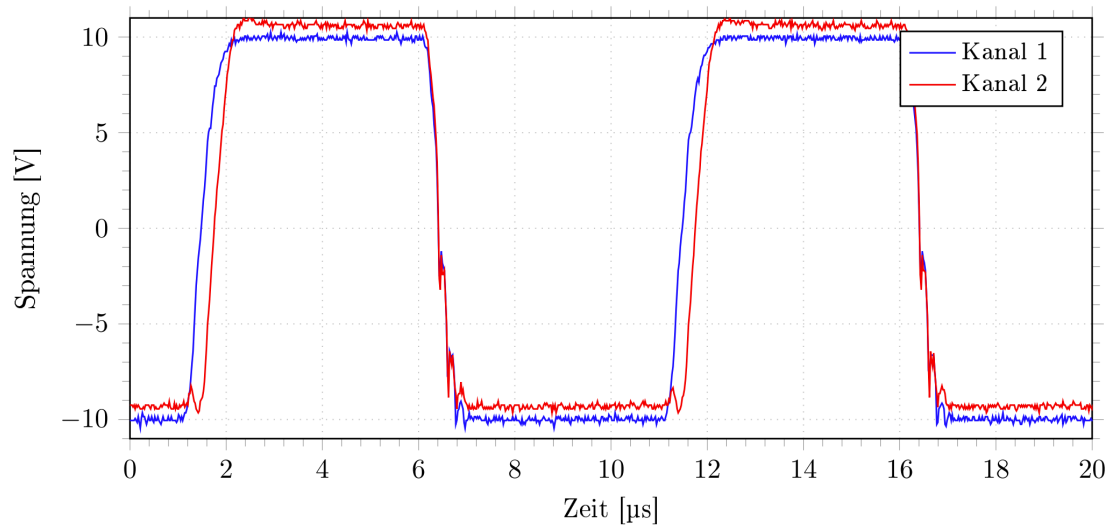


(a) Relativer Amplitudenfehler des DDS-Funktionsgenerators (Rechteck) Kanal 1

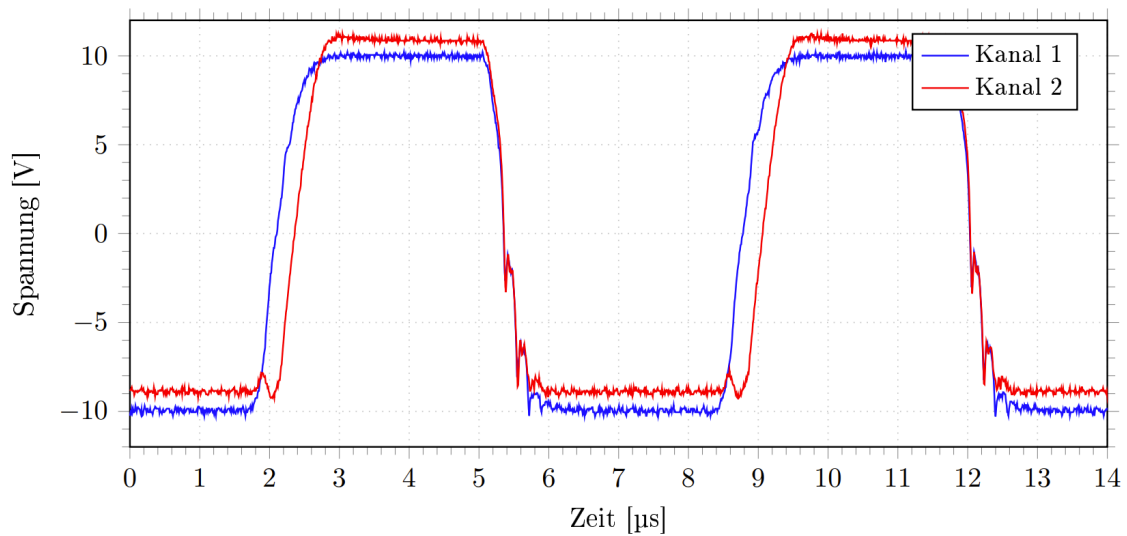


(b) Relativer Amplitudenfehler des DDS-Funktionsgenerators (Rechteck) Kanal 2

Abbildung 34: Relativer Amplitudenfehler des DDS-Funktionsgenerators (Rechteck) in Abhängigkeit der Soll-Amplitude für alle getesteten Frequenzstufen

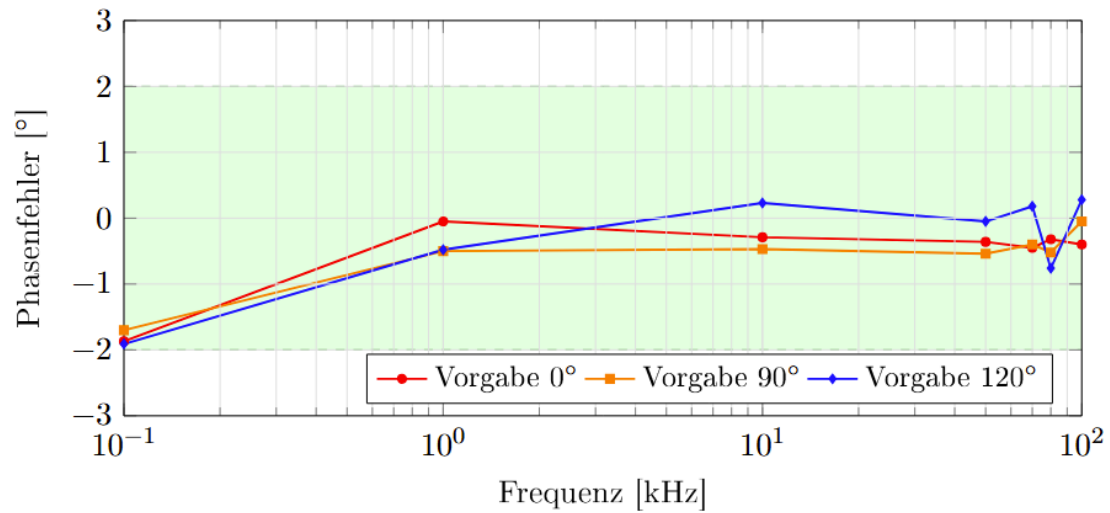


(a) Ausgangssignale des DDS-Funktionsgenerators bei einer Ausgangsfrequenz von 100 kHz und einer Spannungsvorgabe von 10 V

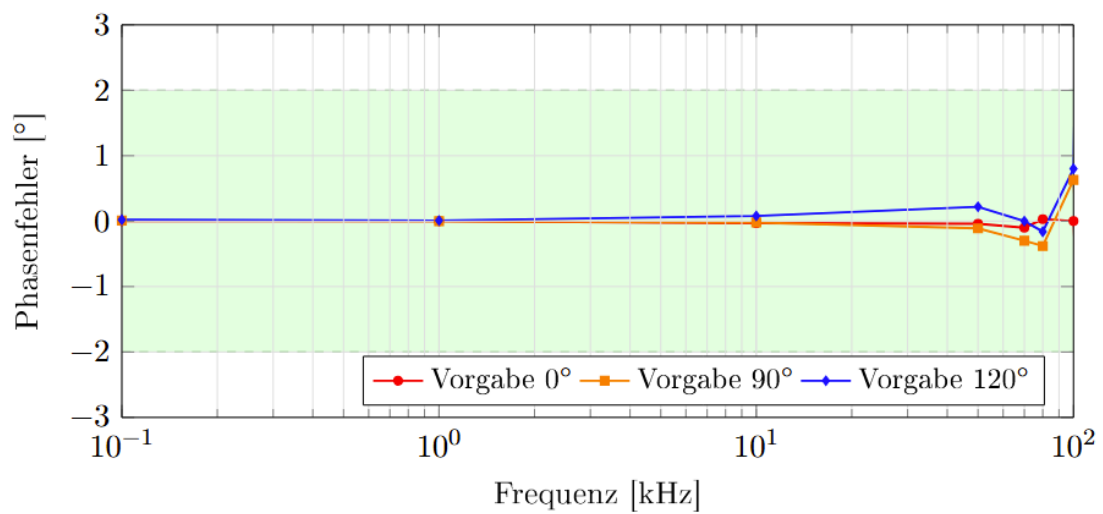


(b) Ausgangssignale des DDS-Funktionsgenerators bei einer Ausgangsfrequenz von 150 kHz und einer Spannungsvorgabe von 10 V

Abbildung 35: Ausgangssignale des DDS-Funktionsgenerators bei einer Ausgangsfrequenz von 100 kHz und 150 kHz unter der Spannungsvorgabe von 10 V



(a) Phasenfehler in Abhängigkeit aller getesteten Frequenzstufen (Sinus)



(b) Phasenfehler in Abhängigkeit aller getesteten Frequenzstufen (Rechteck)

Abbildung 36: Phasenfehler des DDS-Funktionsgenerators in Abhängigkeit aller getesteten Frequenzstufen bei einer Ausgangsspannung von 10 V

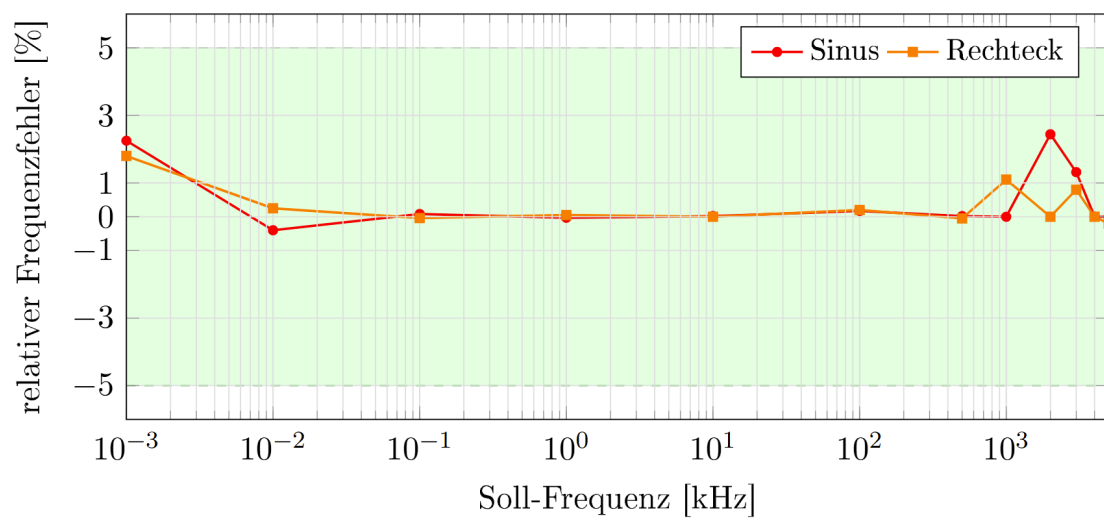
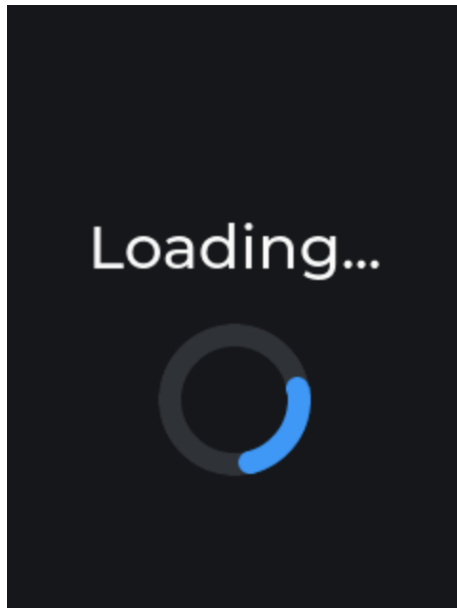


Abbildung 37: Frequenzfehler des DDS-Funktionsgenerators in Abhängigkeit aller getesteten Frequenzstufen bei Sinus- und Rechteckausgabe



A7 Benutzeroberfläche des HMIs

(a) Ladebildschirm, welcher beim Starten des HMIs angezeigt wird

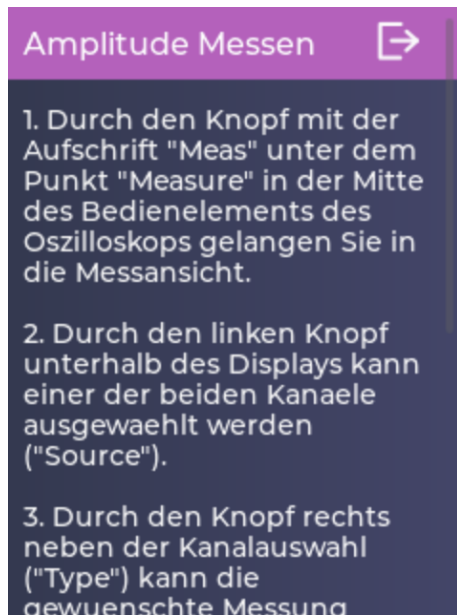


(b) Hauptseite, über welchen die Daten geladen, die Gruppen ausgewählt, die Ausgabe gestartet und gestoppt und die Hilfestellungen aufgerufen werden können

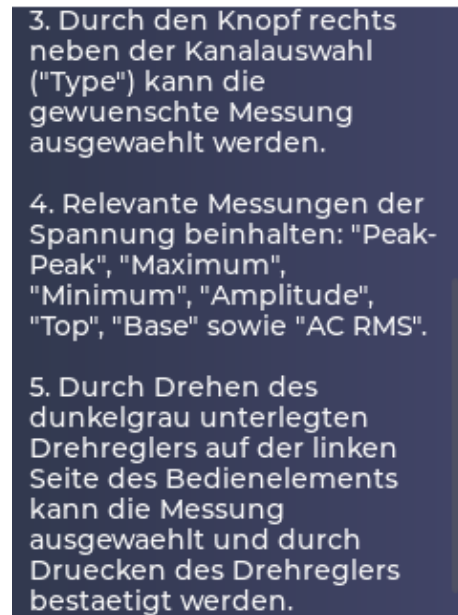


(c) Navigationsseite, über welche zwischen den Hilfestellungen gewechselt werden kann

Abbildung 39: Ladebildschirm, Hauptseite und Navigationsansicht der Benutzeroberfläche

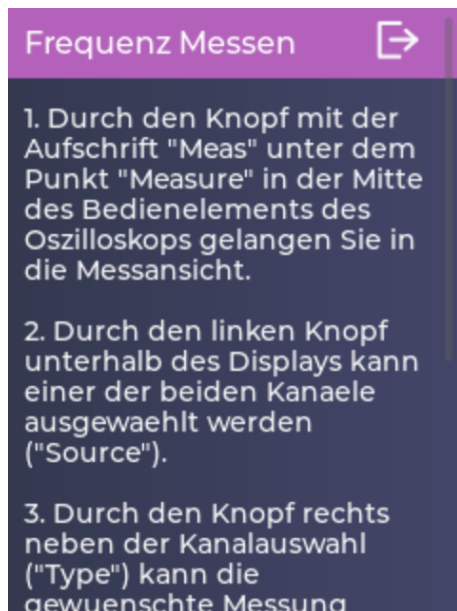


(a)

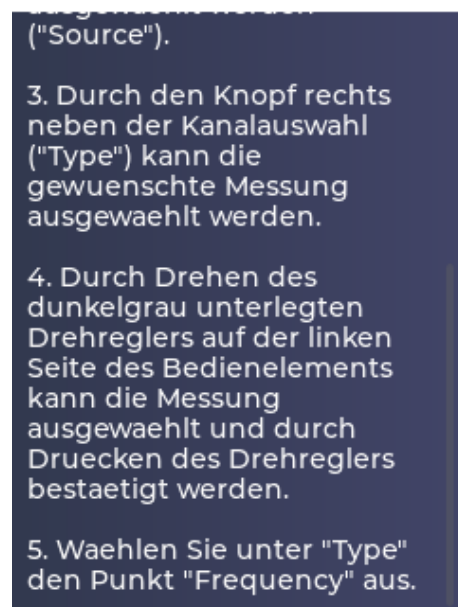


(b)

Abbildung 40: Hilfestellung für das Messen der Amplitude mittels des Oszilloskops

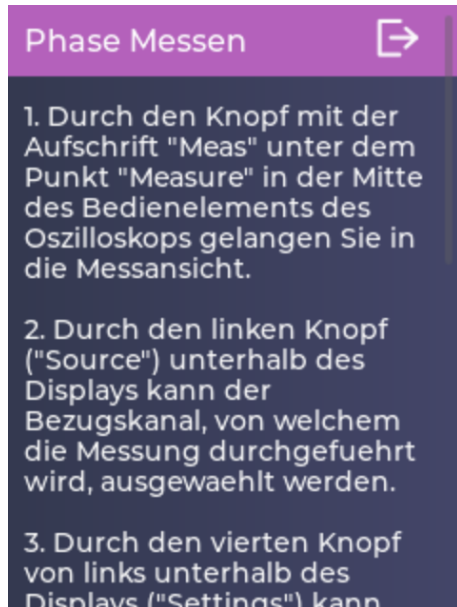


(a)

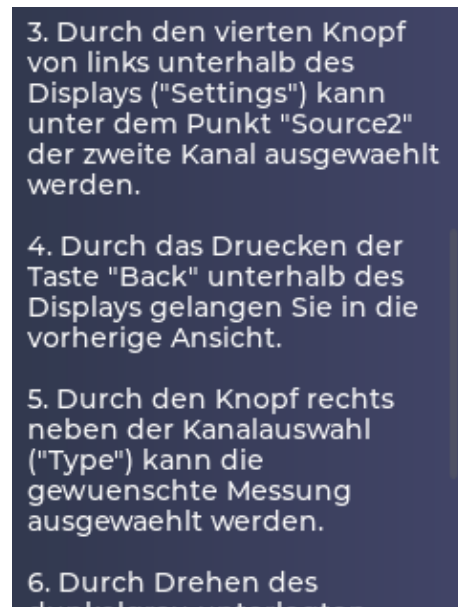


(b)

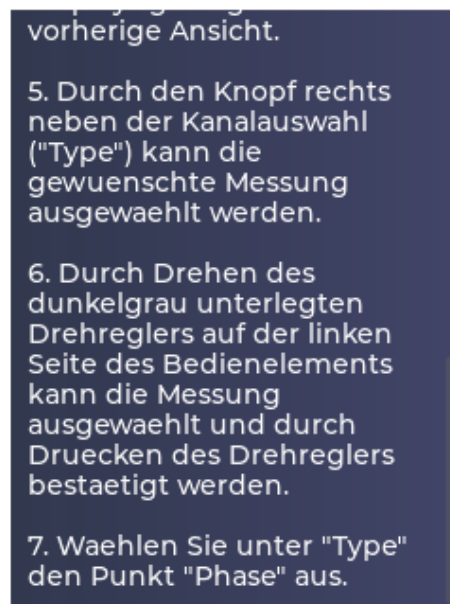
Abbildung 41: Hilfestellung für das Messen der Frequenz mittels des Oszilloskops



(a)

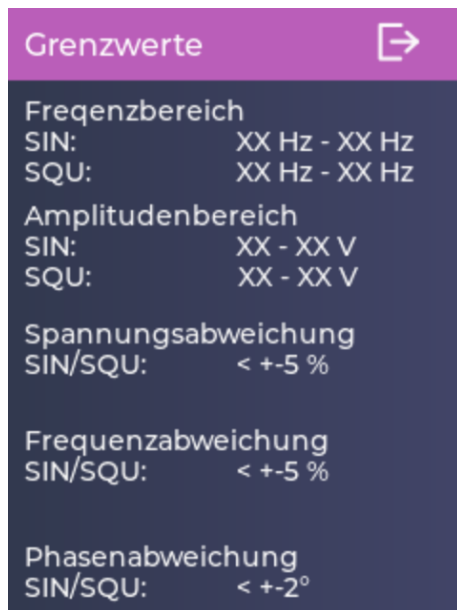


(b)

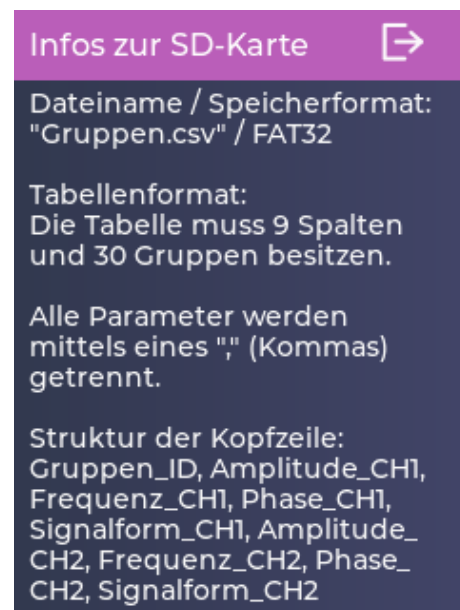


(c)

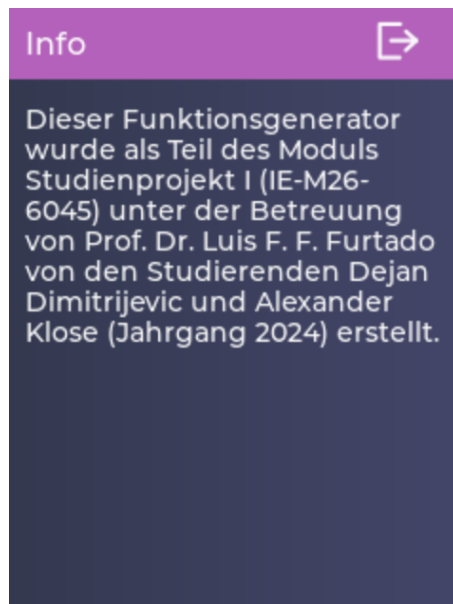
Abbildung 42: Hilfestellung für das Messen der Phasenverschiebung mittels des Oszilloskops



(a) Grenzwerte des Funktionsgenerators (X wird als Platzhalter verwendet)



(b) Information zur Formatierung der microSD-Karte, sowie Struktur der CSV-Datei

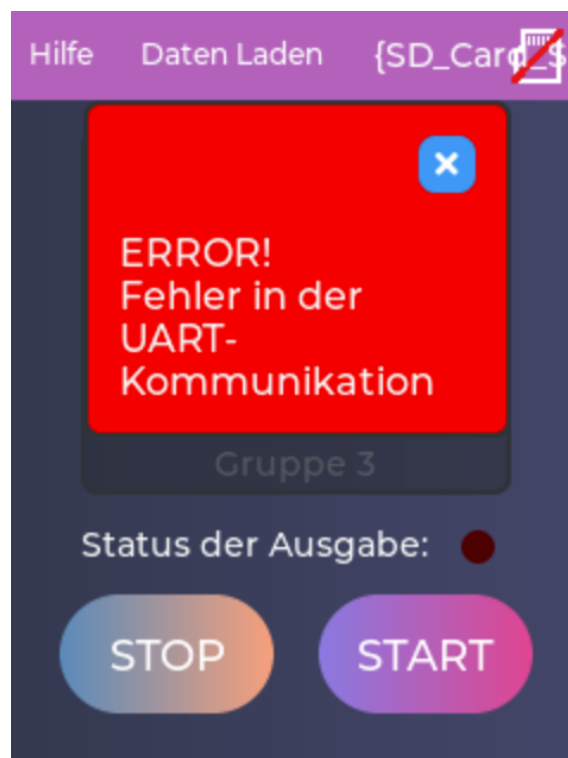


(c) Allgemeine Beschreibung des Funktionsgenerators

Abbildung 43: Seiten für die Übersicht der Grenzwerte, Informationen zur Formatierung der microSD-Karte und Allgemeine Beschreibung des Funktionsgenerators



(a) Fehlermeldung bei fehlender oder fehlerhafter CSV-Datei



(b) Fehlermeldung bei fehlerhafter UART-Kommunikation

Abbildung 44: Mögliche Fehlermeldungen, welche auf der Benutzeroberfläche angezeigt werden können

Listing 12: Inhalt der Funktionen action_start() (actions.cpp, Zeilen 90-110)

```
90  /**
91   * Wenn die Start-Taste gedrückt wird, soll die Status-LED auf volle Helligkeit
      eingestellt und ein Ton ausgegeben werden.
92   * Für die Entprellung der Tasten wird ein Timer verwendet, der die Zeit seit
      der letzten Betätigung der Taste misst. Wenn die Taste innerhalb der
      Entprellungszeit erneut gedrückt wird, wird die Aktion ignoriert.
93   */
94  extern "C" void action_start(lv_event_t* e) {
95      int64_t t_action = esp_timer_get_time();
96      uint32_t now = millis();
97      if (now - last_press_time < DEBOUNCE_MS) return;
98      last_press_time = now;
99      buzzer_sound();
100
101      uint8_t group = (uint8_t)get_var_active_group() + 1; // weil EEZ ab 0 zä
      hlt jede Gruppe +1
102      Serial.printf("Starte Gruppe %u\n", group);
103      lv_led_set_brightness(objects.status_led, 255);
104
105      if (!uart_group_start(group)) {
106          Serial.println("START-Befehl fehlgeschlagen");
107          lv_led_set_brightness(objects.status_led, 0);
108          create_error_uart();
109      }
110  }
```

Listing 13: Inhalt der Funktionen action_stop() (actions.cpp, Zeilen 112-124)

```
112  /**
113   * Wenn die Stop-Taste gedrückt wird, soll die Status-LED ausgeschaltet und ein
      Ton ausgegeben werden.
114   * Für die Entprellung der Tasten wird ein Timer verwendet, der die Zeit seit
      der letzten Betätigung der Taste misst. Wenn die Taste innerhalb der
      Entprellungszeit erneut gedrückt wird, wird die Aktion ignoriert.
115   */
116  extern "C" void action_stop(lv_event_t* e) {
117      uint32_t now = millis();
118      if (now - last_press_time < DEBOUNCE_MS) return;
119      last_press_time = now;
120      buzzer_sound();
121
122      lv_led_set_brightness(objects.status_led, 0);
123      uart_group_stop();
124  }
```

Listing 14: Inhalt der Funktionen action_load_data() (actions.cpp, Zeilen 126-155)

```
126  /**
127   * Wenn die Load Data-Taste gedrückt wird, soll ein Ton ausgegeben werden und
      die Daten geladen werden.
128   * Für die Entprellung der Tasten wird ein Timer verwendet, der die Zeit seit
      der letzten Betätigung der Taste misst. Wenn die Taste innerhalb der
      Entprellungszeit erneut gedrückt wird, wird die Aktion ignoriert.
129   */
130  extern "C" void action_load_data(lv_event_t* e) {
131      uint32_t now = millis();
132      if (now - last_press_time < DEBOUNCE_MS) return;
133      last_press_time = now;
134      buzzer_sound();
135
136      if (!csv_read(groups, group_count)) {
137          Serial.println("Fehler beim Laden der CSV");
138          create_error_data();
139          return;
140      }
141
142      Serial.println("CSV geladen, sende per UART...");
143      for (uint8_t i = 0; i < group_count; i++) {
144          csv_debug(groups[i]);
145      }
146
147      if (!uart_send_groups(groups, group_count)) {
148          Serial.println("UART-Übertragung fehlgeschlagen");
149          create_error_uart();
150          return;
151      }
152
153      Serial.println("Alle Gruppen erfolgreich übertragen");
154
155  }
```

Listing 15: Inhalt der Funktionen `action_help()` (actions.cpp, Zeilen 157-170)

```
157  /**
158   * Wenn die Help-Taste gedrückt wird, soll ein Ton ausgegeben werden und die
159   * Hilfsseite geladen werden.
160   * Für die Entprellung der Tasten wird ein Timer verwendet, der die Zeit seit
161   * der letzten Betätigung der Taste misst. Wenn die Taste innerhalb der
162   * Entprellungszeit erneut gedrückt wird, wird die Aktion ignoriert.
163   */
164  extern "C" void action_help(lv_event_t* e) {
165      // Überprüfen, ob die Taste innerhalb der Entprellungszeit erneut gedrückt
166      // wurde
167      uint32_t now = millis();
168      if (now - last_press_time < DEBOUNCE_MS) return;
169      last_press_time = now;
170      // Aktion ausführen: Ton ausgeben und Hilfsseite laden
171      buzzer_sound();
172      loadScreen(SCREEN_ID_HELP_OVERVIEW);
173  }
```

A8 Technische Zeichnung der Implementierungsmethode A (DAC)

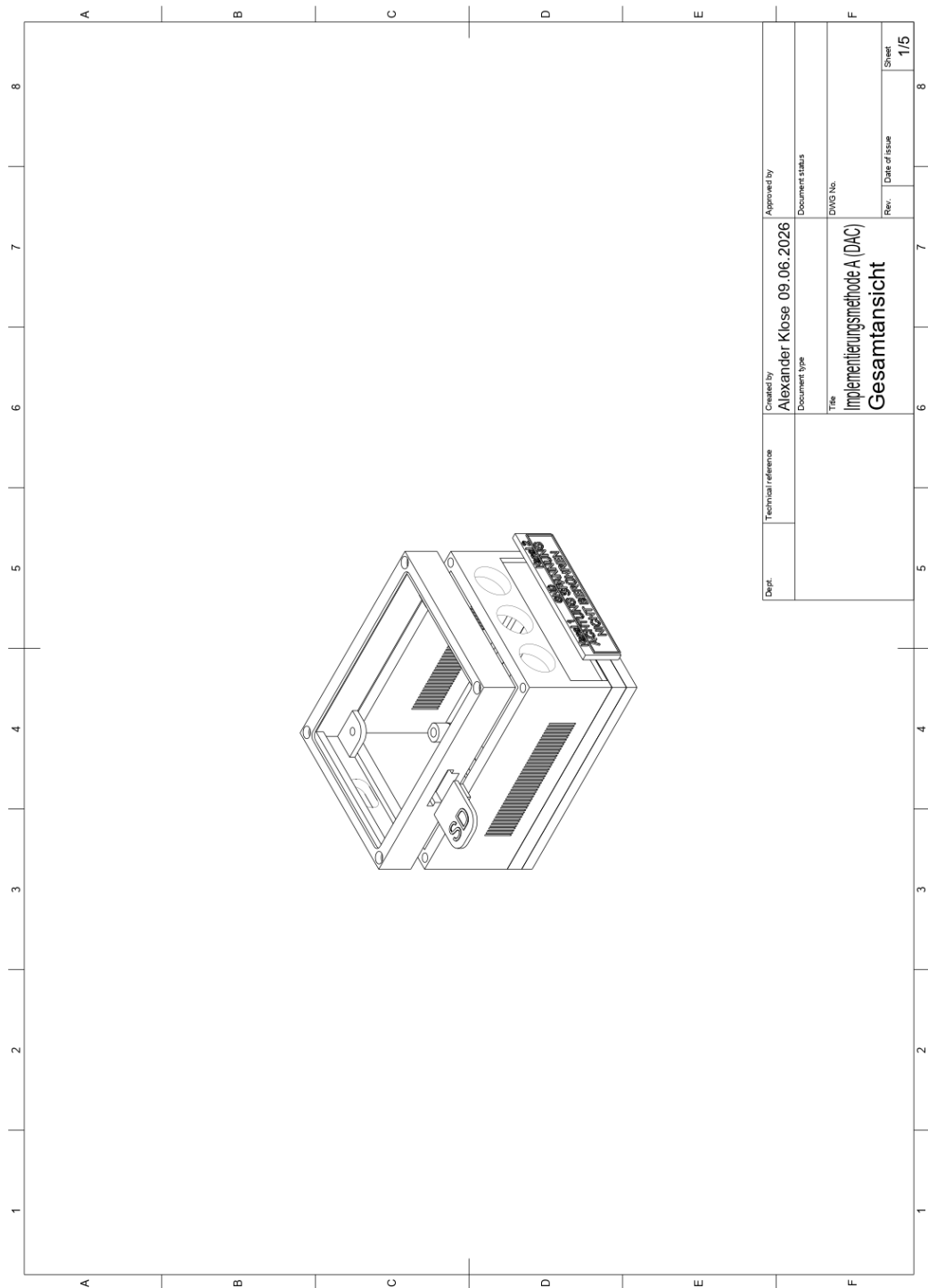


Abbildung 45: Technische Zeichnung Implementierungsmethode A (DAC) - Gesamtansicht

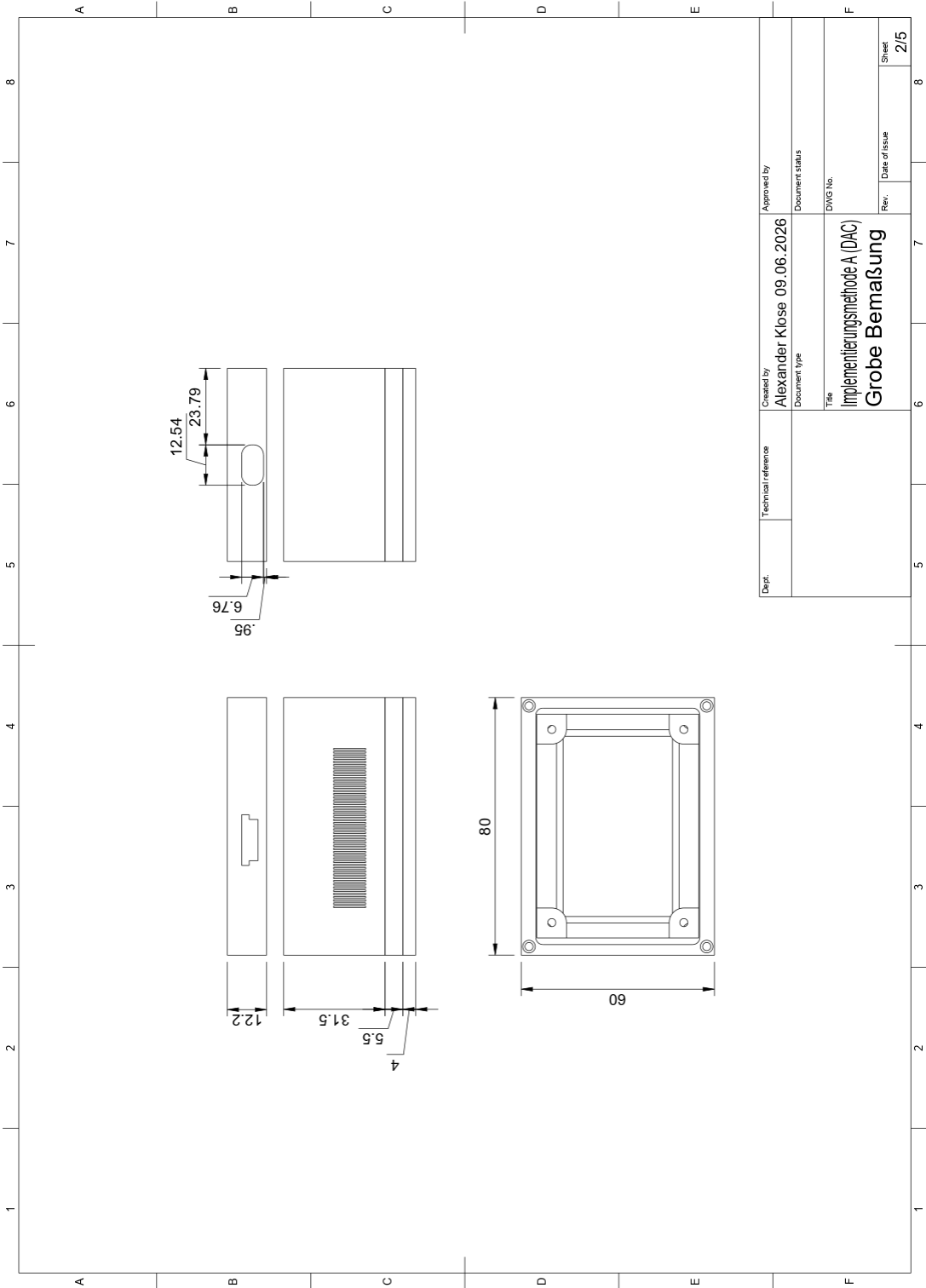


Abbildung 46: Technische Zeichnung Implementierungsmethode A (DAC) - grobe Bemessung



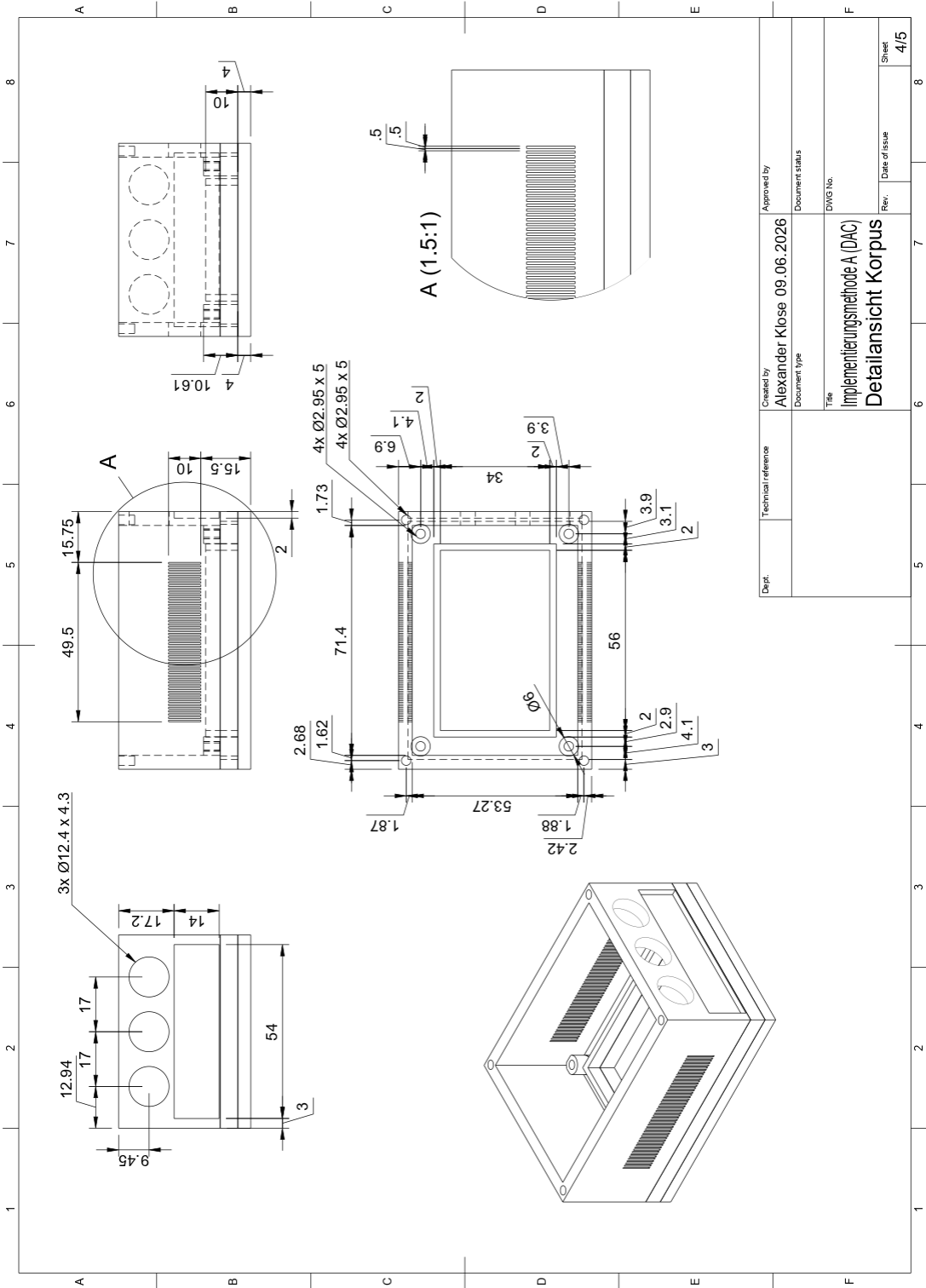


Abbildung 48: Technische Zeichnung Implementierungsmethode A (DAC) - Detailansicht Korpus

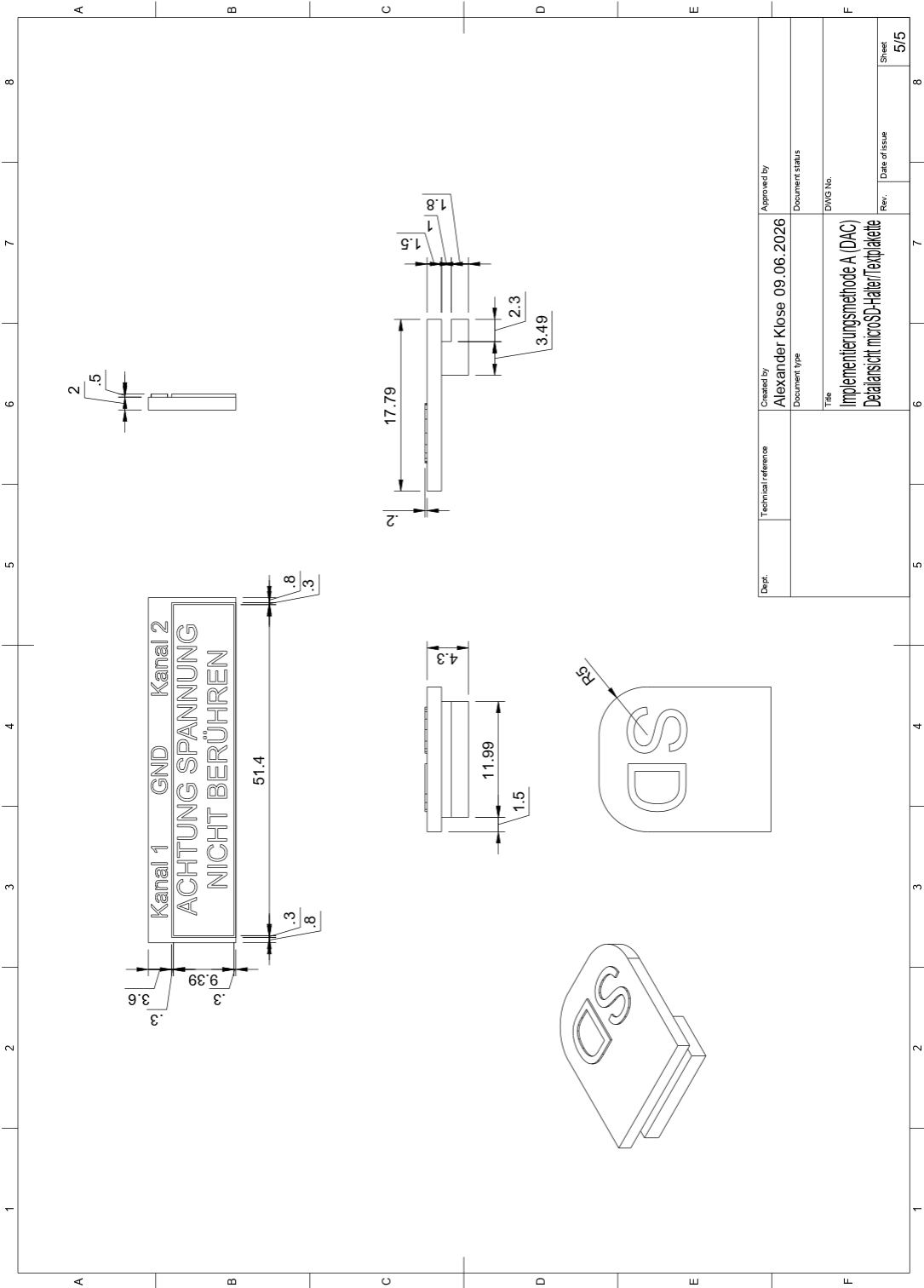


Abbildung 49: Technische Zeichnung Implementierungsmethode A (DAC) - Detailansicht microSD-Halter und Textplakette

A9 Technische Zeichnung der Implementierungsmethode B (DDS)

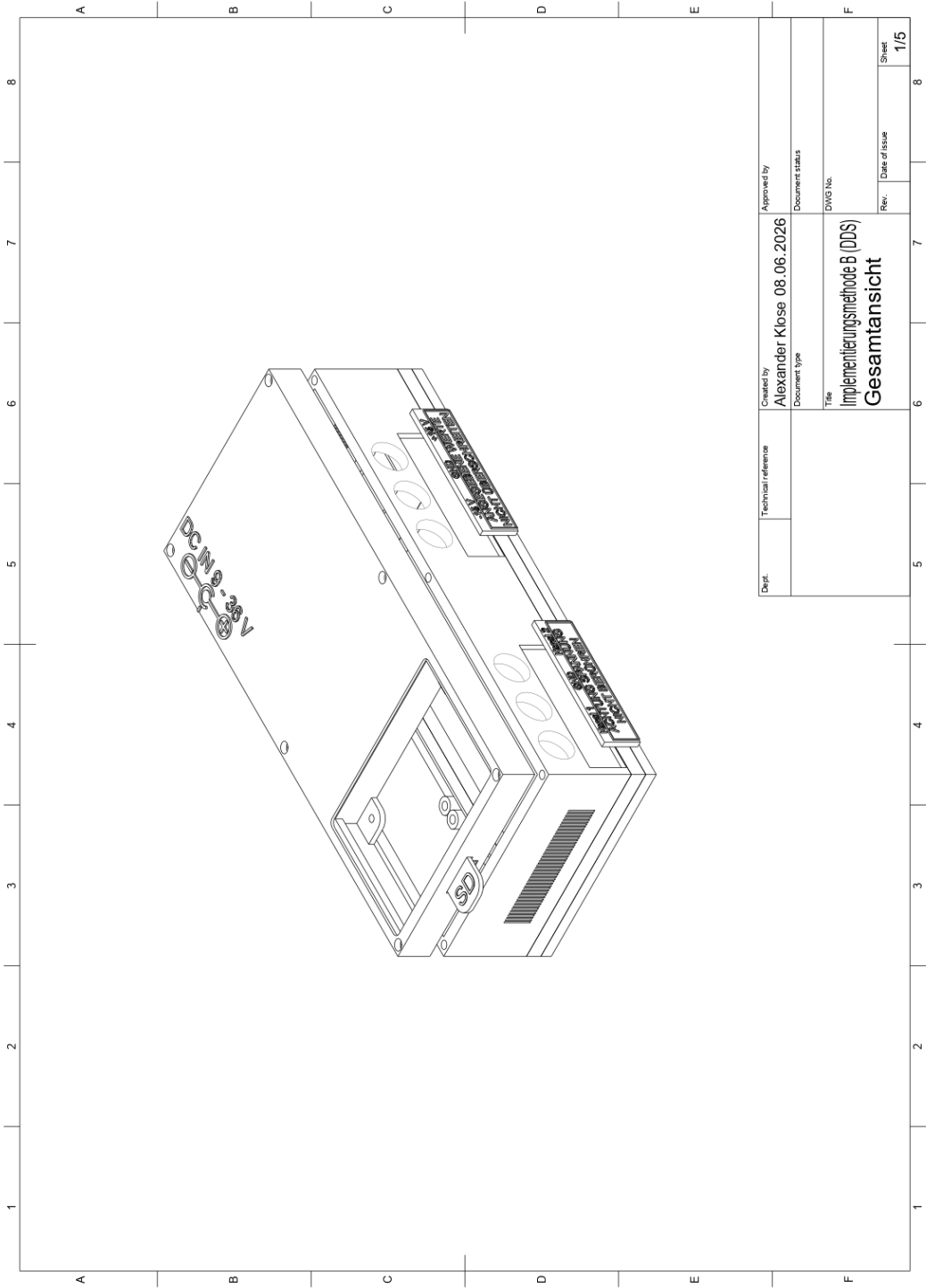


Abbildung 50: Technische Zeichnung Implementierungsmethode B (DDS) - Gesamtansicht

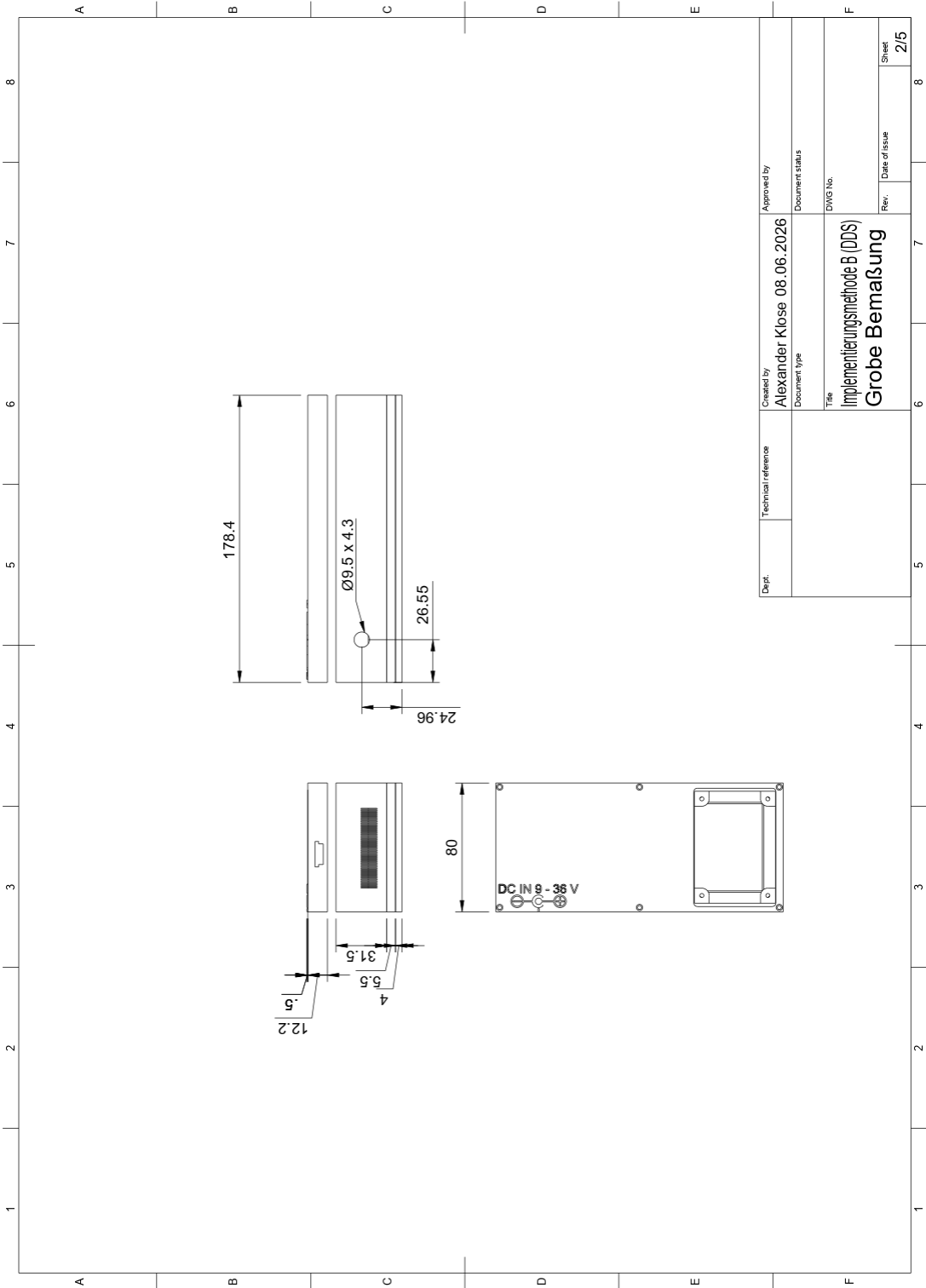


Abbildung 51: Technische Zeichnung Implementierungsmethode B (DDS) - grobe Bemäßung





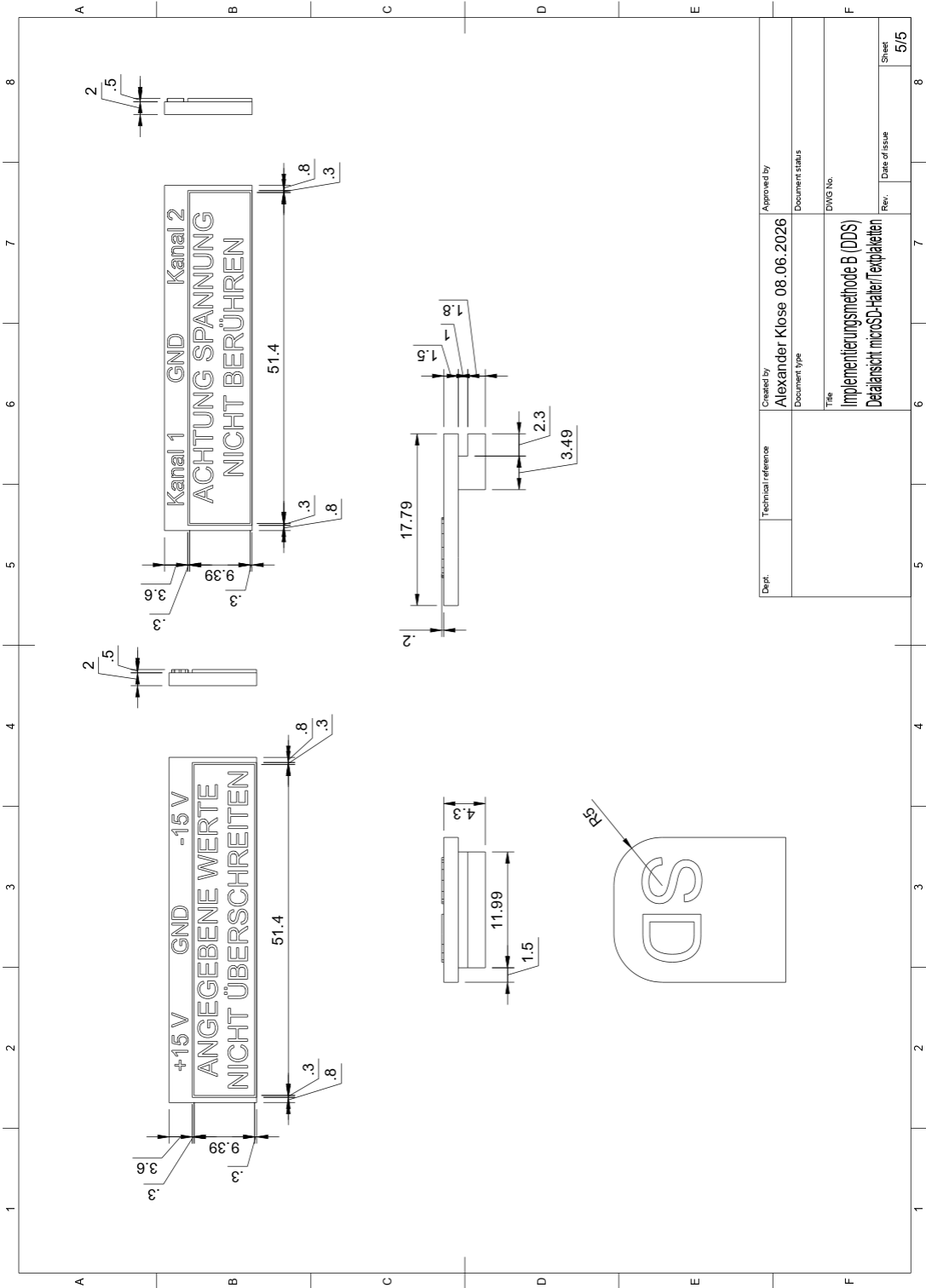


Abbildung 54: Technische Zeichnung Implementierungsmethode B (DDS) - Detailsicht microSD-Halter und Textplakette

A10 Kostenübersicht der Implementierungsmethoden A und B

Tabelle 17: Kostenübersicht der beiden Implementierungsmethoden A und B

Implementierungsmethode A: DAC	
Komponente	Kosten
ESP32-S3 2.8inch Display Development Board	25,50 €
ESP32 DevKit C V4	12,99 €
3 Bananenbuchsen	1,50 €
ca. 20 cm WS2812B LED Streifen	1,30 €
73,5 g Filament (PLA)	1,47 €
Buzzer	0,40 €
Schrauben und Gewindeeinsätze	0,37 €
Widerstand (330 Ω)	0,07 €
2 Kondensatoren (100 nF)	0,08 €
Lochplatine	0,33 €
Gesamtkosten	44,01 €
Implementierungsmethode B: DDS	
ESP32-S3 2.8inch Display Development Board	25,50 €
ESP32 DevKit C V4	12,99 €
DC-DC Buck Converter	2,20 €
9 V DC Netzteil	7,98 €
2 AD9833	18,78 €
2 MCP42010	12,45 €
OPA1612	4,59 €
6 Bananenbuchsen	3,00 €
ca. 43 cm WS2812B LED Streifen	2,74 €
183,7 g Filament (PLA)	3,68 €
Schrauben und Gewindeeinsätze	0,78 €
2 DIP IC Sockel	0,12 €
11 Widerstände	0,77 €
4 Kondensatoren (100 nF)	0,16 €
4 Dioden	0,16 €
Buzzer	0,40 €
3 Lochplatten	0,99 €
Gesamtkosten	97,29 €